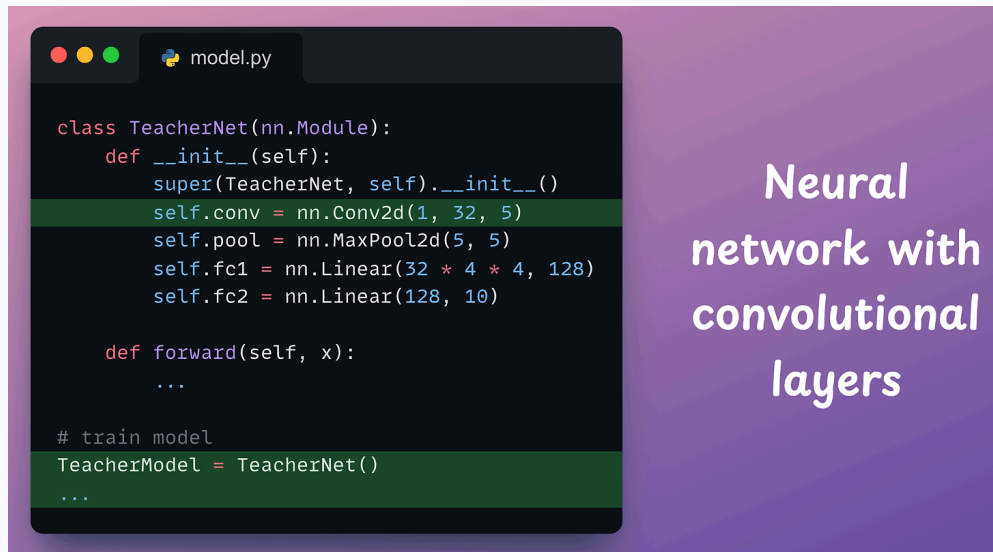


Knowledge distillation demo

In the interest of time, let's say we have already trained the following CNN model on the MNIST dataset (I have provided the full Jupyter notebook towards the end, don't worry):



The epoch-by-epoch training loss and validation accuracy is depicted below:

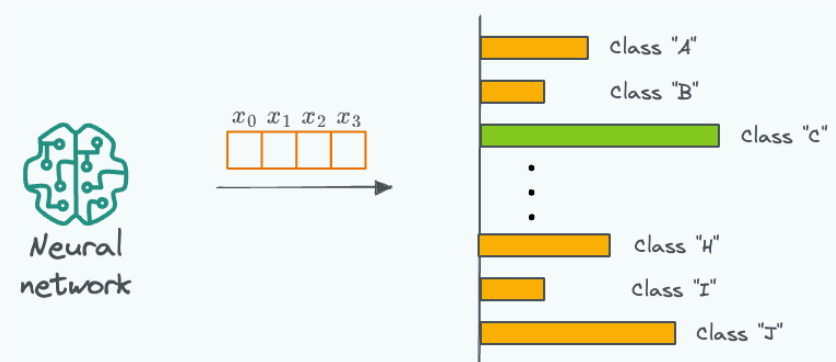
The image shows a table with 5 rows and 3 columns. The first column lists epochs from 1 to 5. The second column shows the loss, and the third column shows the accuracy percentage. The text is displayed in a white, sans-serif font on a purple gradient background.

Epoch 1,	Loss: 0.20,	Acc: 98.79%
Epoch 2,	Loss: 0.17,	Acc: 98.91%
Epoch 3,	Loss: 0.15,	Acc: 98.78%
Epoch 4,	Loss: 0.13,	Acc: 98.89%
Epoch 5,	Loss: 0.11,	Acc: 98.63%

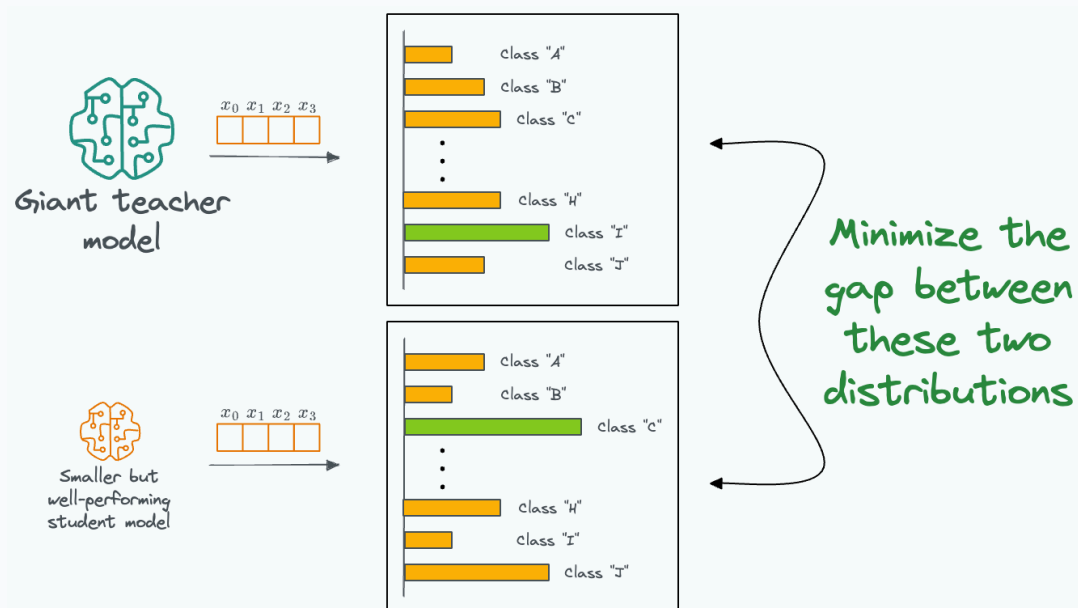
Next, let's define a simpler model without any convolutional layers:



Being a classification model, the output will be a probability distribution over the $<N>$ classes:



Thus, we can train the student model such that its probability distribution matches that of the teacher model.



One way to do this is to use KL divergence as a loss function.

$$D_{KL}(P \parallel Q) = \sum_x P(x) \cdot \log \left(\frac{P(x)}{Q(x)} \right)$$

It measures how much information is lost when we use distribution Q to approximate distribution P.

A question for you: What will be the KL divergence if $P=Q$?

Thus, in our case:

- P will be the probability distribution from the teacher model.
- Q will be the probability distribution from the student model.

The loss function is implemented below:

```
def KL_loss(student_logits, teacher_logits):  
  
    # convert teacher model outputs to probabilities  
    p_teacher = F.softmax(teacher_logits, dim=1)  
  
    # convert student model outputs to probabilities  
    p_student = F.log_softmax(student_logits, dim=1)  
  
    # compute KL divergence loss (PyTorch's method)  
    return F.kl_div(p_student, p_teacher, reduction='batchmean')
```

**KL divergence
loss function**

Finally, we train the student model as follows:

```
for epoch in range(5):  
    ...  
  
    student_optimizer.zero_grad()  
  
    # student outputs  
    student_logits = student_model(inputs)  
  
    # teacher outputs (detach to avoid backpropagation)  
    teacher_logits = teacher_model(inputs).detach()  
  
    # KL Divergence loss  
    loss = KL_loss(student_logits, teacher_logits)  
  
    # backpropagation step  
    loss.backward()  
    student_optimizer.step()
```

**Student
model
training loop**

Done!

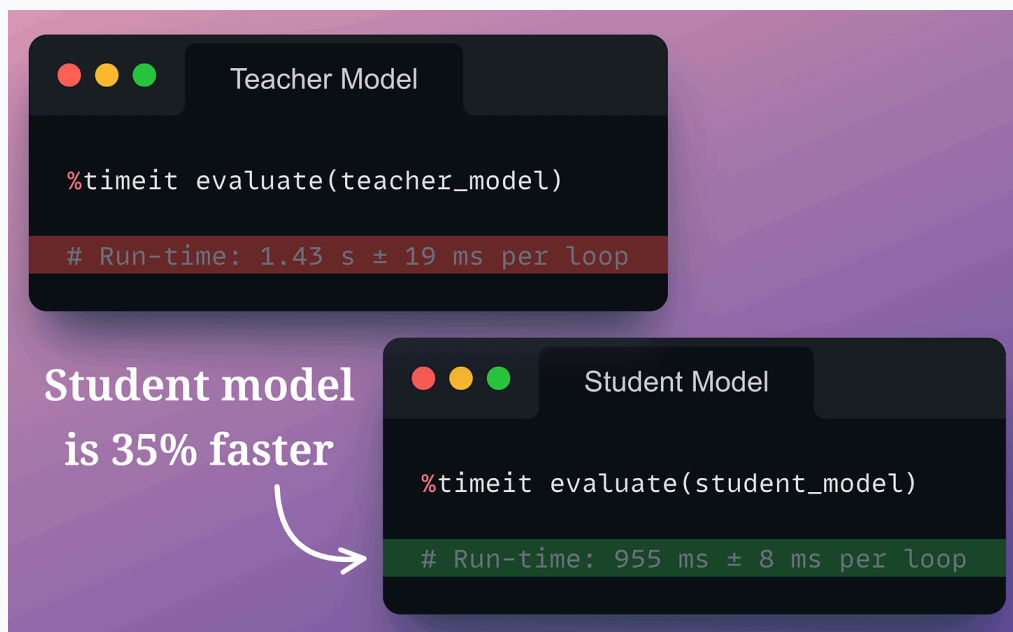
The following image compares the training loss and validation accuracy of the two models:

Teacher Model			Student Model		
Epoch 1,	Loss: 0.20,	Accuracy: 98.79%	Epoch 1,	Loss: 2.12,	Accuracy: 93.17%
Epoch 2,	Loss: 0.17,	Accuracy: 98.91%	Epoch 2,	Loss: 1.05,	Accuracy: 95.65%
Epoch 3,	Loss: 0.15,	Accuracy: 98.78%	Epoch 3,	Loss: 0.74,	Accuracy: 95.65%
Epoch 4,	Loss: 0.13,	Accuracy: 98.89%	Epoch 4,	Loss: 0.59,	Accuracy: 96.47%
Epoch 5,	Loss: 0.11,	Accuracy: 98.63%	Epoch 5,	Loss: 0.48,	Accuracy: 96.54%

Of course, as shown in the highlighted lines above, the performance of the student model is not as good as the teacher model, which is expected.

However, it is still pretty promising, given that it was only composed of simple feed-forward layers.

Also, as depicted below, the student model is approximately 35% faster than the teacher model, which is a significant increase in the inference run-time of the model for about a 1-2% drop in the test accuracy.



That said, one of the biggest downsides of knowledge distillation is that one must still train a larger teacher model first to train the student model.

But in a resource-constrained environment, it may not be feasible to train a large teacher model.

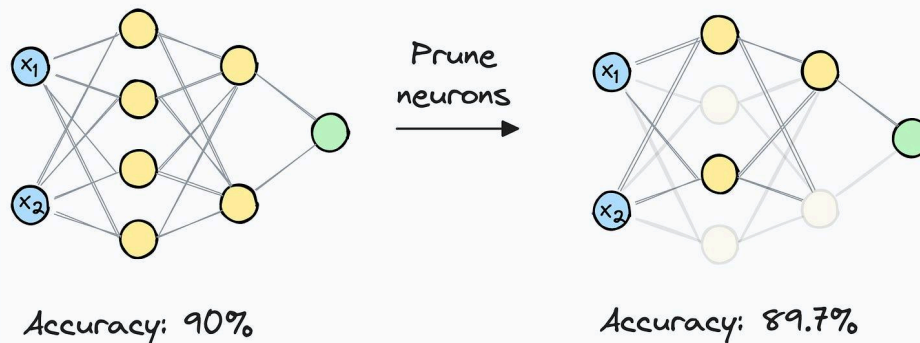
So this technique assumes that we are not resource-constrained at least in the development environment.

In the next chapter, let's discuss one more technique to compress ML models and reduce their memory footprint.

Activation Pruning

Once we complete network training, we are almost always left with plenty of useless neurons — ones that make nearly zero contribution to the network's performance, but they still consume memory.

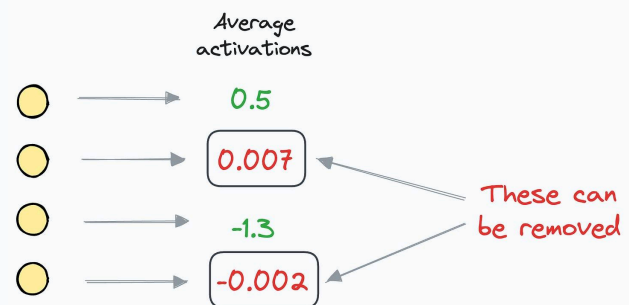
In other words, there is a high percentage of neurons, which, if removed from the trained network, will not affect the performance remarkably:



And, of course, I am not saying this as a random and uninformed thought. I have experimentally verified this over and over across my projects.

Here's the core idea.

After training is complete, we run the dataset through the model (no backpropagation this time) and analyze the average activation of individual neurons. Here, we often observe that many neuron activations are always close to near-zero values.



Thus, they can be pruned from the network, as they will have very little impact on the model's output.

For pruning, we can decide on a pruning threshold (λ) and prune all neurons whose activations are less than this threshold.

This makes intuitive sense as well.

More specifically, if a neuron rarely possesses a high activation value, then it is fair to assume that it isn't contributing to the model's output, and we can safely prune it.

The following table compares the accuracy of the pruned model with the original (full) model across a range of pruning thresholds (λ):

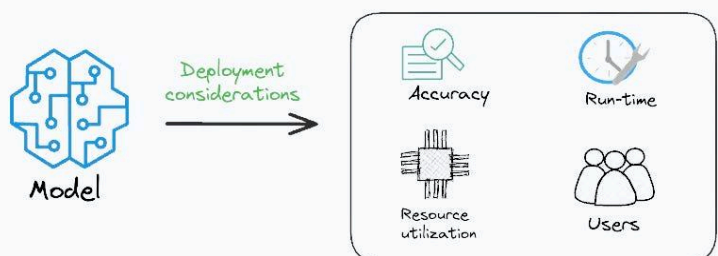
	Pruning Threshold (λ)	Validation Accuracy	Parameter Reduction
Original Model	-	96.7	-
Pruned Models	0.1	96.62	62%
	0.2	96.41	66%
	0.3	96.31	69%
	0.4	96.08	72%
	0.5	94.88	74%
	0.6	94.49	76%
	0.7	92.19	79%
	0.8	90.65	81%
	0.9	88.39	83%
	1	87.87	84%

At a pruning threshold $\lambda=0.4$, the validation accuracy of the model drops by just 0.62%, but the number of parameters drops by 72%.

That is a huge reduction, while both models being almost equally good! Of course, there is a trade-off because we are not doing as well as the original model. But in many cases, especially when deploying ML models, accuracy is not the only primary metric that decides these.

Instead, several operational metrics like efficiency, speed, memory consumption, etc., are also a key deciding factor.

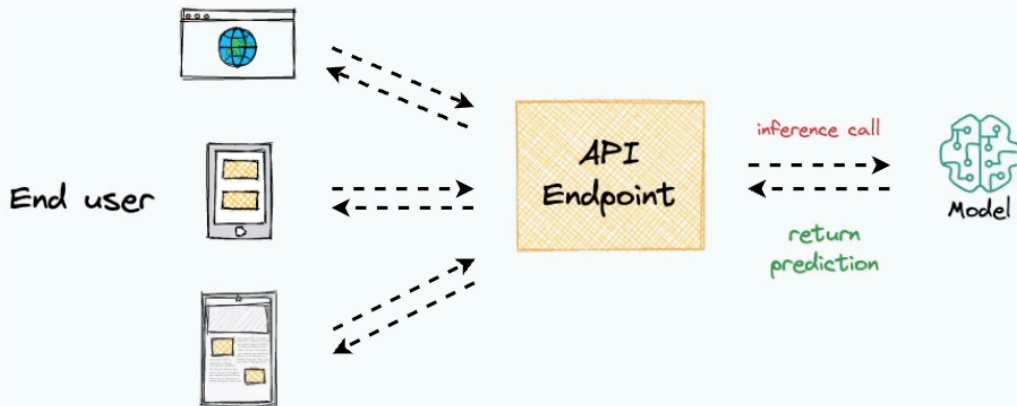
That is why model compression techniques are so crucial in such cases.



Deployment

Deploy ML Models from Jupyter Notebook

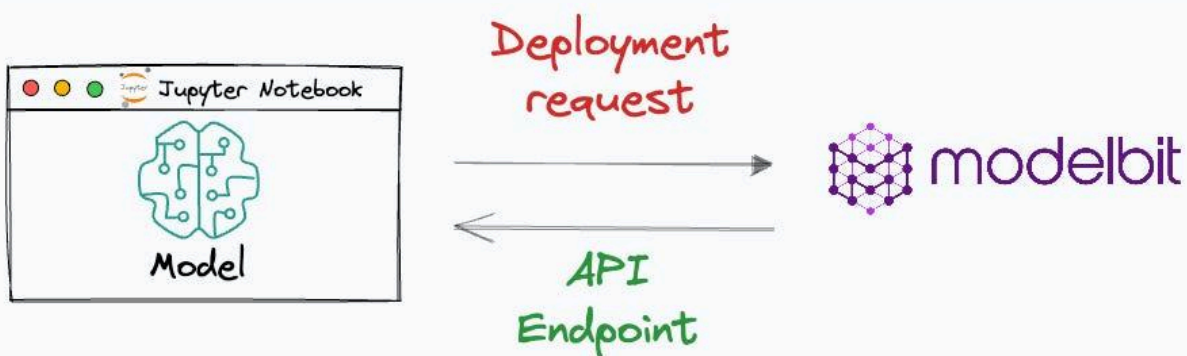
The core objective of model deployment is to obtain an API endpoint that can be used for inference purposes:



While this sounds simple, deployment is typically quite a tedious and time-consuming process. One must maintain environment files, configure various settings, ensure all dependencies are correctly installed, and many more.

So, in this chapter, I want to help you simplify this process. More specifically, we shall learn how to deploy any ML model right from a Jupyter Notebook in just three simple steps using the Modelbit API.

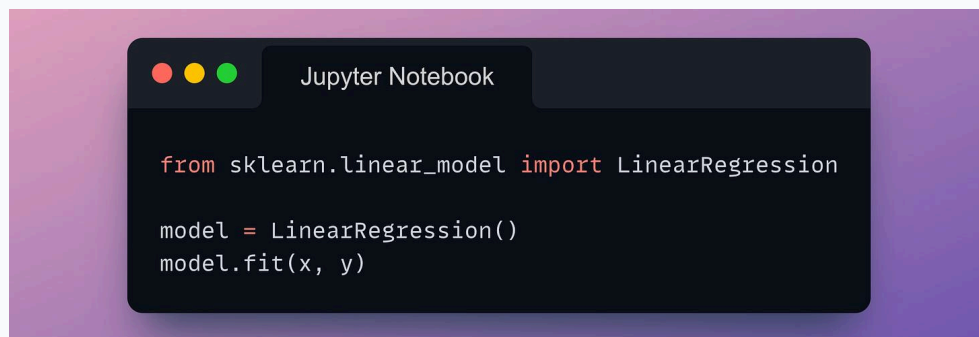
Modelbit lets us seamlessly deploy ML models directly from our Python notebooks (or git) to Snowflake, Redshift, and REST.



Deployment with Modelbit

Assume we have already trained our model.

For simplicity, let's assume it to be a linear regression model trained using sklearn, but it can be any other model as well:



```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(x, y)
```

Let's see how we can deploy this model with Modelbit!

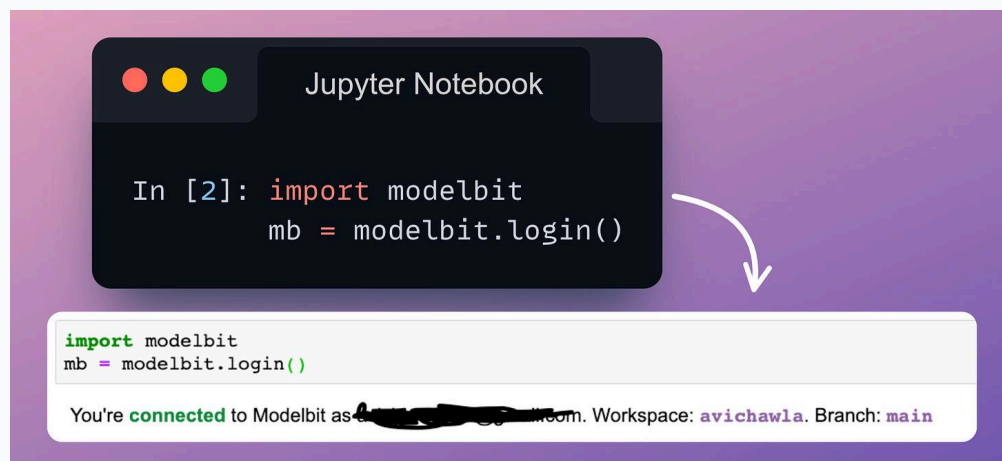
- First, we install the Modelbit package via pip:



```
$ pip install modelbit
```

```
In [1]: !pip install modelbit
```

- Next, we log in to Modelbit from our Jupyter Notebook (make sure you have created an account here: [Modelbit](https://modelbit.com))



```
In [2]: import modelbit
        mb = modelbit.login()
```

You're **connected** to Modelbit as **avichawla** on **modelbit.com**. Workspace: **avichawla**. Branch: **main**

- Finally, we deploy it, but here's an important point to note:

To deploy a model using Modelbit, we must define an inference function.

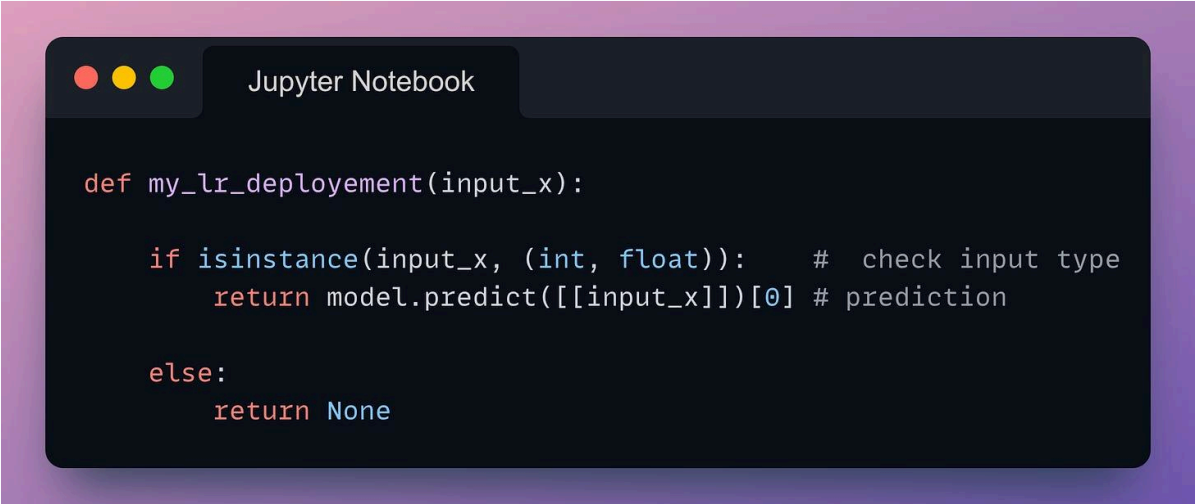
Simply put, this function contains the code that will be executed at inference. Thus, it will be responsible for returning the prediction.

```
def inference_function():  
    # prediction code
```

A function that
returns model
predictions

We must specify the input parameters required by the model in this method. Also, we can name it anything we want.

For our linear regression case, the inference function can be as follows:

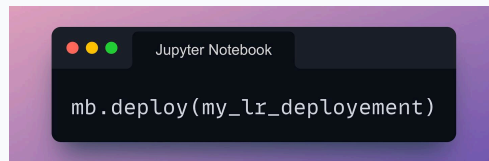


```
def my_lr_deployment(input_x):  
  
    if isinstance(input_x, (int, float)): # check input type  
        return model.predict([[input_x]])[0] # prediction  
  
    else:  
        return None
```

- We define a function `my_lr_deployment()`.
- Next, we specify the input of the model as a parameter of this method.
- We validate the input for its data type.
- Finally, we return the prediction.

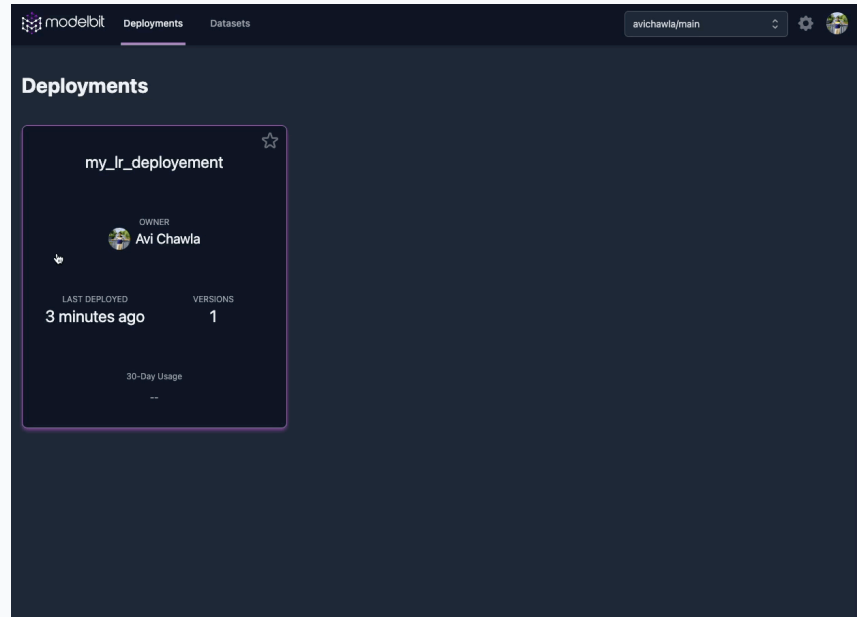
One good thing about Modelbit is that every dependency of the function (the model object in this case) is pickled and sent to production automatically along

with the function. Thus, we can reference any object in this method. Once we have defined the function, we can proceed with deployment as follows:

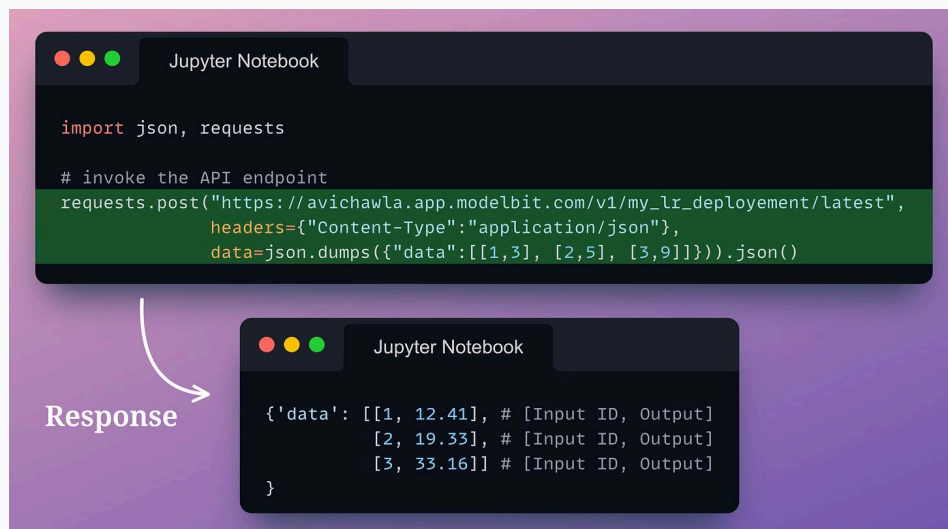


```
mb.deploy(my_lr_deployment)
```

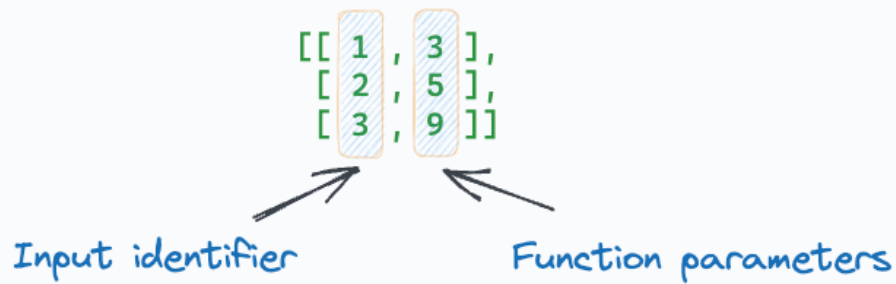
We have successfully deployed the model in three simple steps, that too, right from the Jupyter Notebook! Once our model has been successfully deployed, it will appear in our Modelbit dashboard.



As shown above, Modelbit provides an API endpoint. We can use it for inference purposes as follows:



In the above request, data passed to the endpoint is a list of lists.



The first number in the list is the input ID. All entries following the ID in a list are the function parameters.

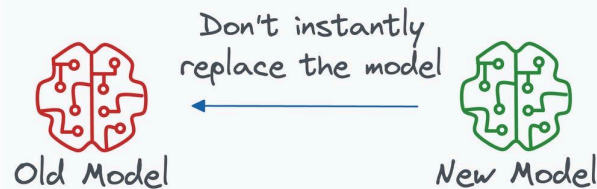
Lastly, we can also specify specific versions of the libraries or Python used while deploying our model. This is depicted below:

```
mb.deploy(my_lr_deployment,  
python_packages = ["scikit-learn=1.1.2", "pandas=1.5.0"],  
python_version = "3.9")
```

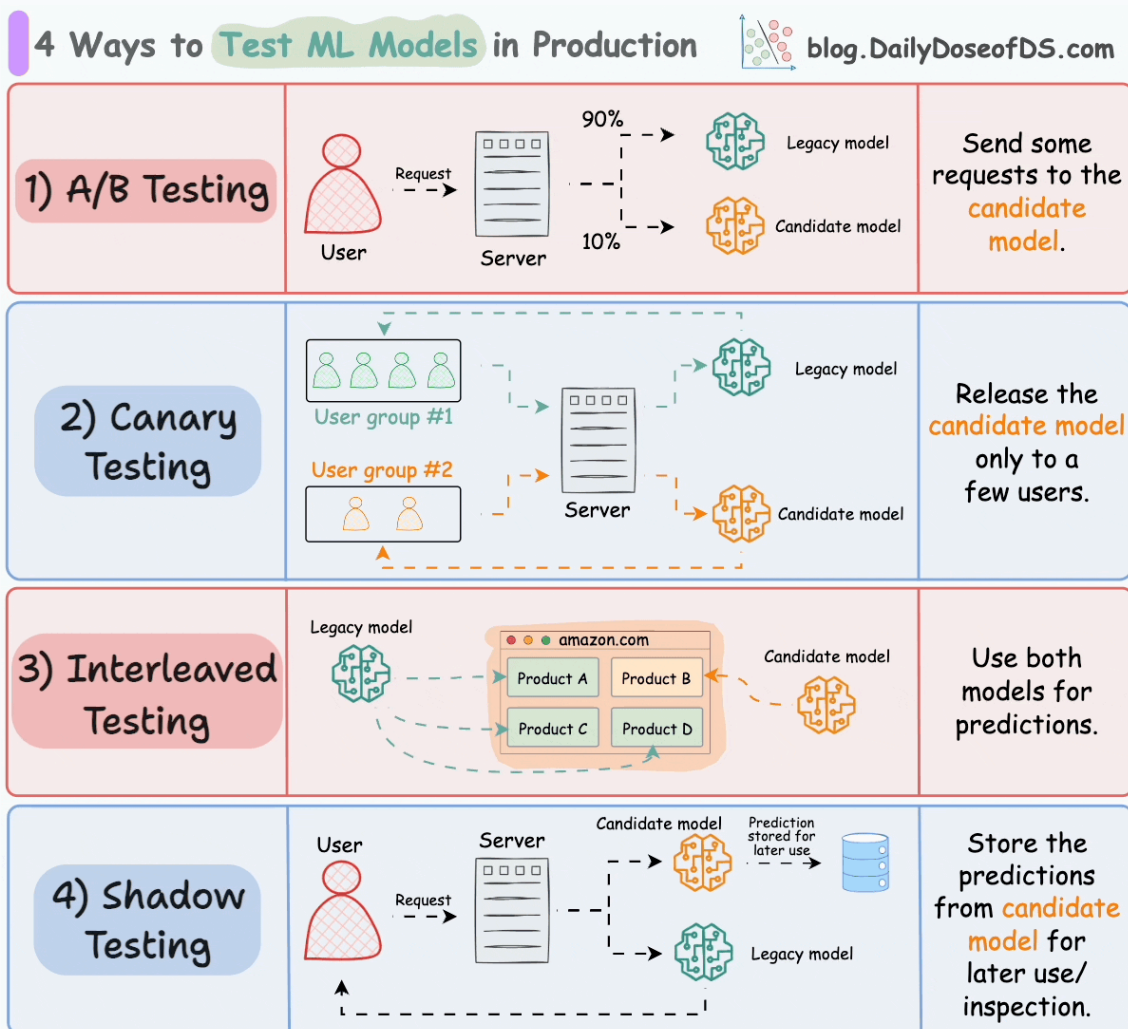
Isn't that cool, simple, and elegant over traditional deployment approaches?

4 Ways to Test ML Models in Production

Despite rigorously testing an ML model locally (on validation and test sets), it could be a terrible idea to instantly replace the previous model with a new model.

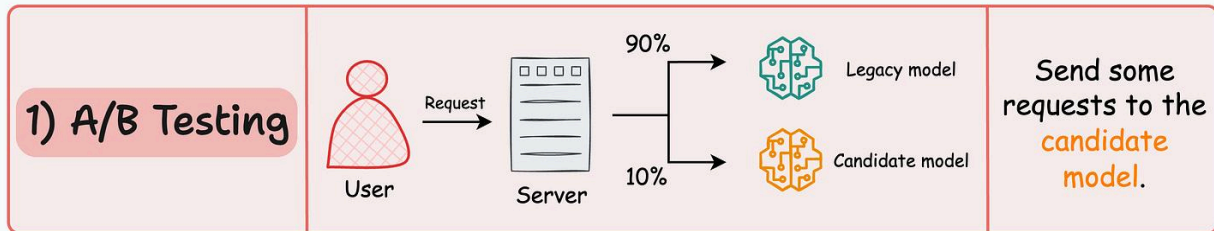


A more reliable strategy is to test the model in production (yes, on real-world incoming data). While this might sound risky, ML teams do it all the time, and it isn't that complicated. The following visual depicts 4 common strategies to do so:



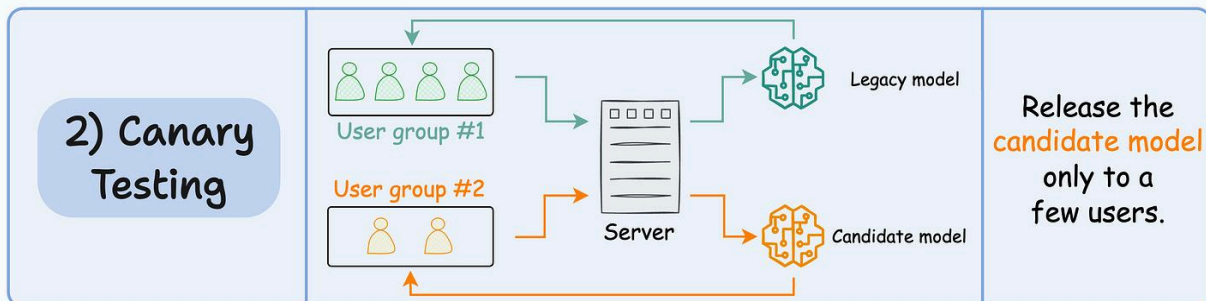
- The current model is called the legacy model.
- The new model is called the candidate model.

#1) A/B testing



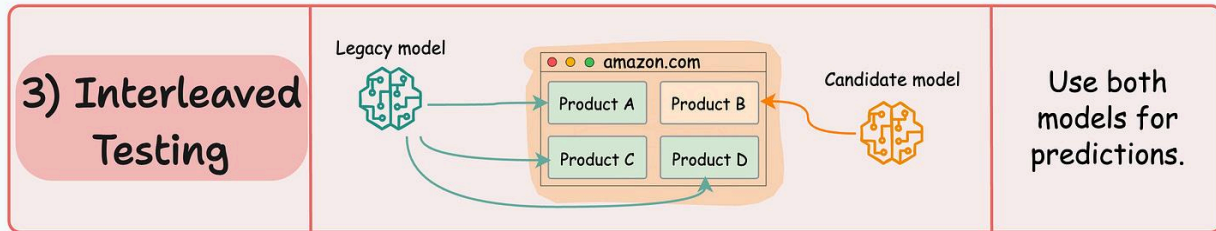
- Distribute the incoming requests non-uniformly between the legacy model and the candidate model.
- Intentionally limit the exposure of the candidate model to avoid any potential risks. Thus, the number of requests sent to the candidate model must be low.

#2) Canary testing



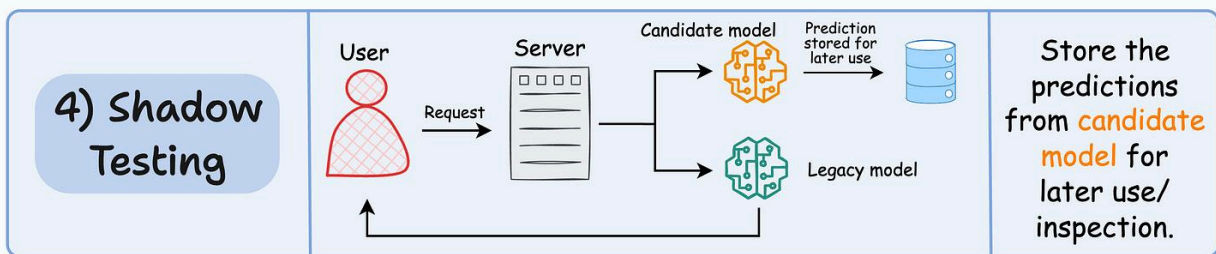
- In A/B testing, since traffic is randomly redirected to either model irrespective of the user, it can potentially affect all users.
- In canary testing, the candidate model is released to a small subset of users in production and gradually rolled out to more users.

#3) Interleaved testing



- This involves mixing the predictions of multiple models in the response.
- Consider Amazon's recommendation engine. In interleaved deployments, some product recommendations displayed on the homepage can come from the legacy model, while some can come from the candidate model.

#4) Shadow testing



- All of the above techniques affect some (or all) users.
- Shadow testing (or dark launches) lets us test a new model in a production environment without affecting the user experience.
- The candidate model is deployed alongside the existing legacy model and serves requests like the legacy model. However, the output is not sent back to the user. Instead, the output is logged for later use to benchmark its performance against the legacy model.
- We explicitly deploy the candidate model instead of testing offline because the production environment is difficult to replicate offline.

Shadow testing offers risk-free testing of the candidate model in a production environment.

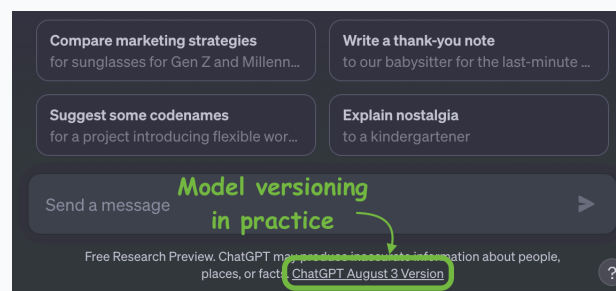
Version Controlling and Model Registry

Real-world ML deployment is never just about “deployment” — host the model somewhere, obtain an API endpoint, integrate it into the application, and you are done!

This is because, in reality, plenty of things must be done post-deployment to ensure the model’s reliability and performance.

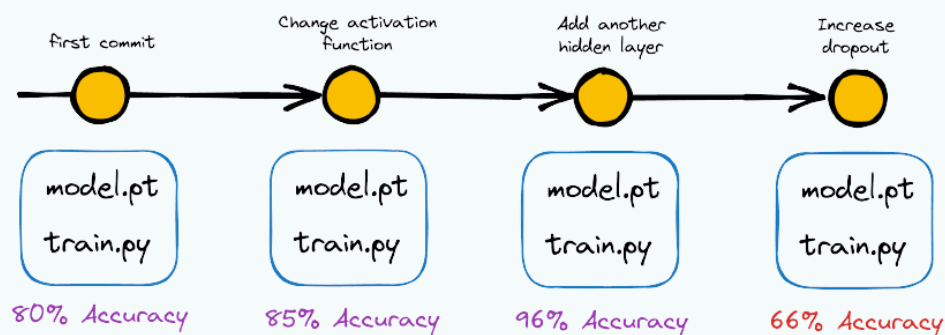
#1) Version control

To begin, it is immensely crucial to version control ML deployments. You may have noticed this while using ChatGPT, for instance.



But updating does not simply mean overwriting the previous version.

Instead, ML models are always version-controlled (using git tools).



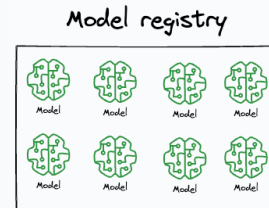
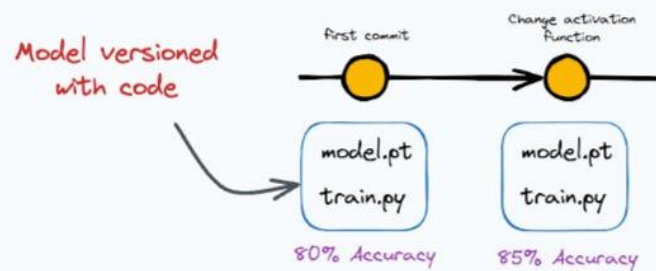
The advantages of version-controlling ML deployments are pretty obvious:

- In case of sudden mishaps post-deployment, we can instantly roll back to an older version.
- We can facilitate parallel development with branching, and many more.

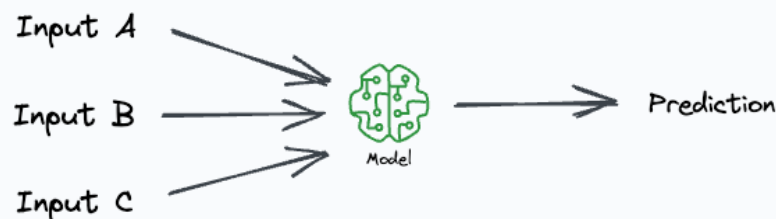
#2) Model registry

Another practical idea is to maintain a model registry for deployments. Let's understand what it is.

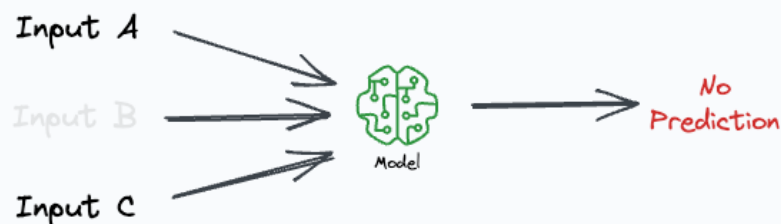
Simply put, a model registry can be considered repository of models. See, typically, we might be inclined to version our code and the ML model together:



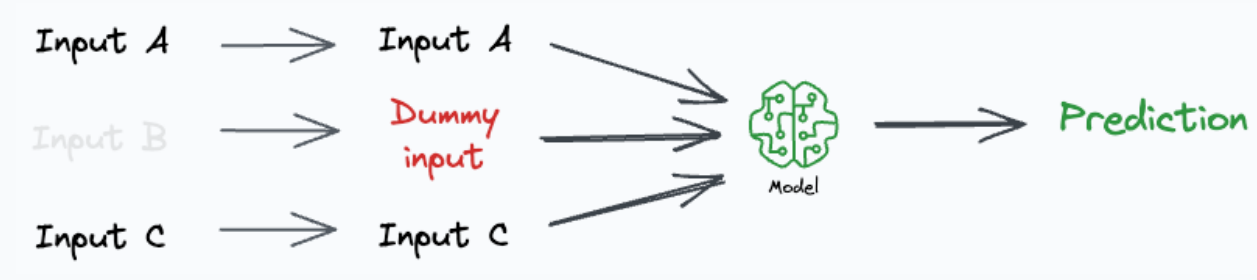
However, when we use a model registry, we version models separately from the code. Let me give you an intuitive example to understand this better. Imagine our deployed model takes three inputs to generate a prediction:



While writing the inference code, we overlooked that, at times, one of the inputs might be missing. We realized this by analyzing the model's logs.



We may want to fix this quickly (at least for a while) before we decide on the next steps more concretely. Thus, we may decide to update the inference code by assigning a dummy value for the missing input.

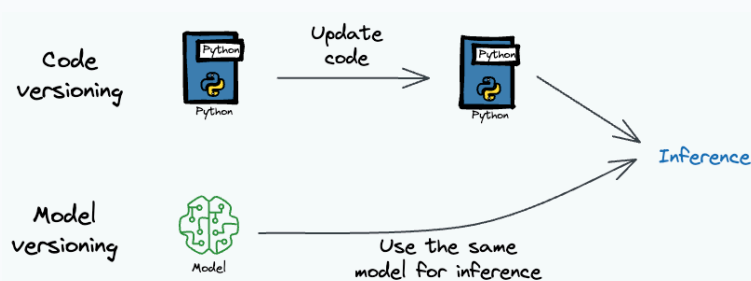


This will allow the model to still process the incoming request.

Let me ask you a question: “Did we update the model?”

No, right?

Here, we only need to update the inference code. The model will remain the same.



But if we were to version the model and code together, it would lead to a redundant model and take up extra space.

However, by maintaining a model registry:

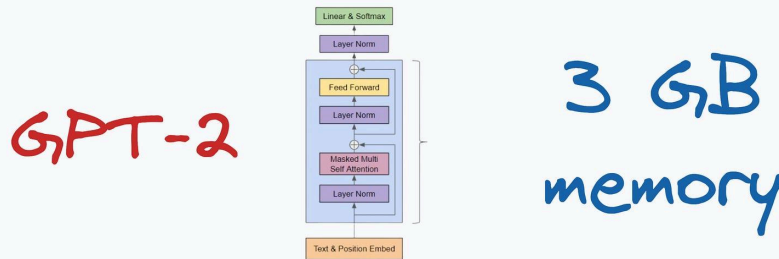
- We can only update the inference code.
- Avoid pushing a new (yet unwanted) model to deployment.

This makes intuitive sense as well, doesn't it?

LLMs

Where Did the GPU Memory Go?

GPT-2 (XL) has 1.5 Billion parameters, and its parameters consume ~3GB of memory in 16-bit precision.



Under 16-bit precision, one parameter takes up 2 bytes of memory, so 1.5B parameters will consume 3GB of memory.

What's your estimate for the minimum memory needed to train GPT-2 on a single GPU?

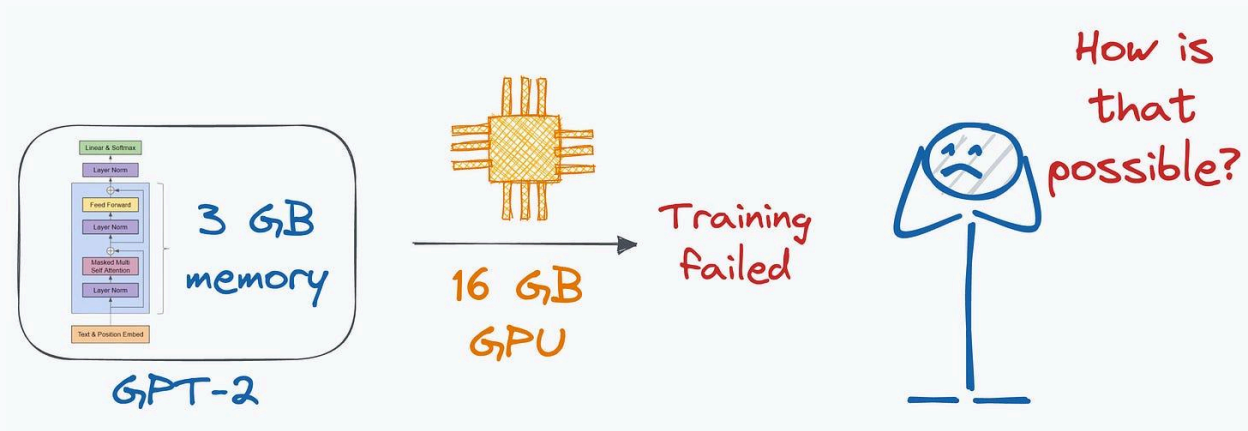
- Optimizer → Adam
- Batch size → 32
- Number of transformer layers → 48
- Sequence length → 1000

What's your estimate for the minimum memory needed to train a GPT-2 model (size 3GBs) on a single GPU?

- 4-6 GB
- 8-10 GB
- 12-15 GB
- 32-35 GB
- 50+ GB

The answer might surprise you.

One can barely train a 3GB GPT-2 model on a single GPU with 32GB of memory.



But how could that be even possible? Where does all the memory go?

Let's understand.

There are so many fronts on which the model consistently takes up memory during training.

#1) Optimizer states, gradients, and parameter memory

Mixed precision training is widely used to speed up model training.

As the name suggests, the idea is to utilize lower-precision `float16` (wherever feasible, like in convolutions and matrix multiplications) along with `float32` — that is why the name “mixed precision.”

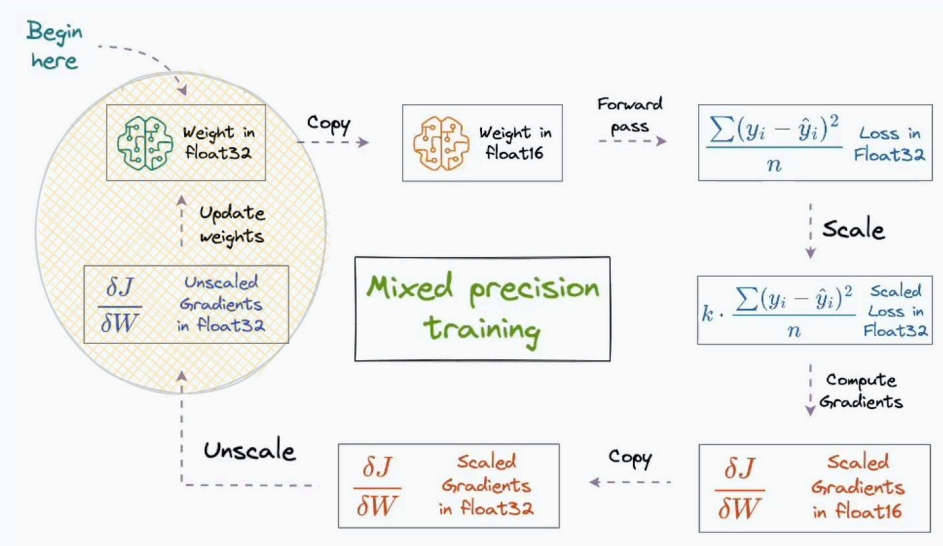
Both the forward and backward propagation are performed using the 16-bit representations of weights and gradients.

Thus, if the model has Φ parameters, then:

- Weights will consume $2 \times \Phi$ bytes.
- Gradients will consume $2 \times \Phi$ bytes.

Here, the figure “2” represents a memory consumption of 2 bytes/parameter (16-bit).

Moreover, the updates at the end of the backward propagation are still performed under 32-bit for effective computation. I am talking about the circled step in the image below:



Adam is one of the most popular optimizers for model training.

While many practitioners use it just because it is popular, they don't realize that during training, Adam stores two optimizer states to compute the updates — momentum and variance of the gradients:

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

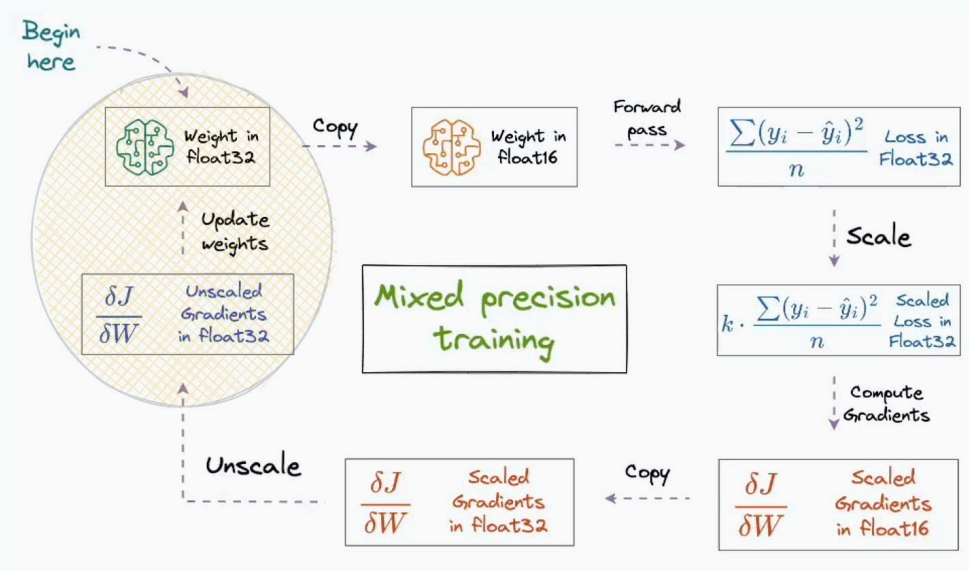
The diagram highlights the two optimizer states in the Adam update formula: $\hat{m}_t$ is labeled 'Momentum' and $\hat{v}_t$ is labeled 'Variance'.

Thus, if the model has Φ parameters, then these two optimizer states will consume:

- $4 \cdot \Phi$ bytes for momentum.

- Another $4 * \Phi$ bytes for variance.

Here, the figure “4” represents a memory consumption of 4 bytes/paramter (32-bit).



Lastly, as shown in the figure above, the final updates are always adjusted in the 32-bit representation of the model weights. This leads to:

- Another $4 * \Phi$ bytes for model parameters.

Let's sum them up:

$$16 * \phi$$

$2 * \phi$	+	$2 * \phi$	+	$4 * \phi$	+	$4 * \phi$	+	$4 * \phi$
Parameters in 16-bit		Gradients in 16-bit		Momentum in 32-bit		Variance in 32-bit		Parameters in 32-bit

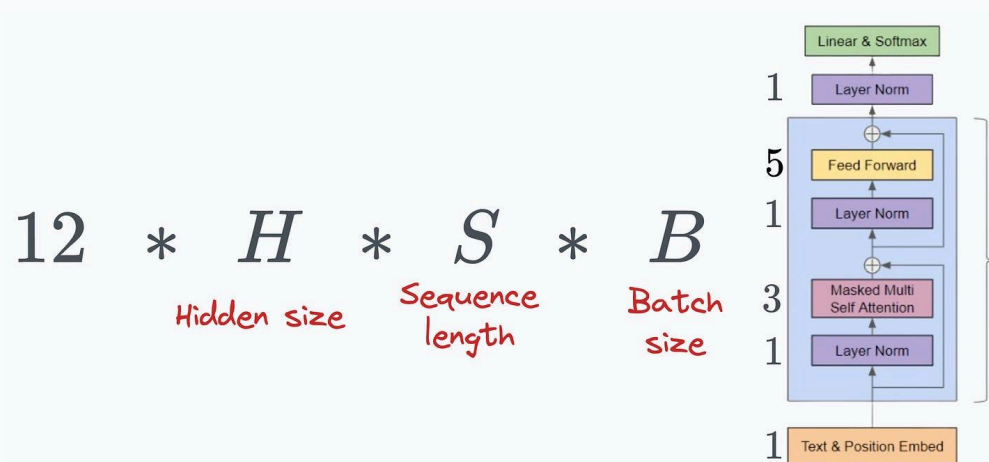
That's $16 * \Phi$, or 24GB of memory, which is ridiculously higher than the 3GB memory utilized by 16-bit parameters.

And we haven't considered everything yet.

#2) Activations

For big deep learning models, like LLMs, Activations take up significant memory during training.

More formally, the total number of activations computed in one transform block of GPT-2 are:



Thus, across all transformer blocks, this comes out to be:

$$12 * H * S * B * N$$

Hidden size Sequence length Batch size Transformer layers

This is the configuration for GPT2-XL:

$$12 * 1600 * 1000 * 32 * 48$$

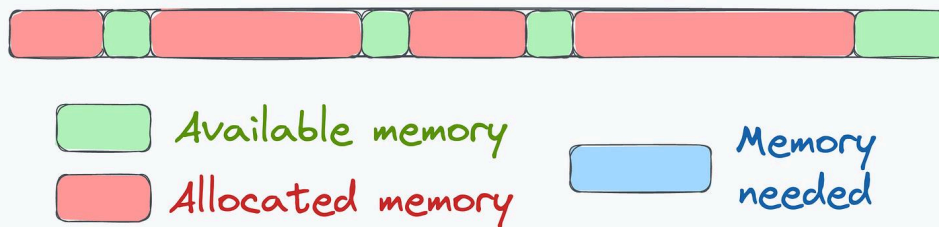
Hidden size Sequence length Batch size Transformer layers

This comes out to be ~30B activations. As each activation is represented in 16-bit, all activations collectively consume 60GB of memory.

With techniques like gradient checkpointing (discussed in the previous chapter), this could be brought down to about 8-9GB at the expense of 25-30% more run-time.

This technique takes complete memory consumption to about 32-35GB range, which I mentioned earlier, for a meager 3GB model, and that too with a pretty small batch size of just 32. On top of this, there are also some more overheads involved, like memory fragmentation.

It occurs when there are small, unused gaps between allocated memory blocks, leading to inefficient use of the available memory.



Memory allocation requests fail because of the unavailability of contiguous memory blocks.

Conclusion

In the above discussion, we considered a relatively small model — GPT-2 (XL) with 1.5 Billion parameters, which is tiny compared to the scale of models being trained these days.

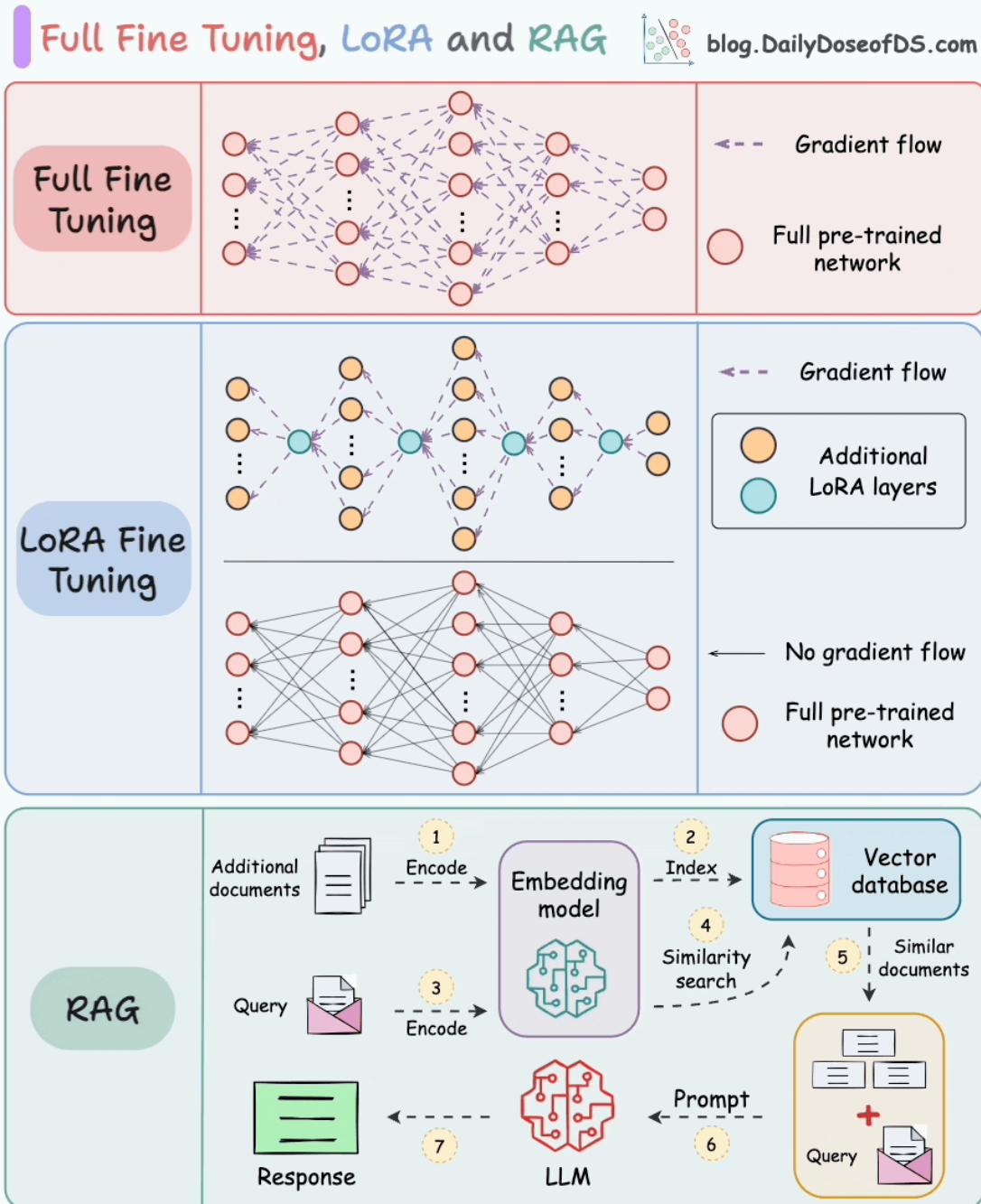
However, the discussion may have helped you reflect on the inherent challenges of building LLMs. Many people often say that GPTs are only about stacking more and more layers in the model and making the network bigger.

If it was that easy, everybody would have been doing it. From this discussion, you may have understood that it's not as simple as appending more layers.

Even one additional layer can lead to multiple GBs of additional memory requirement. Multi-GPU training is at the forefront of these models, which we covered in an earlier chapter in this book.

Full-model Fine-tuning vs. LoRA vs. RAG

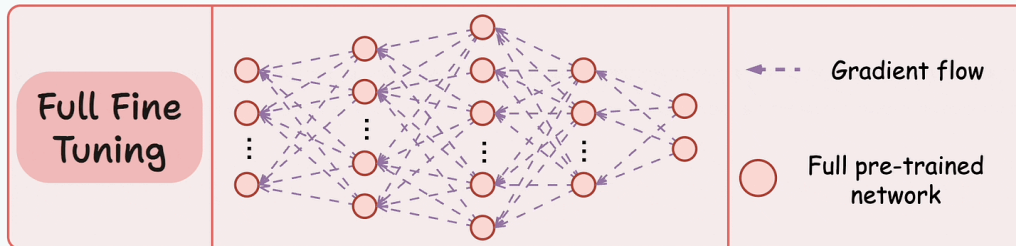
Here's a visual which illustrates "full-model fine-tuning," "fine-tuning with LoRA," and "retrieval augmented generation (RAG)."



All three techniques are used to augment the knowledge of an existing model with additional data.

#1) Full fine-tuning

Fine-tuning means adjusting the weights of a pre-trained model on a new dataset for better performance.

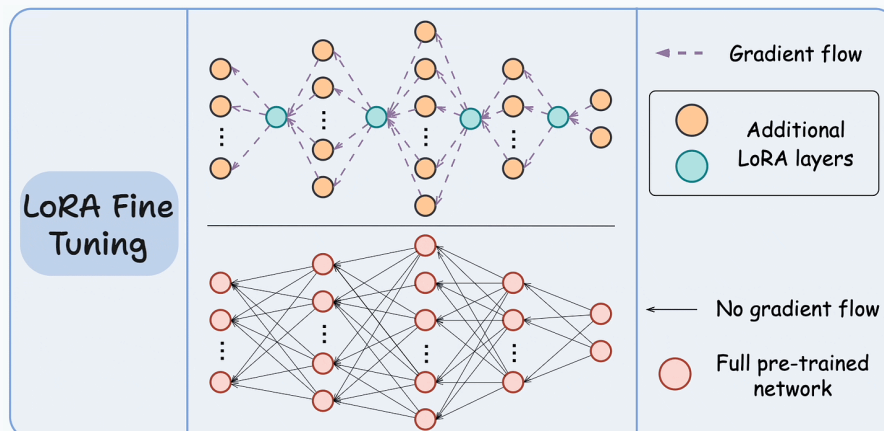


While this fine-tuning technique has been successfully used for a long time, problems arise when we use it on much larger models — LLMs, for instance, primarily because of:

- Their size.
- The cost involved in fine-tuning all weights.
- The cost involved in maintaining all large fine-tuned models.

#2) LoRA fine-tuning

LoRA fine-tuning addresses the limitations of traditional fine-tuning. The core idea is to decompose the weight matrices (some or all) of the original model into low-rank matrices and train them instead. For instance, in the graphic below, the bottom network represents the large pre-trained model, and the top network represents the model with LoRA layers.



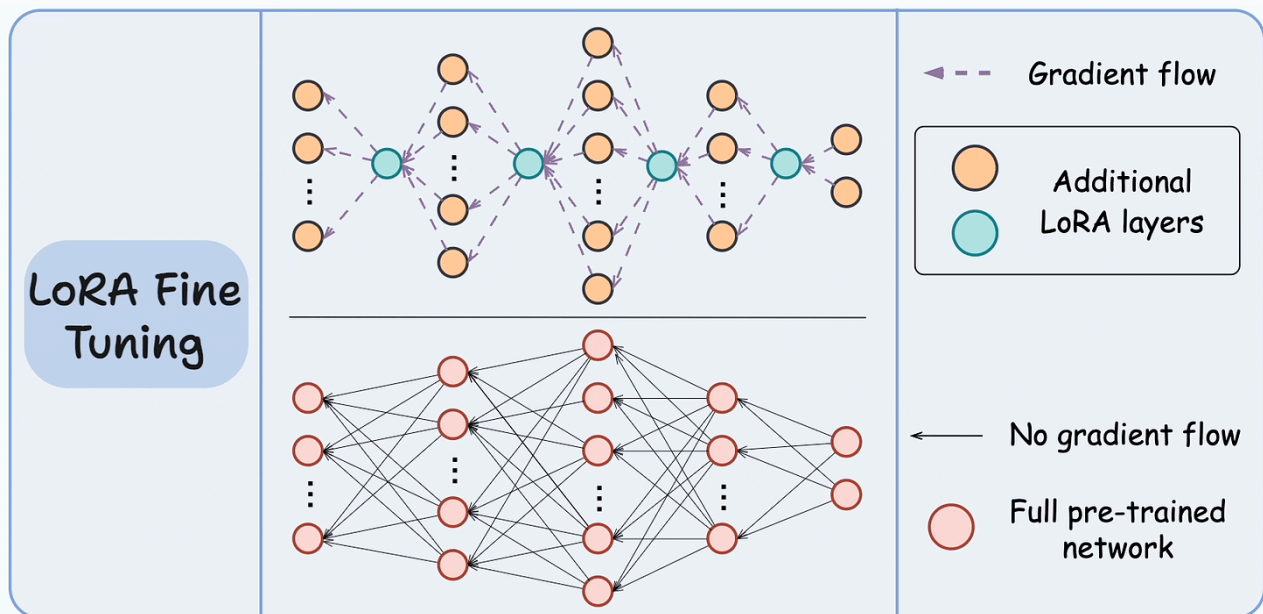
The idea is to train only the LoRA network and freeze the large model.

Looking at the above visual, you might think:

But the LoRA model has more neurons than the original model. How does that help? To understand this, you must make it clear that neurons don't have anything to do with the memory of the network.

They are just used to illustrate the dimensionality transformation from one layer to another.

It is the weight matrices (or the connections between two layers) that take up memory. Thus, we must be comparing these connections instead:

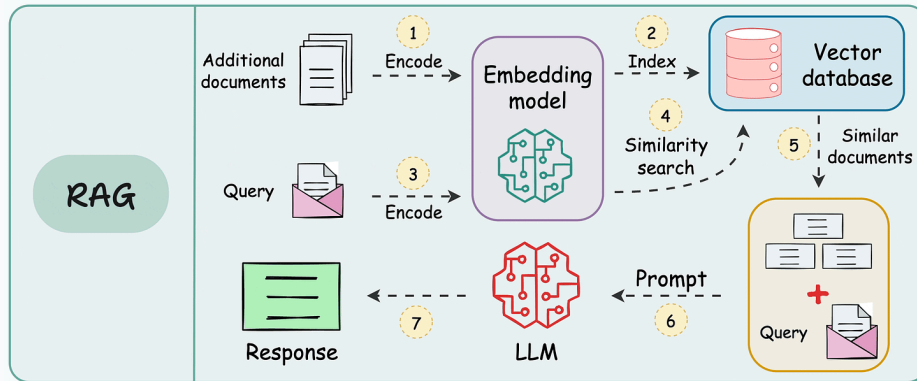


Looking at the above visual, it is pretty clear that the LoRA network has relatively very few connections.

#3) RAG

Retrieval augmented generation (RAG) is another pretty cool way to augment neural networks with additional information, without having to fine-tune the model.

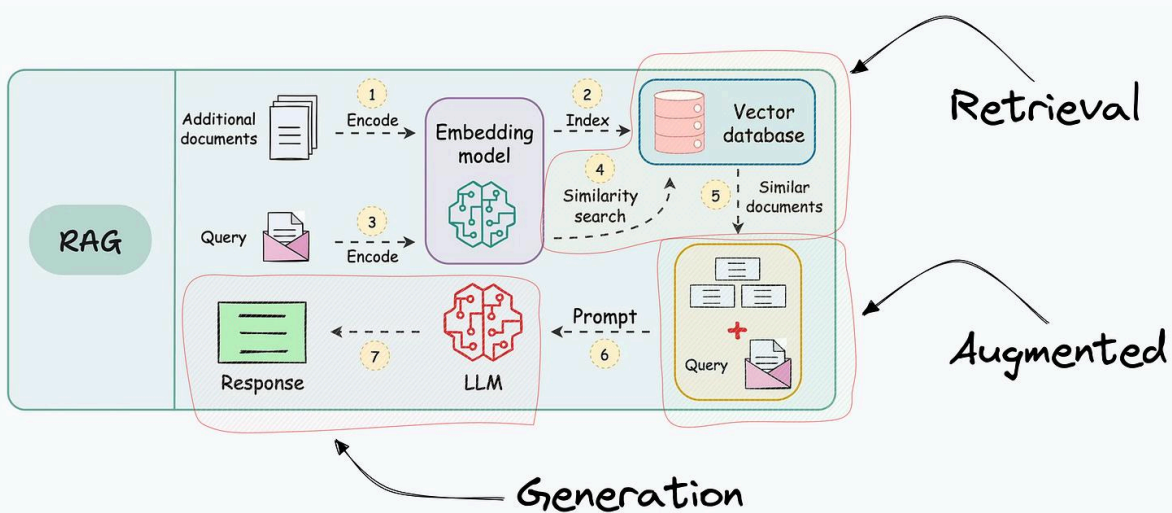
This is illustrated below:



There are 7 steps, which are also marked in the above visual:

- Step 1-2: Take additional data, and dump it in a vector database after embedding. (This is only done once. If the data is evolving, just keep dumping the embeddings into the vector database. There's no need to repeat this again for the entire data)
- Step 3: Use the same embedding model to embed the user query.
- Step 4-5: Find the nearest neighbors in the vector database to the embedded query.
- Step 6-7: Provide the original query and the retrieved documents (for more context) to the LLM to get a response.

In fact, even its name entirely justifies what we do with this technique:



- Retrieval: Accessing and retrieving information from a knowledge source, such as a database or memory.
- Augmented: Enhancing or enriching something, in this case, the text generation process, with additional information or context.
- Generation: The process of creating or producing something, in this context, generating text or language.

Of course, there are many problems with RAG too, such as:

- RAGs involve similarity matching between the query vector and the vectors of the additional documents. However, questions are structurally very different from answers.
- Typical RAG systems are well-suited only for lookup-based question-answering systems. For instance, we cannot build a RAG pipeline to summarize the additional data. The LLM never gets info about all the documents in its prompt because the similarity matching step only retrieves top matches.

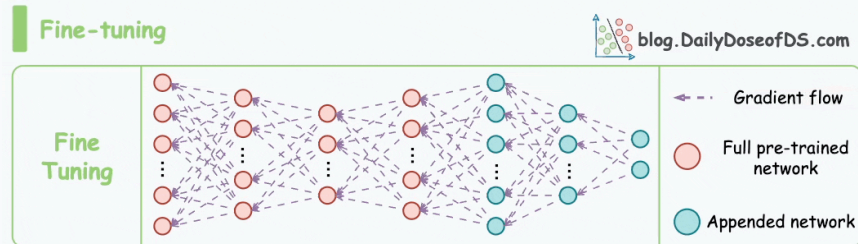
So, it's pretty clear that RAG has both pros and cons.

- We never have to fine-tune the model, which saves a lot of computing power.
- But this also limits the applicability to specific types of systems.

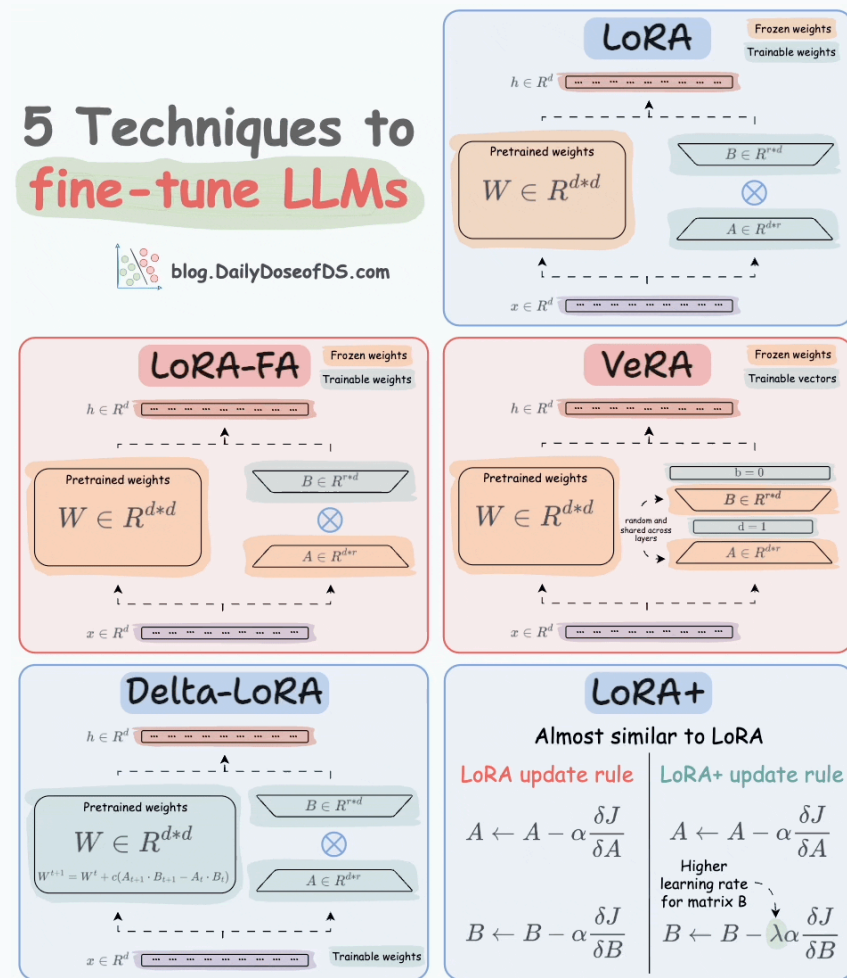
Let's continue the discussion on LLM fine-tuning in the next chapter.

5 LLM Fine-tuning Techniques

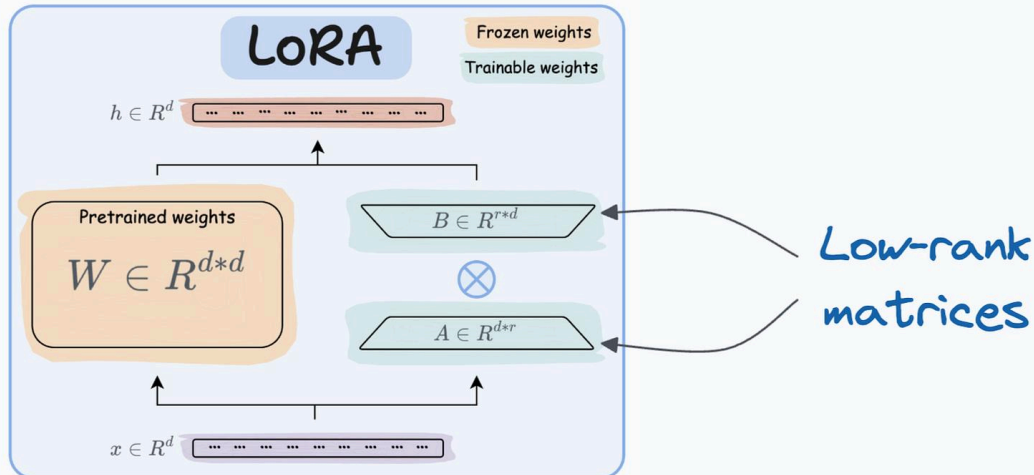
Traditional fine-tuning (depicted below) is infeasible with LLMs because these models have billions of parameters and are hundreds of GBs in size, and not everyone has access to such computing infrastructure.



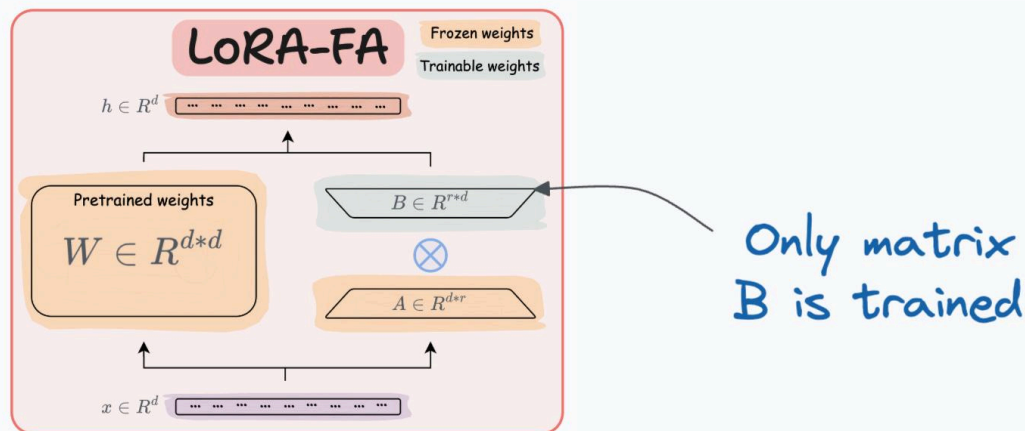
But today, we have many optimal ways to fine-tune LLMs, and five popular techniques are depicted below:



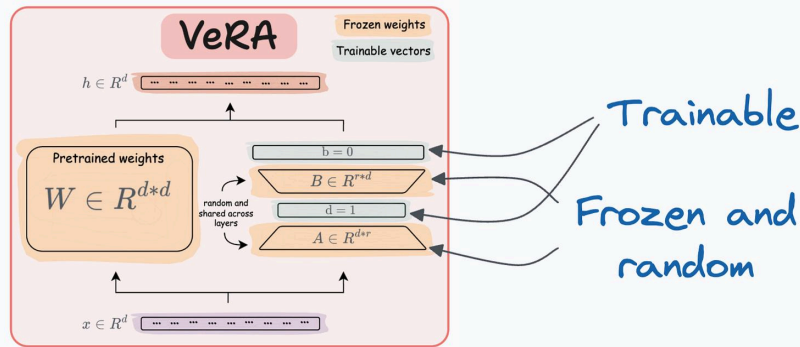
- LoRA: Add two low-rank matrices A and B alongside weight matrices, which contain the trainable parameters. Instead of fine-tuning W, adjust the updates in these low-rank matrices.



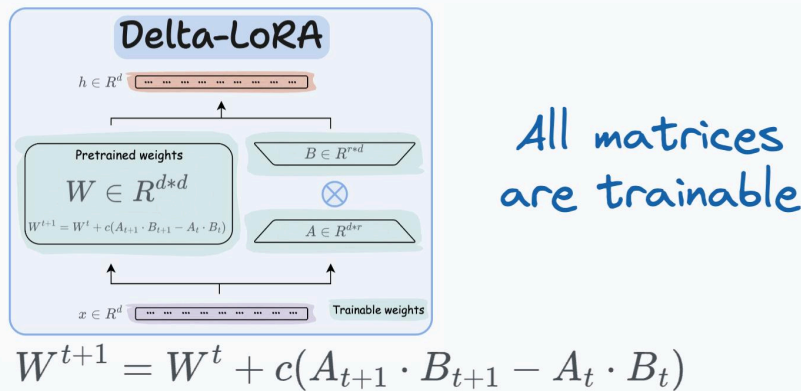
- LoRA-FA: While LoRA considerably decreases the total trainable parameters, it still requires substantial activation memory to update the low-rank weights. LoRA-FA (FA stands for Frozen-A) freezes the matrix A and only updates matrix B.



- VeRA: In LoRA, every layer has a different pair of low-rank matrices A and B, and both matrices are trained. In VeRA, however, matrices A and B are frozen, random, and shared across all model layers. VeRA focuses on learning small, layer-specific scaling vectors, denoted as b and d, which are the only trainable parameters in this setup.

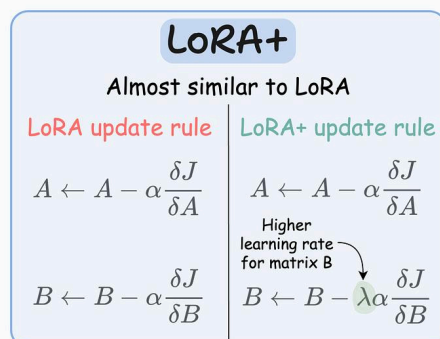


- Delta-LoRA: Here, in addition to training low-rank matrices, the matrix W is also adjusted but not in the traditional way. Instead, the difference (or delta) between the product of the low-rank matrices A and B in two consecutive training steps is added to W :



$$W^{t+1} = W^t + c(A_{t+1} \cdot B_{t+1} - A_t \cdot B_t)$$

- LoRA+: In LoRA, both matrices A and B are updated with the same learning rate. Authors found that setting a higher learning rate for matrix B results in more optimal convergence.



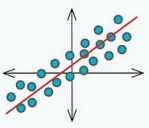
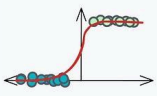
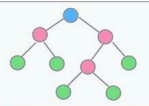
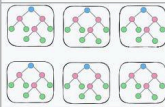
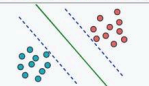
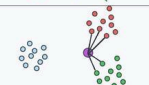
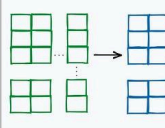
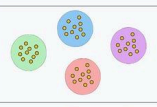
LoRA+ assigns higher learning rate for updating matrix B

Classical ML

ML Fundamentals

Training and Inference Time Complexity of 10 ML Algorithms

Here's the run-time complexity of the 10 most popular ML algorithms.

Time Complexity of 10 Most Popular ML Algorithms  blog.DailyDoseofDS.com			
		Training	Inference
	Linear Regression (OLS)	$O(nm^2 + m^3)$	$O(m)$
	Linear Regression (SGD)	$O(n_{epoch}nm)$	$O(m)$
	Logistic Regression (Binary)	$O(n_{epoch}nm)$	$O(m)$
	Logistic Regression (Multiclass OvR)	$O(n_{epoch}nmc)$	$O(mc)$
	Decision Tree	$O(n \cdot \log(n) \cdot m)$ $O(n^2 \cdot m)$ * Worst case	$O(d_{tree})$
	Random Forest Classifier	$O(n_{trees} \cdot n \cdot \log(n) \cdot m)$	$O(n_{trees} \cdot d_{tree})$
	Support Vector Machines (SVMs)	$O(n^2m + n^3)$	$O(m \cdot n_{SV})$
	k-Nearest Neighbors	—	$O(nm)$
$P(B A) = \frac{P(B \cap A)}{P(A)}$	Naive Bayes	$O(nm)$	$O(mc)$
	Principal Component Analysis (PCA)	$O(nm^2 + m^3)$	—
	t-SNE	$O(n^2m)$	—
	KMeans Clustering	$O(iknm)$	??
n: samples m: dimensions n_{epoch}: epochs c: classes d_{tree}: depth n_{SV}: Support vectors k: clusters i: iterations			

But why even care about run time? There are multiple reasons why I always care about run time and why you should too.

To begin, we know that everyone is a big fan of sklearn implementations. It literally takes just two (max three) lines of code to run any ML algorithm with sklearn. Yet, in my experience, due to this simplicity, most users often overlook:

- The core understanding of an algorithm.
- The data-specific conditions that allow us to use an algorithm.

For instance, you'll be up for a big surprise if you use SVM or t-SNE on a dataset with plenty of samples.

- SVM's run-time grows cubically with the total number of samples.
- t-SNE's run-time grows quadratically with the total number of samples.

Another advantage of figuring out the run-time is that it helps us understand how an algorithm works end-to-end. Of course, in the above table, I have made some assumptions here and there. For instance:

- In a random forest, all decision trees may have different depths. But here, I have assumed that they are equal.
- During inference in kNN, we first find the distance to all data points. This gives a list of distances of size n (total samples).
 - Then, we find the k -smallest distances from this list.
 - The run-time to determine the k -smallest values may depend on the implementation.
 - Sorting and selecting the k -smallest values will be $O(n \log n)$.
 - But if we use a priority queue, it will take $O(n \log(k))$.
- In t-SNE, there's a learning step. Since the major run-time comes from computing the pairwise similarities in the high-dimensional space, we have ignored that step.

Nonetheless, the table still accurately reflects the general run-time of each of these algorithms.

As an exercise, I would encourage you to derive these run-time complexities yourself. This activity will provide you so much confidence in algorithmic understanding.

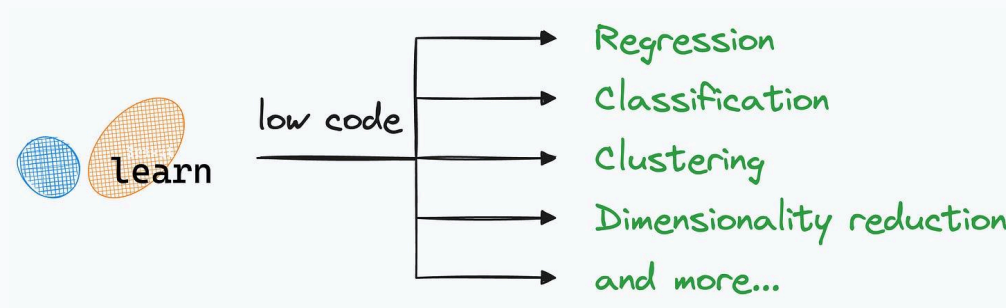
25 Most Important Mathematical Definitions in Data Science

"Is mathematical knowledge important in data science and machine learning?"

This is a question that so many people have, especially those who are just getting started.

Short answer: Yes, it's important, and here's why I say so.

See...these days, one can do "ML" without understanding any mathematical details of an algorithm. For instance (and thanks to sklearn, by the way):



- One can use any clustering algorithm in 2-3 lines of code.
- One can train classification models in 2-3 lines of code.
- And more.

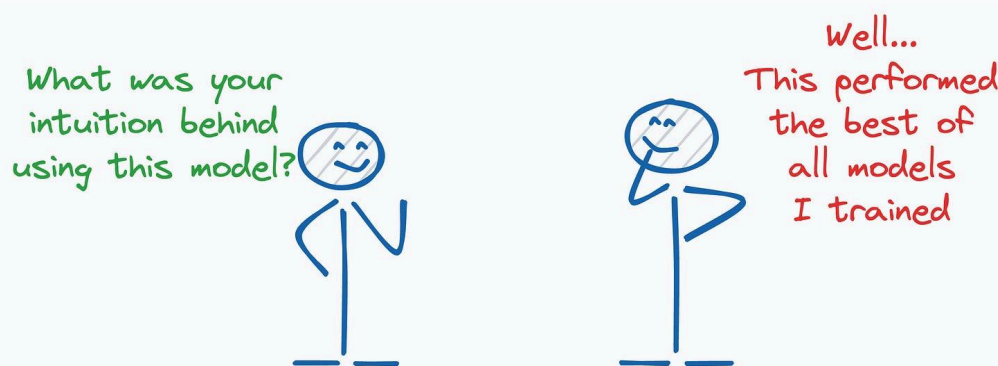
This is both good and bad:

- It's good because it saves us time.
- It's bad because this tempts us to ignore the underlying details.

In fact, I know many data scientists (mainly on the applied side) who do not entirely understand the mathematical details but can still build and deploy models.

Nothing wrong.

However, when I talk to them, I also see some disconnect between “What they are using” and “Why they are using it.”



Due to a lack of understanding of the underlying details:

- They find it quite difficult to optimize their models.
- They struggle to identify potential areas of improvement.
- They take a longer time to debug when things don't work well.
- They do not fully understand the role of specific hyperparameters.
- They use any algorithm without estimating their time complexity first.

If it feels like you are one of them, it's okay. This problem can be solved.

That said, if you genuinely aspire to excel in this field, building a curiosity for the underlying mathematical details holds exponential returns.

- Algorithmic awareness will give you confidence.
- It will decrease your time to build and iterate.

Gradually, you will go from a hit-and-trial approach to “I know what should work.”

To help you take that first step, I prepared the following visual, which lists some of the most important mathematical formulations used in Data Science and Statistics (in no specific order).

Before reading ahead, look at them one by one and calculate how many of them do you already know:

25 Most important Mathematical Definitions in Data Science

 blog.DailyDoseofDS.com

1) Gradient Descent $\theta_{j+1} = \theta_j - \alpha \nabla J(\theta_j)$	2) Normal distribution $f(x \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$	3) Z-score $z = \frac{x - \mu}{\sigma}$	
4) Sigmoid $\sigma(x) = \frac{1}{1 + e^{-x}}$	5) Correlation $\text{Correlation} = \frac{\text{Cov}(X, Y)}{\text{Std}(X) \cdot \text{Std}(Y)}$	6) Cosine Similarity $\text{similarity} = \frac{A \cdot B}{\ A\ \ B\ }$	
7) Naive Bayes $P(y x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i y)}{P(x_1, \dots, x_n)}$	8) MLE $\text{argmax}_{\theta} \prod_{i=1}^n P(x_i \theta)$	9) OLS $\hat{\beta} = (X^T X)^{-1} X^T y$	
10) F1 Score $\frac{2 \cdot P \cdot R}{P + R}$	11) ReLU $\max(0, x)$	12) Softmax $P(y = j x) = \frac{e^{x^T w_j}}{\sum_{k=1}^K e^{x^T w_k}}$	13) R2 score $R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$
14) MSE $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	15) MSE + L2 Reg $\text{MSE}_{\text{regularized}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$	16) Eigen vectors $Av = \lambda v$	
17) Entropy $\text{Entropy} = - \sum_i p_i \log_2(p_i)$	18) KMeans $\text{argmin}_S \sum_{i=1}^k \sum_{x \in S_i} \ x - \mu_i\ ^2$	19) KL Divergence $D_{\text{KL}}(P\ Q) = \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{Q(x)}\right)$	
20) Log-loss $-\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$	21) SVM $\min_{w, b} \frac{1}{2} \ w\ ^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w \cdot x_i - b))$		
22) Linear regression $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$		23) SVD $A = U \Sigma V^T$	
24) Lagrange multiplier $\max f(x) ; g(x) = 0$ $L(x, \lambda) = f(x) - \lambda * g(x)$		25) What will you add? $\dots$	

Some of the terms are pretty self-explanatory, so I won't go through each of them, like:

- Gradient Descent, Normal Distribution, Sigmoid, Correlation, Cosine similarity, Naive Bayes, F1 score, ReLU, Softmax, MSE, MSE + L2 regularization, KMeans, Linear regression, SVM, Log loss.

Here are the remaining terms:

- MLE (Maximum Likelihood Estimation): A method for estimating the parameters of a statistical model by maximizing the likelihood of the observed data. We covered it in the previous chapter.

- Z-score: A standardized value that indicates how many standard deviations a data point is from the mean.
- Ordinary Least Squares: A closed-form solution for linear regression obtained using the MLE step mentioned above.
- Entropy: A measure of the uncertainty of a random variable.
- Eigen Vectors: The non-zero vectors that do not change their direction when a linear transformation is applied. It is widely used in dimensionality reduction techniques like PCA.
- R2 (R-squared): A statistical measure that represents the proportion of variance explained by a regression model.
- KL divergence: Assess how much information is lost when one distribution is used to approximate another distribution. It is used as a loss function in the t-SNE algorithm.
- SVD: A factorization technique that decomposes a matrix into three other matrices, often noted as U , Σ , and V . It is fundamental in linear algebra for applications like dimensionality reduction, noise reduction, and data compression.
- Lagrange multipliers: They are commonly used mathematical techniques to solve constrained optimization problems. For instance, consider an optimization problem with an objective function $f(x)$ and assume that the constraints are $g(x)=0$ and $h(x)=0$. Lagrange multipliers solve this.

optimize this

$$\max f(x)$$

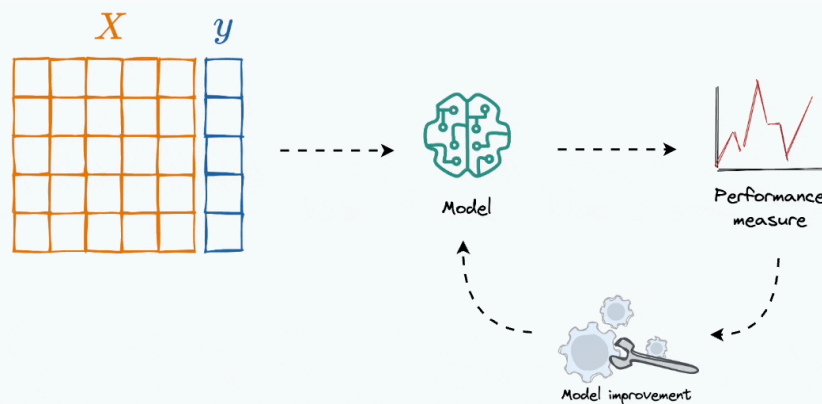
constrained
to this

$$h(x) = 0 ; g(x) = 0$$

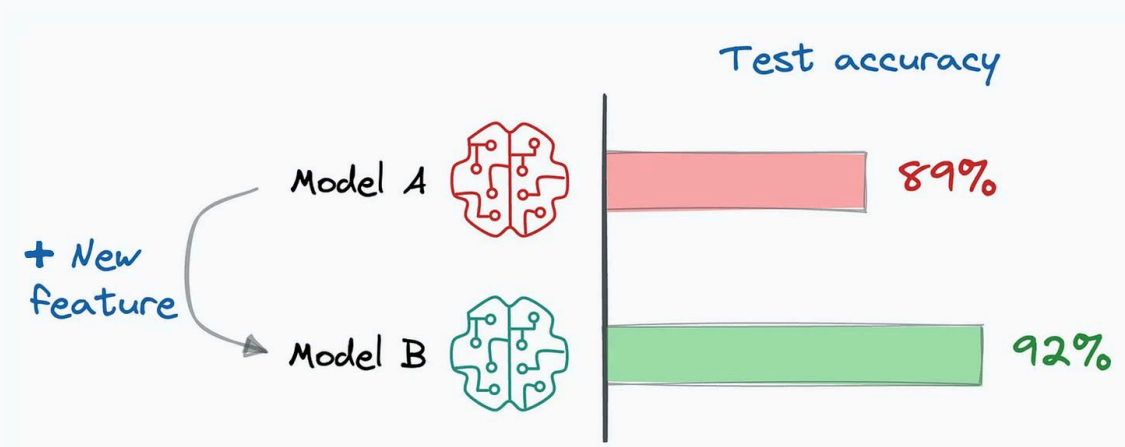
How to Reliably Improve Probabilistic Multiclass-classification Models

ML model building is typically an iterative process. Given some dataset:

- We train a model.
- We evaluate it.
- And we continue to improve it until we are satisfied with the performance.

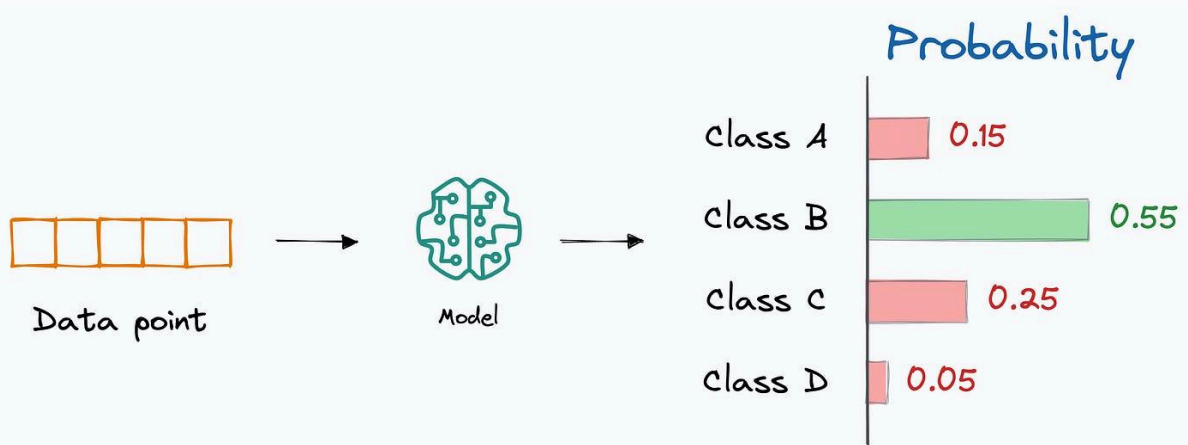


Here, the efficacy of any model improvement strategy (say, introducing a new feature) is determined using some sort of performance metric.



However, I have often observed that when improving probabilistic multiclass-classification models, this technique can be a bit deceptive when the efficacy is determined using “Accuracy.”

Probabilistic multiclass-classification models are those models that output probabilities corresponding to each class, like neural networks.

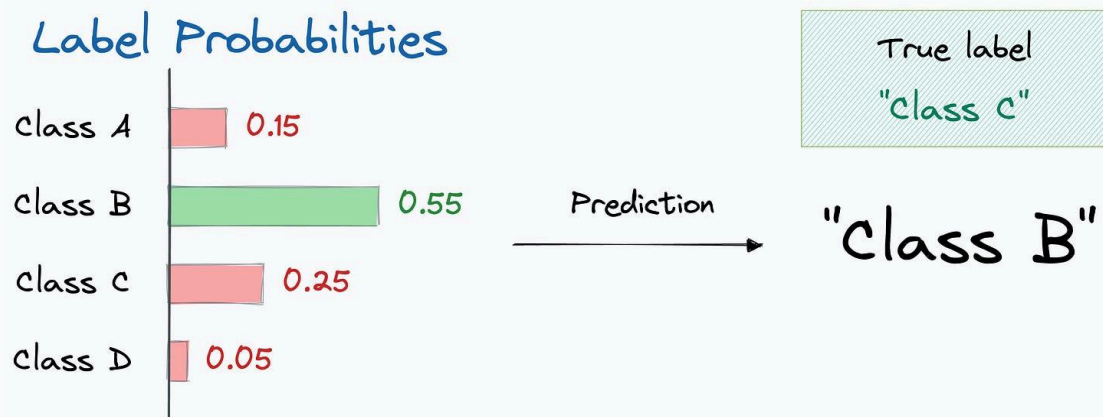


In other words, it is possible that we are actually making good progress in improving the model, but “Accuracy” is not reflecting that (yet).

Let's understand.

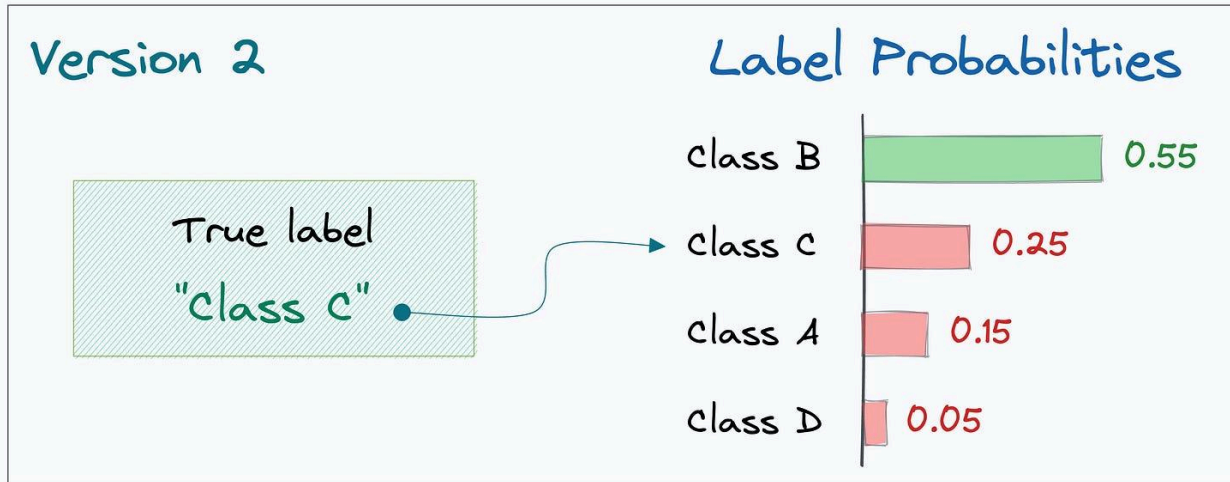
Pitfall of Accuracy

In probabilistic multiclass-classification models, Accuracy is determined using the output label that has the highest probability:

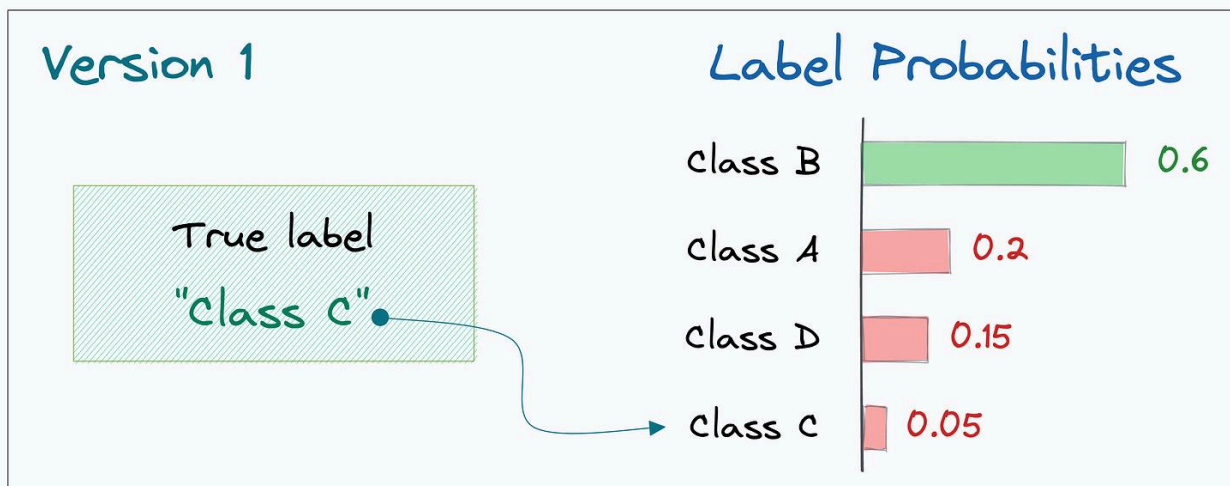


Now, it's possible that the actual label is not predicted with the highest probability by the model, but it's in the top “k” output labels.

For instance, in the image below, the actual label (Class C) is not the highest probability label, but it's at least in the top 2 predicted probabilities (Class B and Class C):



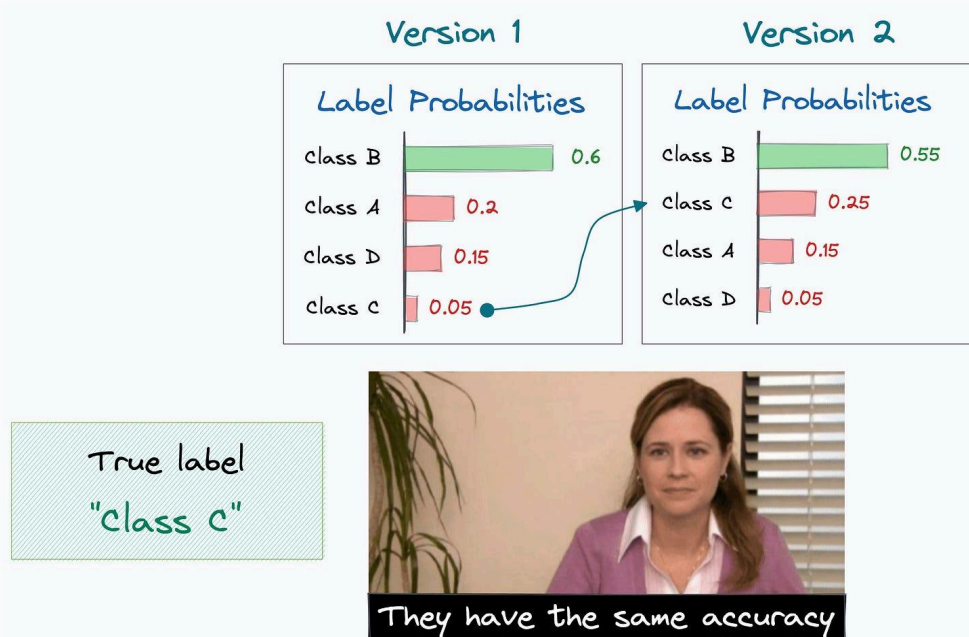
And what if in an earlier version of our model, the output probability of Class C was the lowest, as depicted below:



Now, of course, in both cases, the final prediction is incorrect.

However, while iterating from “Version 1” to “Version 2” using some model improvement techniques, we genuinely made good progress.

Nonetheless, Accuracy entirely discards this as it only cares about the highest probability label.



I hope you understand the problem here.

Solution

Whenever I am building and iteratively improving any probabilistic multiclass classification model, I always use the top-k accuracy score. As the name suggests, it computes whether the correct label is among the top k labels predicted probabilities or not.

As you may have already guessed, top-1 accuracy score is the traditional Accuracy score. This is a much better indicator to assess whether my model improvement efforts are translating into meaningful enhancements in predictive performance or not.

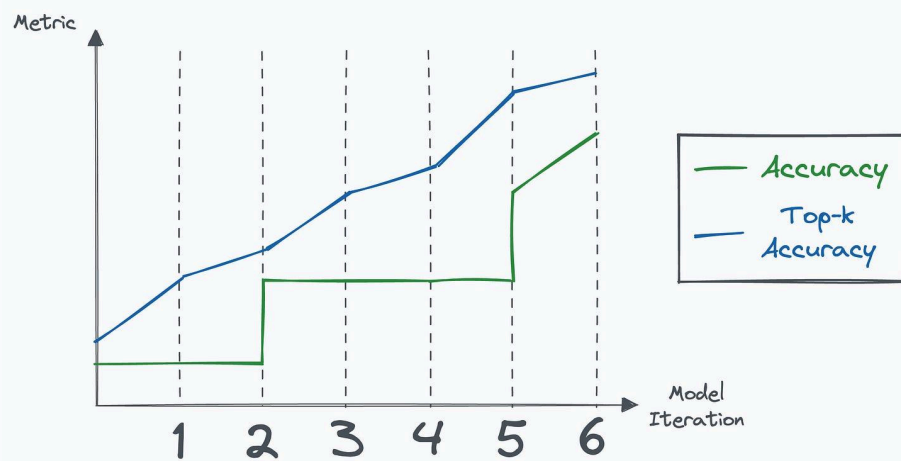
For instance, if the top-3 accuracy score goes from 75% to 90%, this totally suggests that whatever we did to improve the model was effective:

- Earlier, the correct prediction was in the top 3 labels only 75% of the time.
- But now, the correct prediction is in the top 3 labels 90% of the time.

As a result, one can effectively redirect their engineering efforts in the right direction. Of course, what I am saying should only be used to assess the model improvement efforts.

This is because true predictive power will inevitably be determined using traditional model accuracy. So make sure you are gradually progressing on the Accuracy front too.

Ideally, it is expected that “Top-k Accuracy” may continue to increase during model iterations, which reflects improvement in performance. Accuracy, however, may stay the same for a while, as depicted below:



Top-k accuracy score is also available in Sklearn:

sklearn.metrics.top_k_accuracy_score

```
sklearn.metrics.top_k_accuracy_score(y_true, y_score, *, k=2, normalize=True, sample_weight=None, labels=None)
```

[\[source\]](#)

Top-k Accuracy classification score.

This metric computes the number of times where the correct label is among the top k labels predicted (ranked by predicted scores). Note that the multilabel case isn't covered here.

Read more in the [User Guide](#)

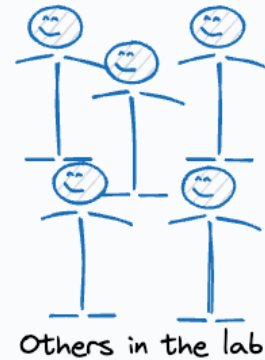
Parameters: **y_true** : array-like of shape (n_samples,)
True labels.

y_score : array-like of shape (n_samples,) or (n_samples, n_classes)
Target scores. These can be either probability estimates or non-thresholded decision values (as returned

Your Entire Model Improvement Efforts Might Be Going in Vain

Back in 2019, I was working with an ML research group in Germany.

One day, a Ph.D. student came up to me (and others in the lab), handed over a small sample of the dataset he was working with, and requested us to label it, despite having true labels.

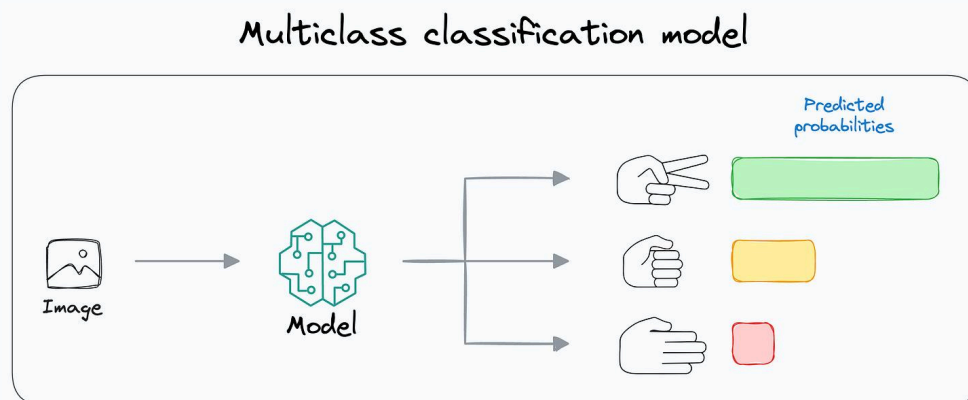


This made me curious about why gathering human labels was necessary for him when he already had ground truth labels available. So I asked.

What I learned that day changed my approach to incremental model improvement, and I am sure you will find this idea fascinating too.

Let me explain what I learned.

Consider we are building a multiclass classification model. Say it's a model that classifies an input image as a rock, paper, or scissors:



For simplicity, let's assume there's no class imbalance. Calculating the class-wise validation accuracies gives us the following results:

class-wise
accuracies

class	Accuracy
Paper	97%
Rock	82%
Scissor	75%

Question: Which class would you most intuitively proceed to inspect further and improve the model on?

After looking at these results, most people believe that "Scissor" is the worst-performing class and should be inspected further.

Scissor class is giving bad results. I should inspect it.



class	Accuracy
Paper	97%
Rock	82%
Scissor	75%

But this might not be true. And this is precisely what that Ph.D. student wanted to verify by collecting human labels. Let's say that the human labels give us the following results:

class-wise
human
accuracies

class	Human Accuracy 		Model Accuracy 
Paper	98%	-1%	97%
Rock	95%	-13%	82%
Scissor	77%	-2%	75%

Based on this, do you still think the model performs the worst on the "Scissor" class?

No, right?

I mean, of course, the model has the least accuracy on the “Scissor” class, and I am not denying it. However, with more context, we notice that the model is doing a pretty good job classifying the “Scissor” class. This is because an average human is achieving just 2% higher accuracy in comparison to what our model is able to achieve.

However, the above results astonishingly reveal that it is the “Rock” class instead that demands more attention. The accuracy difference between an average human and the model is way too high (13%). Had we not known this, we would have continued to improve the “Scissor” class, when in reality, “Rock” requires more improvement.

Ever since I learned this technique, I have found it super helpful to determine my next steps for model improvement, if possible. I say “if possible” because I understand that many datasets are hard for humans to interpret and label. Nonetheless, if it is feasible to set up such a “human baseline,” one can get so much clarity into how the model is performing.

I must inspect the “Rock” class



class	Human Accuracy		Model Accuracy
Paper	98%	-1%	97%
Rock	95%	-13%	82%
Scissor	77%	-2%	75%

As a result, one can effectively redirect their engineering efforts in the right direction.

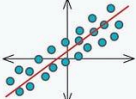
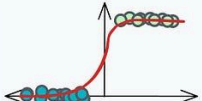
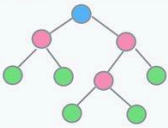
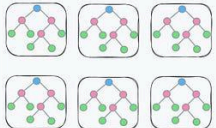
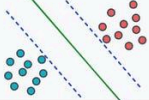
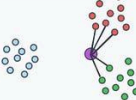
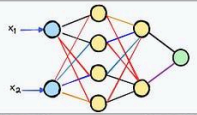
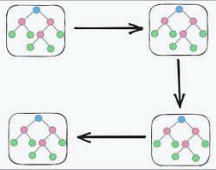
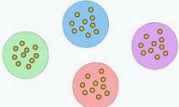
Of course, I am not claiming that this will be universally useful in all use cases.

For instance, if the model is already performing better than the baseline, the model improvements from there on will have to be guided based on past results.

Yet, in such cases, surpassing a human baseline at least helps us validate that the model is doing better than what a human can do.

Loss Function of 16 ML Algos

The below visual depicts the most commonly used loss functions by various ML algorithms.

Loss functions of 16 Most Popular ML Algorithms  blog.DailyDoseofDS.com		
	Linear Regression	Mean Squared Error
	Logistic Regression	Cross-Entropy Loss
	Decision Tree Classifier	Information Gain or Gini impurity
	Decision Tree Regressor	Mean Squared Error
	Random Forest Classifier	Information Gain or Gini impurity
	Random Forest Regressor	Mean Squared Error
	Support Vector Machines (SVMs)	Hinge Loss
	k-Nearest Neighbors	No loss function
$P(B A) = \frac{P(B \cap A)}{P(A)}$	Naive Bayes	No loss function
	Neural Networks	Regression: Mean Squared Error Classification: Cross-Entropy Loss
	AdaBoost	Exponential loss
	Gradient Boosting LightGBM CatBoost XGBoost	Regression: Mean Squared Error Classification: Cross-Entropy Loss
	KMeans Clustering	??
*These are typically used loss functions. It does not mean they are used always in these algorithms.		

10 Most Common Loss Function

The below visual depicts some commonly used loss functions in regression and classification tasks.

10 Most Common Loss Functions in Machine Learning

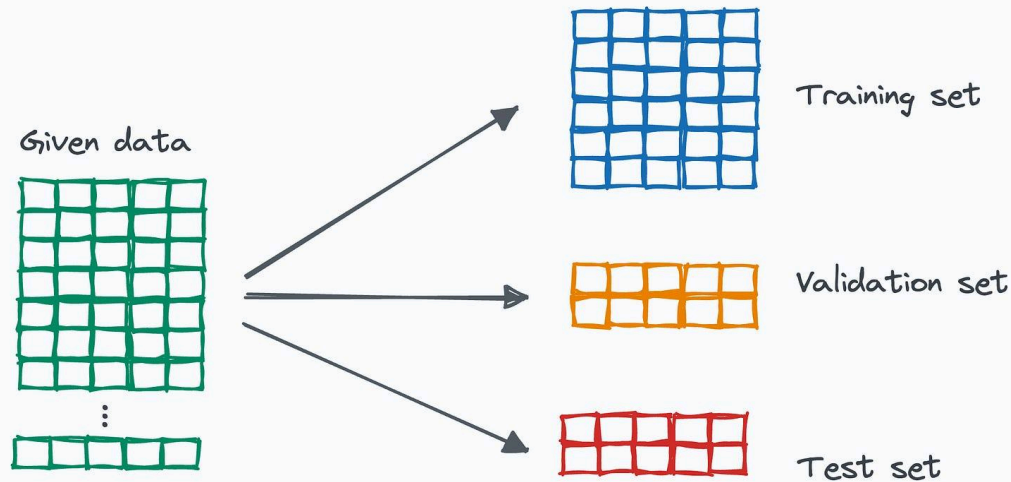


blog.DailyDoseofDS.com

Loss Function Name	Description	Function
Regression Loss Functions		
Mean Bias Error	Captures average bias in prediction. But is rarely used for training.	$\mathcal{L}_{MBE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))$
Mean Absolute Error	Measures absolute average bias in prediction. Also called L1 Loss.	$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N y_i - f(x_i) $
Mean Squared Error	Average squared distance between actual and predicted. Also called L2 Loss.	$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$
Root Mean Squared Error	Square root of MSE. Loss and dependent variable have same units.	$\mathcal{L}_{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2}$
Huber Loss	A combination of MSE and MAE. It is parametric loss function.	$\mathcal{L}_{Huberloss} = \begin{cases} \frac{1}{2}(y_i - f(x_i))^2 & : y_i - f(x_i) \leq \delta \\ \delta(y_i - f(x_i) - \frac{1}{2}\delta) & : otherwise \end{cases}$
Log Cosh Loss	Similar to Huber Loss + non-parametric. But computationally expensive.	$\mathcal{L}_{LogCosh} = \frac{1}{N} \sum_{i=1}^N \log(\cosh(f(x_i) - y_i))$
Classification Loss Functions (Binary + Multi-class)		
Binary Cross Entropy (BCE)	Loss function for binary classification tasks.	$\mathcal{L}_{BCE} = \frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(x_i)) + (1 - y_i) \cdot \log(1 - p(x_i))$
Hinge Loss	Penalizes wrong and right (but less confident) predictions. Commonly used in SVMs.	$\mathcal{L}_{Hinge} = \max(0, 1 - (f(x) \cdot y))$
Cross Entropy Loss	Extension of BCE loss to multi-class classification.	$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(f(x_{ij}))$ $N : \text{samples}; M : \text{classes}$
KL Divergence	Minimizes the divergence between predicted and true probability distribution	$\mathcal{L}_{KL} = \sum_{i=1}^N y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$

How to Actually Use Train, Validation and Test Set

It is pretty conventional to split the given data into train, test, and validation sets.



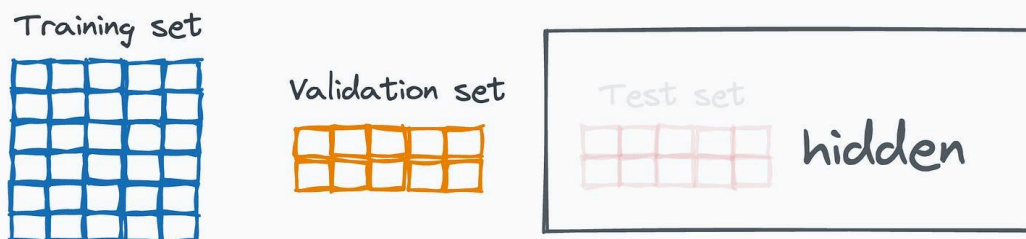
However, there are quite a few misconceptions about how they are meant to be used, especially the validation and test sets.

In this chapter, let's clear them up and see how to truly use train, validation, and test sets.

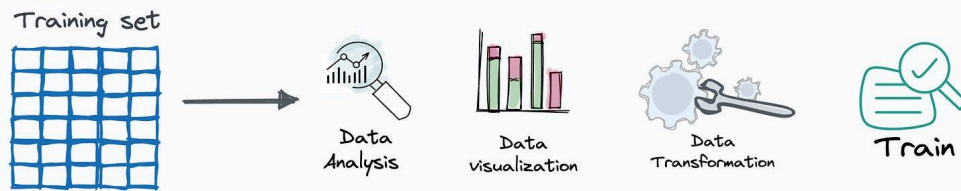
We begin by splitting the data into:

- Train
- Validation
- Test

At this point, just assume that the test data does not even exist. Forget about it instantly.



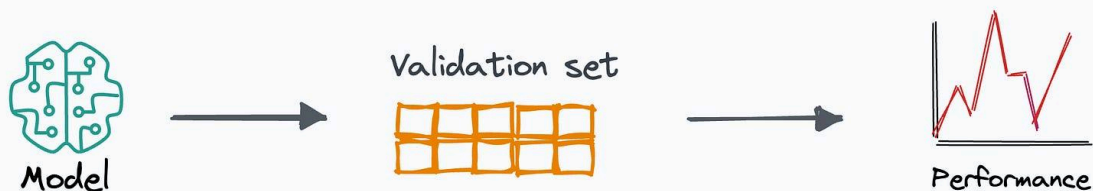
Begin with the train set. This is your whole world now.



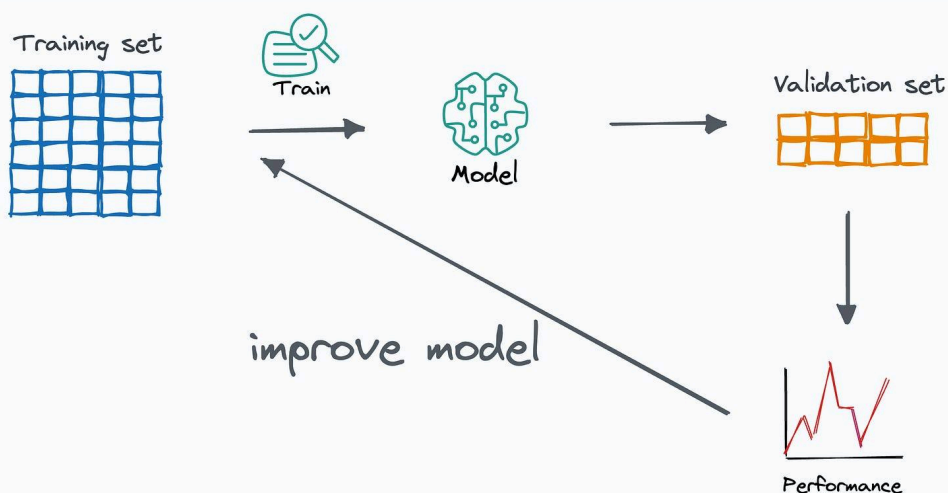
- You analyze it
- You transform it
- You use it to determine features
- You fit a model on it

After modeling, you would want to measure the model's performance on unseen data, wouldn't you?

Bring in the validation set now.



Based on validation performance, improve the model. Here's how you iteratively build your model:



- Train using a train set
- Evaluate it using the validation set
- Improve the model
- Evaluate again using the validation set
- Improve the model again
- and so on.

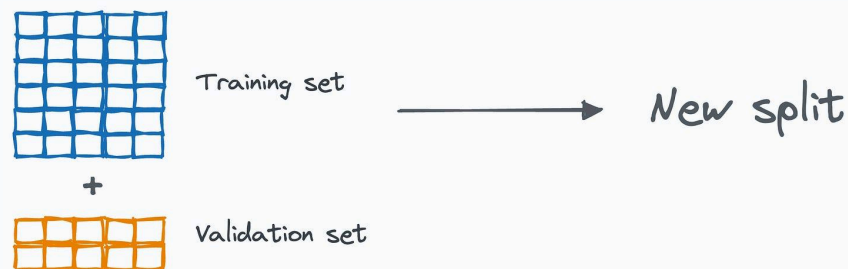
Until...

You reach a point where you start overfitting the validation set.

This indicates that you have exploited (or polluted) the validation set.

No worries.

Merge it with the train set and generate a new split of train and validation.



Note: Rely on cross-validation if needed, especially when you don't have much data. You may still use cross-validation if you have enough data. But it can be computationally intensive.

Now, if you are happy with the model's performance, evaluate it on test data.

✓ What you use a test set for:

- Get a final and unbiased review of the model.

✗ What you DON'T use a test set for:

- Analysis, decision-making, etc.

If the model is underperforming on the test set, no problem. Go back to the modeling stage and improve it.

BUT (and here's what most people do wrong)!

They use the same test set again. This is not allowed!

Think of it this way.

Your professor taught you in the classroom. All in-class lessons and examples are the train set.

The professor gave you take-home assignments, which acted like validation sets.

You got some wrong and some right. Based on this, you adjusted your topic fundamentals, i.e., improved the model.

Now, if you keep solving the same take-home assignment repeatedly, you will eventually overfit it, won't you?

That is why we bring in a new validation set after some iterations.

The final exam day paper is your test set. If you do well, awesome!

But if you fail, the professor cannot give you the exact exam paper next time, can they? This is because you know what's inside.

Of course, by evaluating a model on the test set, the model never gets to "know" the precise examples inside that set. But the issue is that the test set has been exposed now.

Your previous evaluation will inevitably influence any further evaluations on that specific test set. That is why you must always use a specific test set only ONCE.

Once you do, merge it with the train and validation set and generate an entirely new split.

Repeat.

And that is how you use train, validation, and test sets in machine learning.

5 Cross Validation Techniques

Tuning and validating machine learning models on a single validation set can be misleading and sometimes yield overly optimistic results.

This can occur due to a lucky random split of data, which results in a model that performs exceptionally well on the validation set but poorly on new, unseen data.

That is why we often use cross validation instead of simple single-set validation.

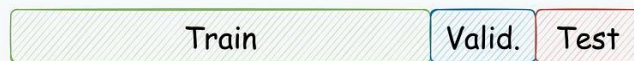
Cross validation involves repeatedly partitioning the available data into subsets, training the model on a few subsets, and validating on the remaining subsets.

The main advantage of cross validation is that it provides a more robust and unbiased estimate of model performance compared to the traditional validation method.

Below are five of the most commonly used and must-know cross validation techniques.

Leave-One-Out Cross Validation

Leave One Out Cross Validation



*One observation is used at a time as validation set

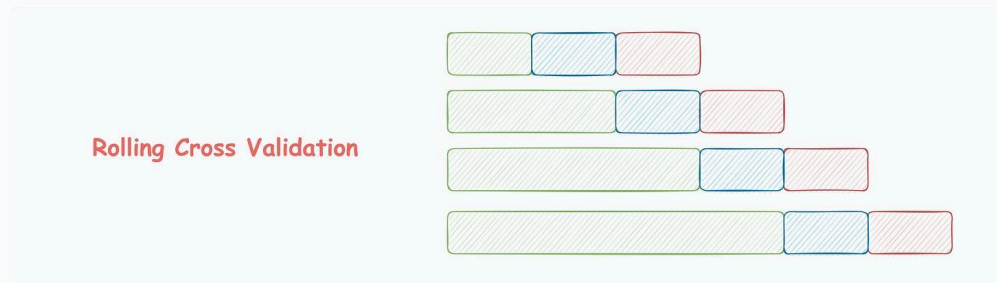
- Leave one data point for validation.
- Train the model on the remaining data points.
- Repeat for all points.
- Of course, as you may have guessed, this is practically infeasible when you have many data points. This is because number of models is equal to number of data points.
- We can extend this to Leave-p-Out Cross Validation, where, in each iteration, p observations are reserved for validation, and the rest are used for training.

K-Fold Cross Validation



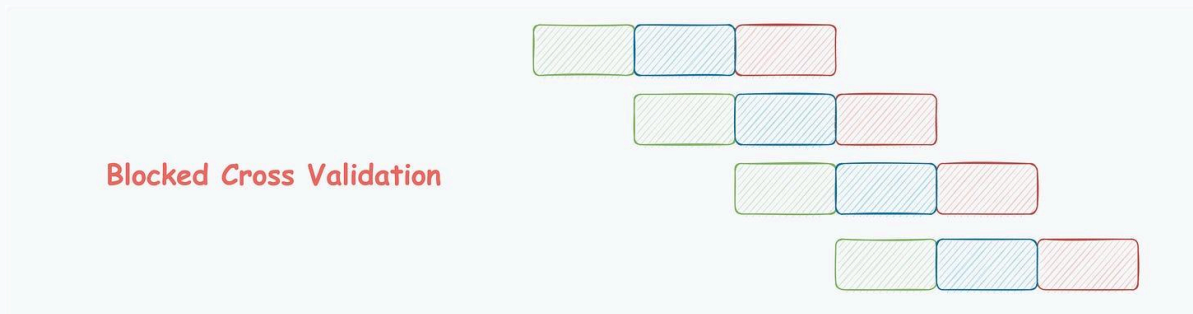
- Split data into k equally-sized subsets.
- Select one subset for validation.
- Train the model on the remaining subsets.
- Repeat for all subsets.

Rolling Cross Validation



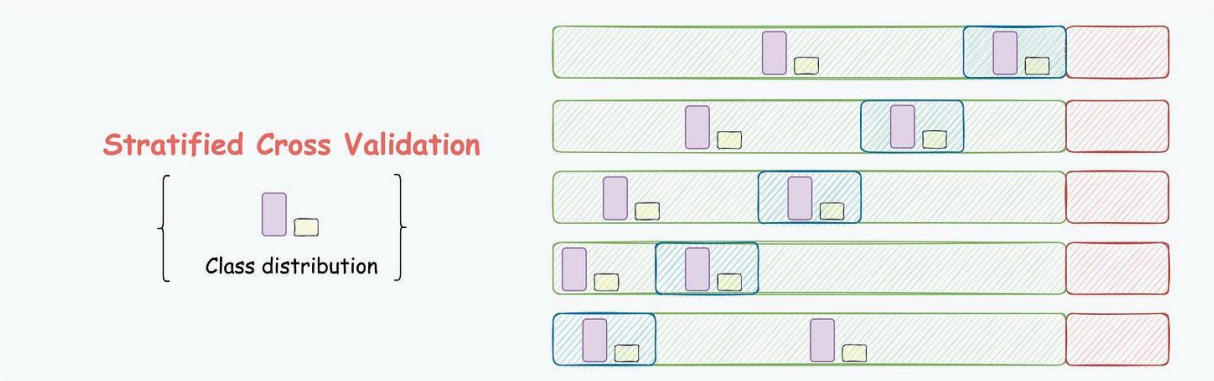
- Mostly used for data with temporal structure.
- Data splitting respects the temporal order, using a fixed-size training window.
- The model is evaluated on the subsequent window.

Blocked Cross Validation



- Another common technique for time-series data.
- In contrast to rolling cross validation, the slice of data is intentionally kept short if the variance does not change appreciably from one window to the next.
- This also saves computation over rolling cross validation.

Stratified Cross Validation



- The above-discussed techniques may not work for imbalanced datasets. Stratified cross validation is mainly used for preserving the class distribution.
- The partitioning ensures that the class distribution is preserved.

Let's continue our discussion on cross validation in the next chapter.

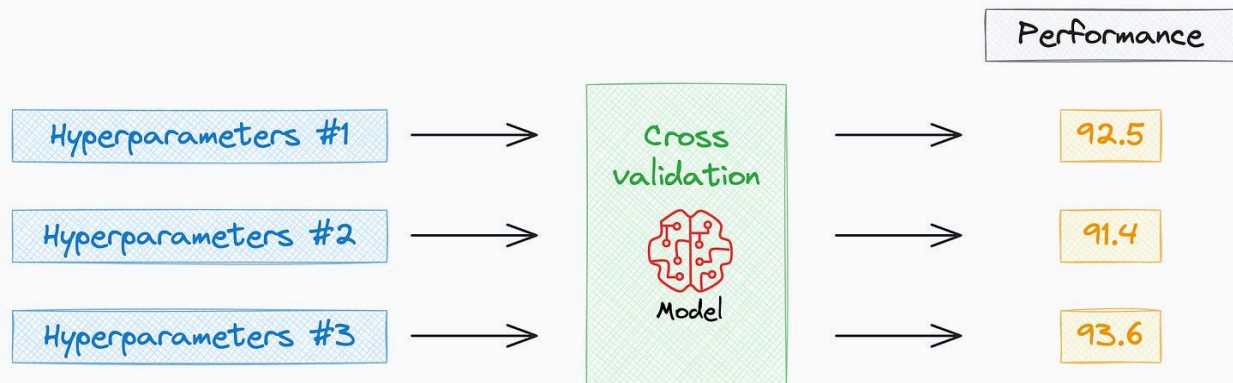
What To Do After Cross Validation?

Let me ask you a question.

But before I do that, I need to borrow your imagination for just a moment.

Imagine you are building some supervised machine learning model. You are using cross-validation to determine an optimal set of hyperparameters.

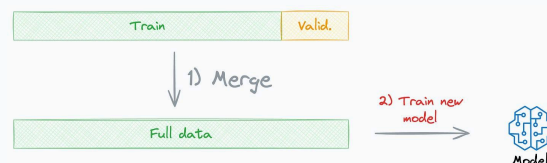
Essentially, every hyperparameter configuration corresponds to a cross-validation performance:



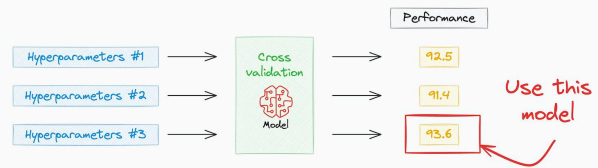
After obtaining the best hyperparameters, we need to finalize a model (say for production); otherwise, what is the point of all this hassle? Now, here's the question:

After obtaining the optimal hyperparameters, what would you be more inclined to do:

1) Retrain the model again on the entire data (train + validation + test) with the optimal hyperparameters? If we do this, remember that we can't reliably validate this new model as there is no unseen data left.



2) Just proceed with the best-performing model based on cross-validation performance itself. If we do this, remember that we are leaving out important data, which we could have trained our model with.



What would you do?

Recommended path

My strong preference has almost always been “retraining a new model with entire data.”



There are, of course, some considerations to keep in mind, which I have learned through the models I have built and deployed. That said, in most cases, retraining is the ideal way to proceed.

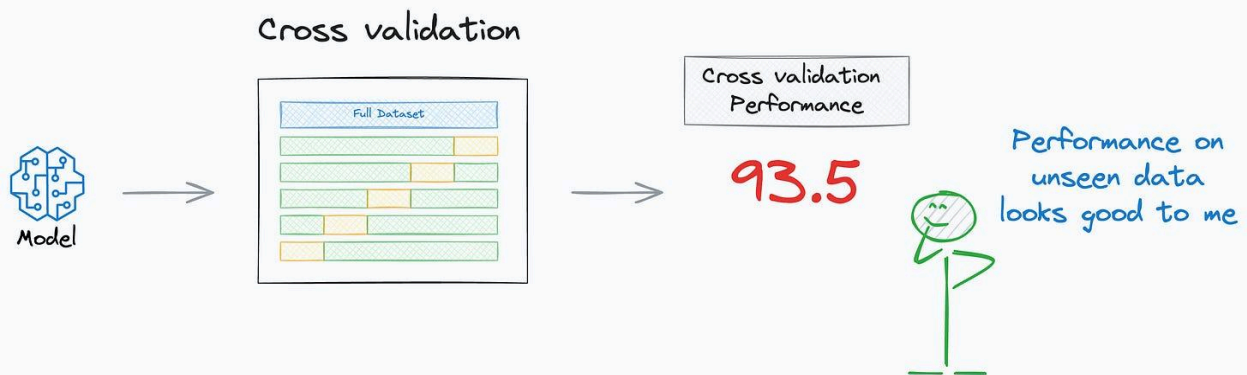
Let me explain.

Why retrain the model?

We would want to retrain a new model because, in a way, we are already satisfied with the cross-validation performance, which, by its very nature, is an out-of-fold metric.

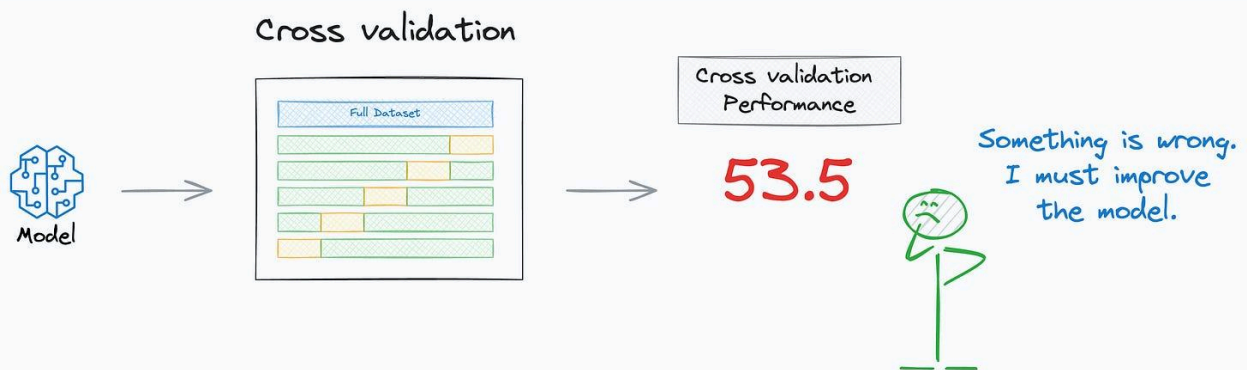
An out-of-fold data is data that has not been seen by the model during the training. An out-of-fold metric is the performance on that data.

In other words, we already believe that the model aligns with how we expect it to perform on unseen data.



Thus, incorporating this unseen validation set in the training data and retraining the model will MOST LIKELY have NO effect on its performance on unseen data after deployment (assuming a sudden covariate shift hasn't kicked in, which is a different issue altogether).

If, however, we were not satisfied with the cross-validation performance itself, we wouldn't even be thinking about finalizing a model in the first place.

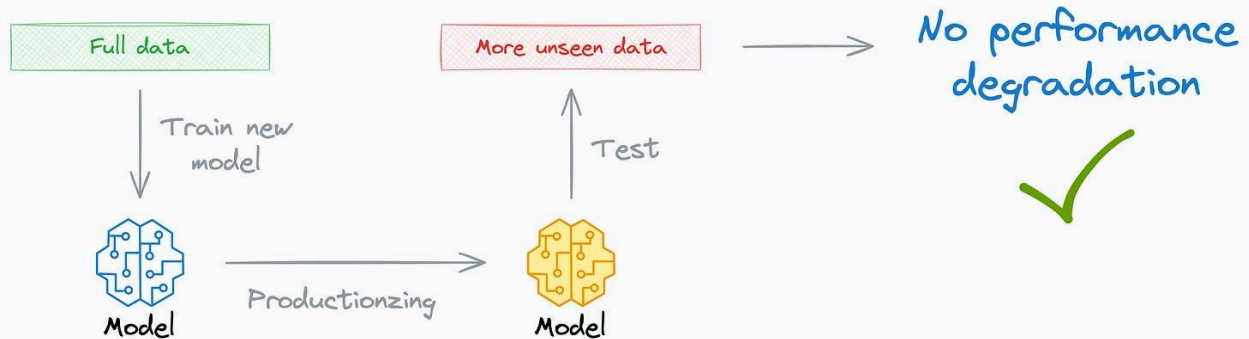


Instead, we would be thinking about ways to improve the model by working on feature engineering, trying new hyperparameters, experimenting with different models, and more.

The reasoning makes intuitive sense as well.

It's hard for me to recall any instance where retraining did something disastrously bad to the overall model.

In fact, I vividly remember one instance wherein, while I was productionizing the model (it took me a couple of days after retraining), the team had gathered some more labeled data.

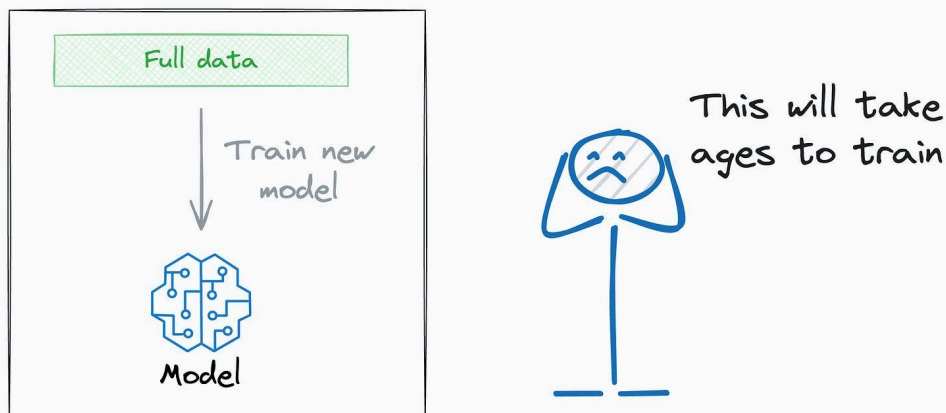


The model didn't show any performance degradation when I evaluated it (just to double-check). As an added benefit, this also helped ensure that I had made no errors while productionizing my model.

Some considerations

Here, please note that it's not a rule that you must always retrain a new model.

The field itself and the tasks one can solve are pretty diverse, so one must be open-minded while solving the problem at hand. One of the reasons I wouldn't want to retrain a new model is that it takes days or weeks to train the model.



In fact, even if we retrain a new model, there are MANY business situations in which stakes are just too high.

Thus, one can never afford to be negligent about deploying a model without re-evaluating it — transactional fraud, for instance.

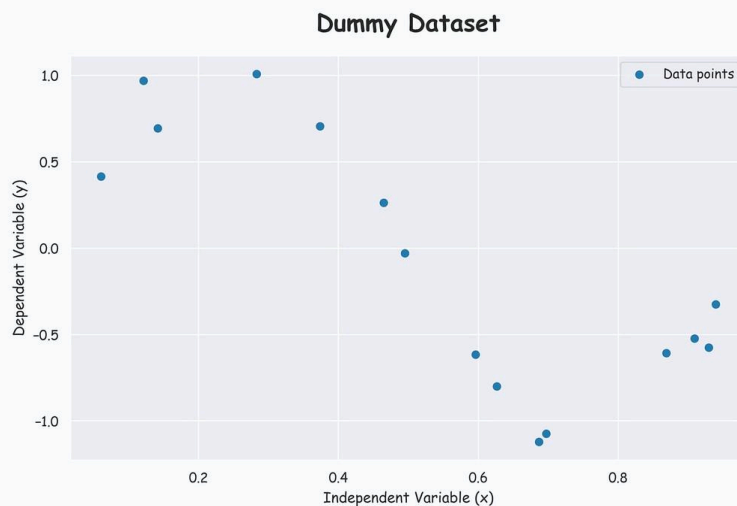
In such cases, I have seen that while a team works on productionizing the model, data engineers gather some more data in the meantime.

Before deploying, the team would do some final checks on that dataset.

The newly gathered data is then considered in the subsequent iterations of model improvements.

Double Descent vs. Bias-Variance Trade-off

It is well-known that as the number of model parameters increases, we typically overfit the data more and more. For instance, consider fitting a polynomial regression model trained on this dummy dataset below:



Dummy
regression
dataset

In case you don't know, this is called a polynomial regression model:

Polynomial
regression model

$$y = \theta_0 + \theta_1 * x^1 + \theta_2 * x^2 + \dots + \theta_m * x^m$$

It is expected that as we'll increase the degree (m) and train the polynomial regression model:

- The training loss will get closer and closer to zero.
- The test (or validation) loss will first reduce and then get bigger and bigger.

This is because, with a higher degree, the model will find it easier to contort its regression fit through each training data point, which makes sense.

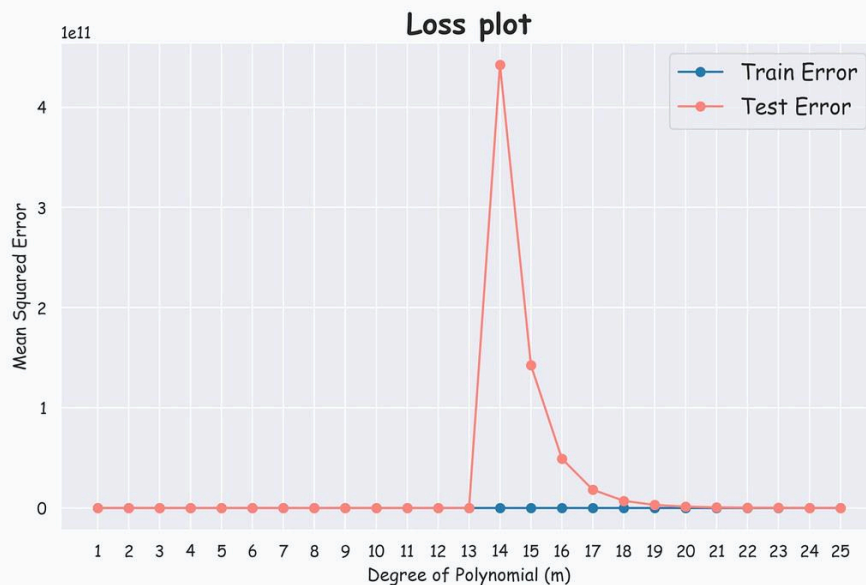
In fact, this is also evident from the following loss plot:



Test loss first
decreases then
increases

Train loss
always
decreases

But notice what happens when we continue to increase the degree (m):



Test loss
decreases
again

That's strange, right?

Why does the test loss increase to a certain point but then decrease?

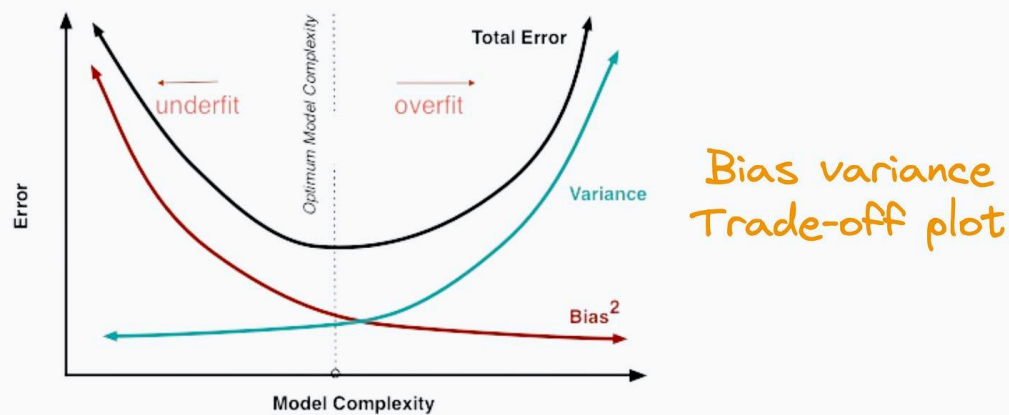
This was not expected, was it?

Well...what you are seeing is called the “double descent phenomenon,” which is quite commonly observed in many ML models, especially deep learning models.

It shows that, counterintuitively, increasing the model complexity beyond the point of interpolation can improve generalization performance.

In fact, this whole idea is deeply rooted to why LLMs, although massively big (billions or even trillions of parameters), can still generalize pretty well.

And it’s hard to accept it because this phenomenon directly challenges the traditional bias-variance trade-off we learn in any introductory ML class:



Putting it another way, training very large models, even with more parameters than training data points, can still generalize well.

To the best of my knowledge, this is still an open question, and it isn’t entirely clear why neural networks exhibit this behavior.

There are some theories around regularization, however, such as this one:

It could be that the model applies some sort of implicit regularization, with which, it can precisely focus on an apt number of parameters for generalization.

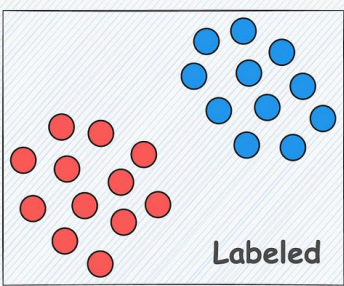
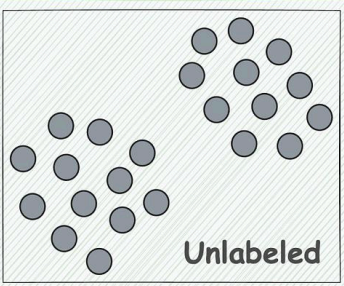
But to be honest, nothing is clear yet.

Statistical Foundations

MLE vs. EM — What's the Difference?

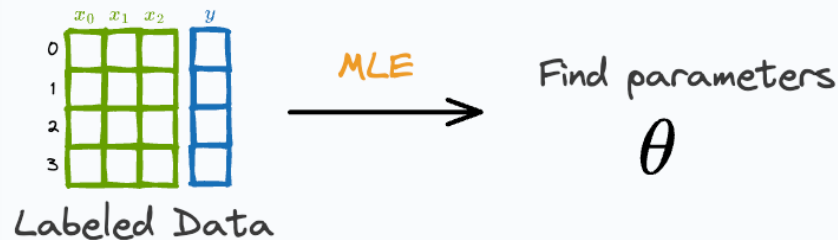
Maximum likelihood estimation (MLE) and expectation maximization (EM) are two popular techniques to determine the parameters of statistical models.

Due to its applicability in MANY statistical models, I have seen it being asked in plenty of data science interviews as well, especially the distinction between the two. The following visual summarizes how they work:

Maximum Likelihood Estimation vs. Expectation Maximization		blog.DailyDoseofDS.com
	Maximum Likelihood Estimation	Expectation Maximization
Type of Data	 Labeled	 Unlabeled
Objective	Find parameters θ that maximize the likelihood.	Find parameters θ that maximize 'full data' likelihood.
True Labels	Known (or observed).	Unknown (or unobserved).
Convergence	One-step optimization (typically).	Iterative convergence by alternating b/w E and M step.
Final Output	The parameters θ .	θ and estimated latent (or unobserved) variable Z .
Used in	Logistic Regression Linear Regression ?	Gaussian Mixture Models Hidden Markov Models ?

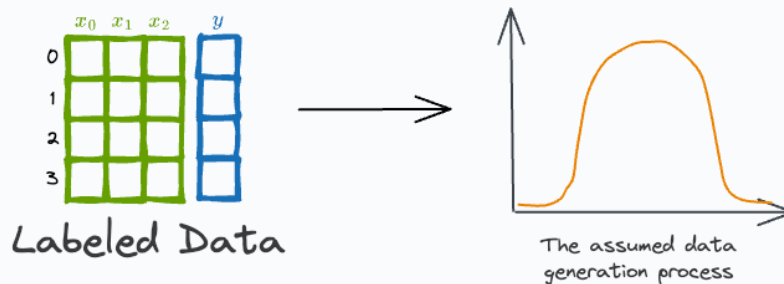
Maximum likelihood estimation (MLE)

MLE starts with a labeled dataset and aims to determine the parameters of the statistical model we are trying to fit.

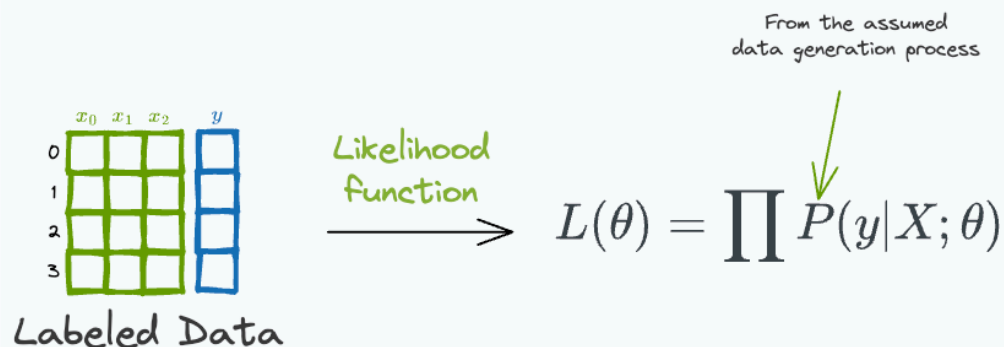


The process is pretty simple and straightforward. In MLE, we:

- Start by assuming a data generation process. Simply put, this data generation process reflects our belief about the distribution of the output label (y), given the input (X).



- Next, we define the likelihood of observing the data. As each observation is independent, the likelihood of observing the entire data is the same as the product of observing individual observations:



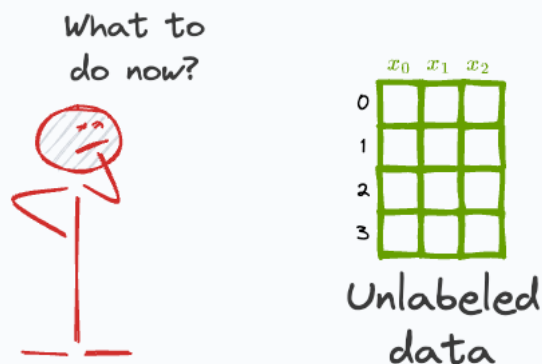
- The likelihood function above depends on parameter values (θ). Our objective is to determine those specific parameter values that maximize the likelihood function. We do this as follows:

$$\frac{\delta L(\theta)}{\delta \theta} = 0 \quad \longrightarrow \quad \text{Solve for } \theta$$

This gives our parameter estimates that would have most likely generated the given data.

That was pretty simple, wasn't it?

But what do we do if we don't have true labels? We still want to estimate the parameters, don't we?

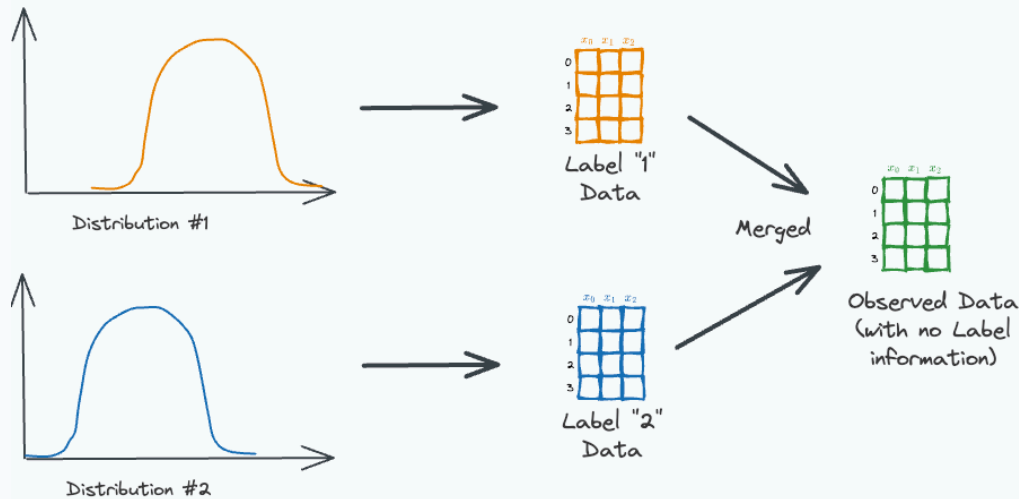


MLE, as you may have guessed, will not be applicable. The true label (y), being unobserved, makes it impossible to define a likelihood function like we did earlier.

In such cases, advanced techniques like expectation maximization are pretty helpful.

Expectation maximization (EM)

EM is an iterative optimization technique to estimate the parameters of statistical models. It is particularly useful when we have an unobserved (or hidden) label. One example situation could be as follows:



As depicted above, we assume that the data was generated from multiple distributions (a mixture). However, the observed/complete data does not contain that information. In other words, the observed dataset does not have information about whether a specific row was generated from distribution 1 or distribution 2.

Had it contained the label (y) information, we would have already used MLE.

EM helps us with parameter estimates of such datasets. The core idea behind EM is as follows:

- Make a guess about the initial parameters (θ).
- Expectation (E) step: Compute the posterior probabilities of the unobserved label (let's call it ' z ') using the above parameters.

$$P(z = 1|X; \theta)$$

$$P(z = 0|X; \theta)$$

- Here, 'z' is also called a latent variable, which means hidden or unobserved.
- Relating it to our case, we know that the true label exists in nature. But we don't know what it is.
- Thus, we replace it with a latent variable 'z' and estimate its posterior probabilities using the guessed parameters.
- Given that we now have a proxy (not precise, though) for the true label, we can define an “expected likelihood” function. Thus, we use the above posterior probabilities to do so:

$$\begin{array}{ccc}
 & \text{dependent} & \\
 & \text{on} & \\
 L(\theta) & \longrightarrow & \begin{array}{l} P(z = 1|X; \theta) \\ P(z = 0|X; \theta) \end{array} \\
 \text{Expected} & & \\
 \text{Likelihood} & & \\
 \text{function} & &
 \end{array}$$

- Maximization (M) step: So now we have a likelihood function to work with. Maximizing it with respect to the parameters will give us a new estimate for the parameters (θ').
- Next, we use the updated parameters (θ') to recompute the posterior probabilities we defined in the expectation step.

$$\begin{array}{c}
 P(z = 1|X; \theta') \\
 P(z = 0|X; \theta')
 \end{array}$$

- We will update the likelihood function (L) using the new posterior probabilities.
- Again, maximizing it will give us a new estimate for the parameters (θ).
- And this process goes on and on until convergence.

The point is that in expectation maximization, we repeatedly iterate between the E and the M steps until the parameters converge. A good thing about EM is that it always converges. Yet, at times, it might converge to a local extrema.

MLE vs. EM is a popular question asked in many data science interviews.

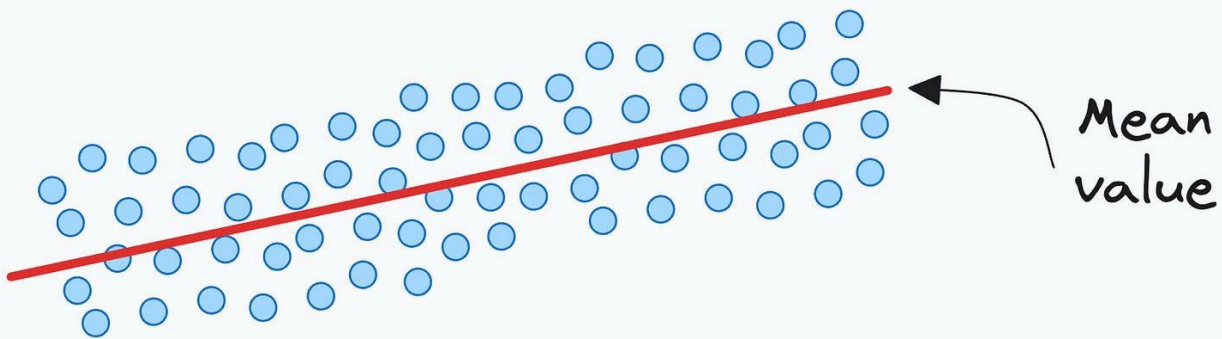
Confidence Interval and Prediction Interval

Statistical estimates always have some uncertainty.

For instance, a linear regression model never predicts an actual value.

Consider a simple example of modeling house prices just based on its area. A prediction wouldn't tell the true value of a house based on its area. This is because different houses of the same size can have different prices.

Instead, what it predicts is the **mean value** related to the outcome at a particular input.



The point is...

There's always some uncertainty involved in statistical estimates, and it is important to communicate it.

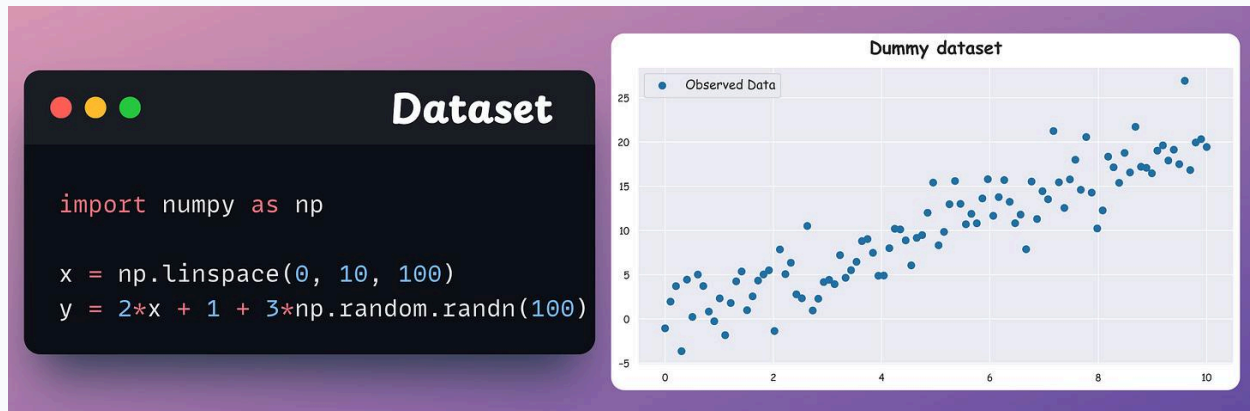
In this specific case, there are two types of uncertainties:

- The uncertainty in estimating the true mean value.
- The uncertainty in estimating the true value.

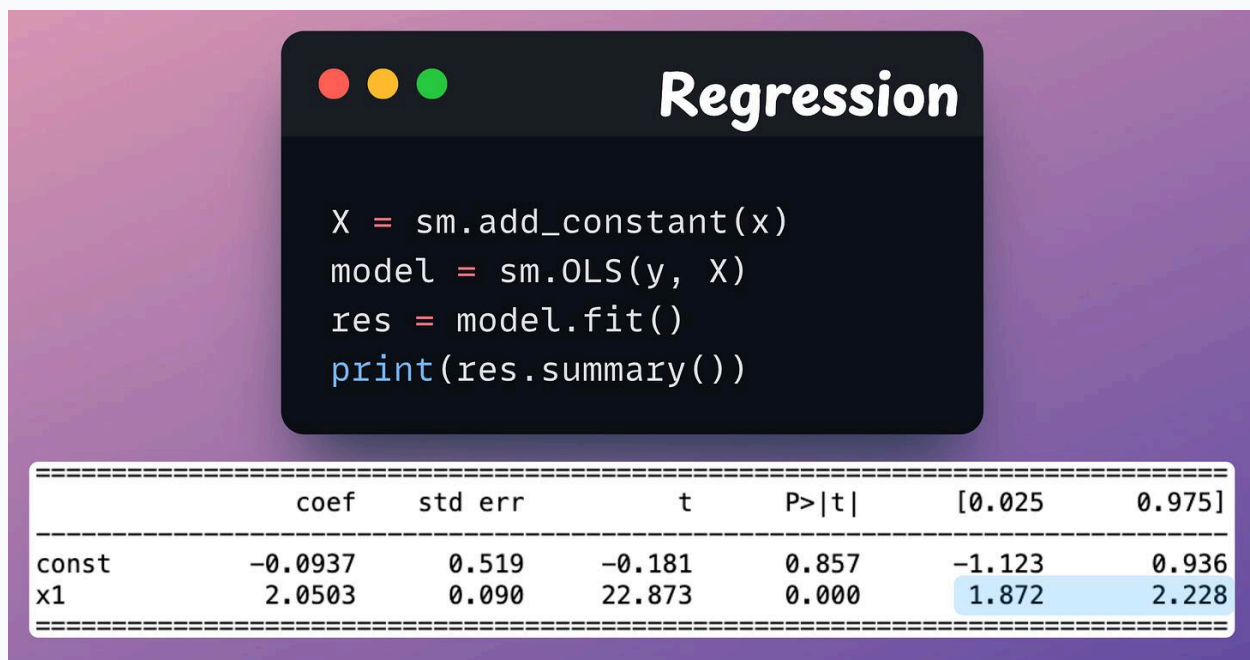
Confidence interval and prediction interval help us capture these uncertainties.

Let's understand.

Consider the following dummy dataset:



Let's fit a linear regression model using statsmodel and print a part of the regression summary:

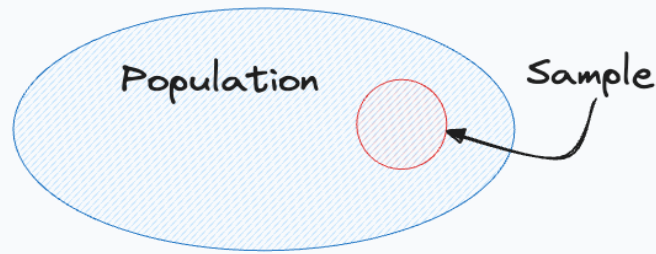


Notice that the coefficient of the predictor “x1” is 2.0503 with a 95% interval of [1.872, 2.228].

It is a 95% interval because $0.975 - 0.025 = 0.95$.

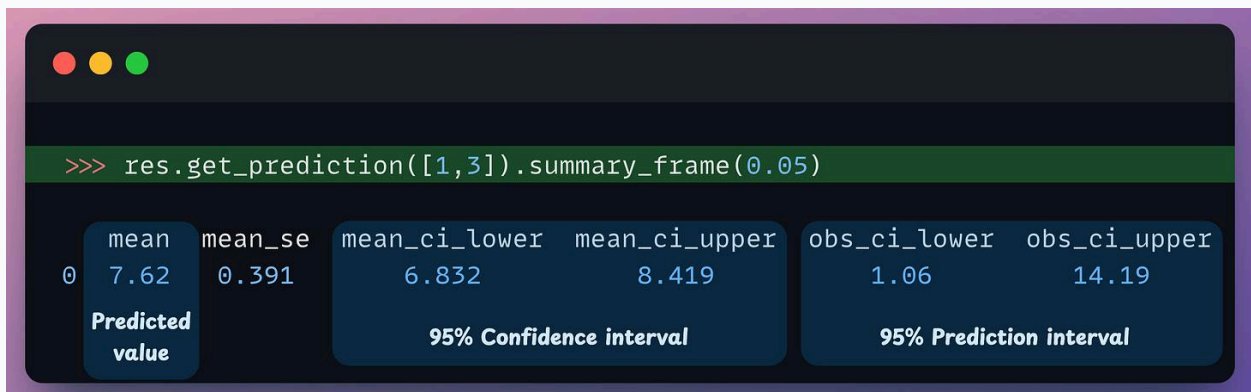
This is known as the confidence interval, which comes from **sampling uncertainty**.

More specifically, this uncertainty arises because the data we used above for modeling is just a **sample** of the population.



So, if we gathered more such samples and fit an OLS to each sample, the true coefficient (which we can only know if we had the data for the entire population) would lie 95% of the time in this confidence interval.

Next, we use this model to make a prediction as follows:



- The predicted value is 7.62 (mean).
- The 95% confidence interval is [6.832, 8.419].
- The 95% prediction interval is [1.06, 14.19].

The confidence interval we saw above was for the coefficient, so what does the confidence interval represent in this case?

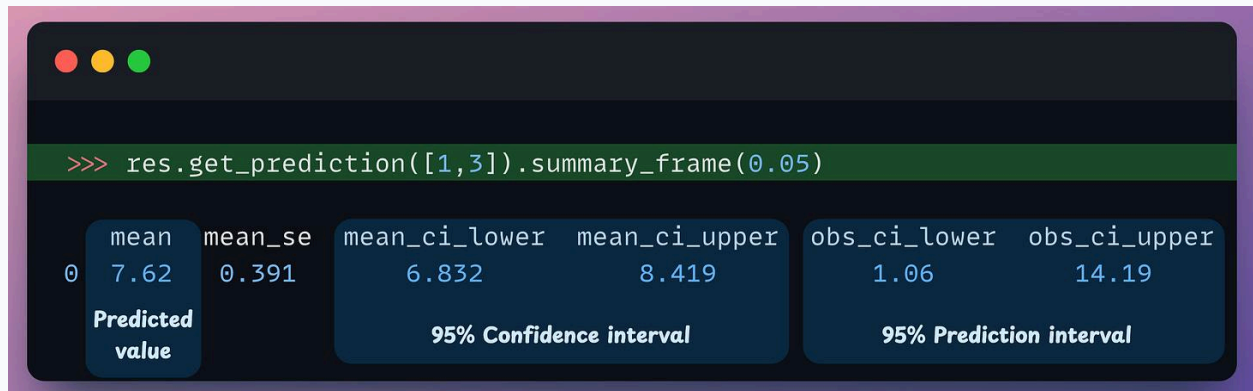
Similar to what we discussed above, the data is just a **sample** of the population.

The regression fit obtained by this sample produced a prediction (some mean value) for the input $x=3$.

However, if we gathered more such samples and fit an OLS to each dataset, the **true mean value** (which we can only know if we had the data for the entire

population) for this specific input ($x=3$) would lie 95% of the time in this confidence interval.

Coming to the prediction interval...

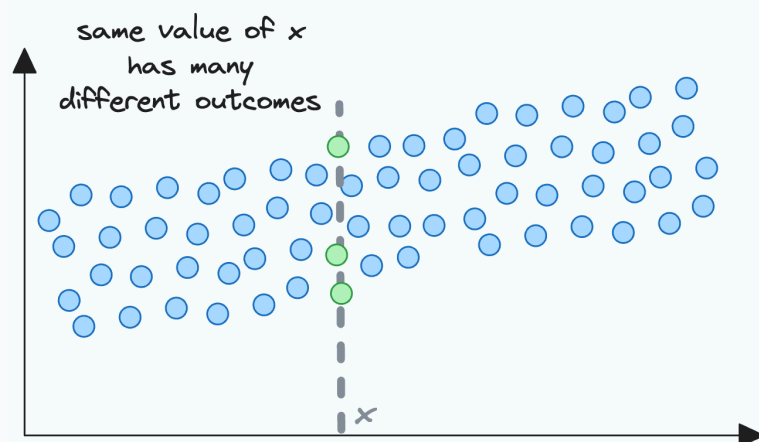


...we notice that it is wider than the confidence interval. Why is it, and what does this interval tell?

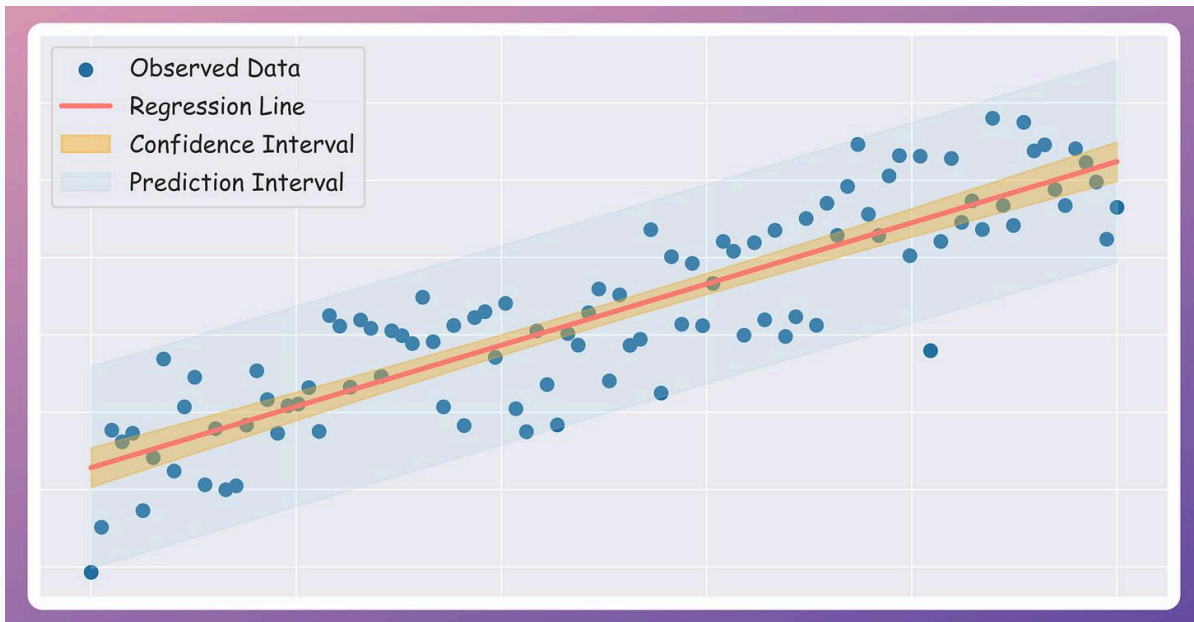
What we saw above with confidence interval was about estimating the true population mean at a specific input.

What we are talking about now is obtaining an interval where the **true value** for an input can lie.

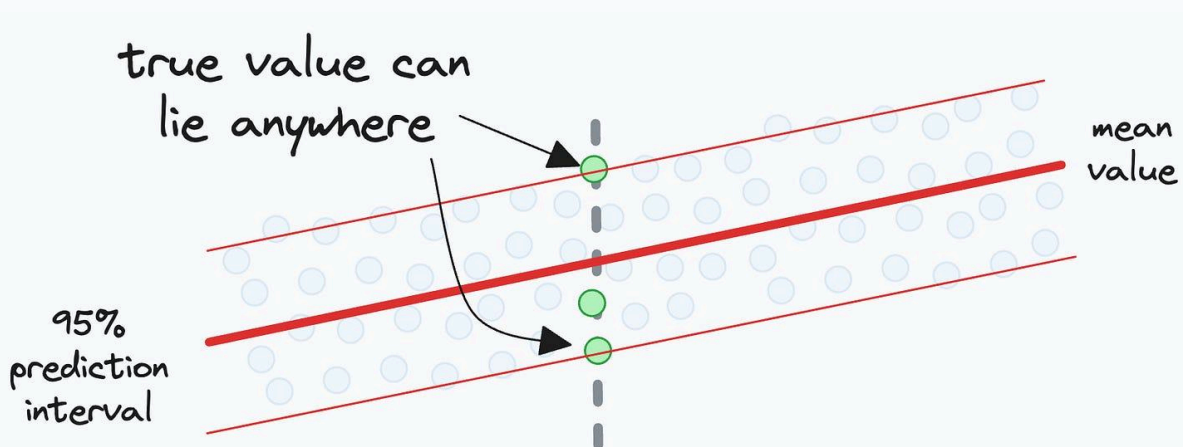
Thus, this additional uncertainty appears because in our dataset, for the same value of input x , there can be multiple different values of the outcome. This is depicted below:



Thus, it is wider than the confidence interval. Plotting it across the entire input range, we get the following plot:



Given that the model is predicting a mean value (as depicted below), we have to represent the prediction uncertainty that the actual value can lie anywhere in the prediction interval:



A 95% prediction interval tells us that we can be 95% sure that the actual value of this observation will fall within this interval.

So to summarize:

- A confidence interval captures the sampling uncertainty. More data means less sampling uncertainty, which in turn leads to a smaller interval.
- In addition to the sampling uncertainty, the prediction interval also represents the uncertainty in estimating the true value of a particular data point. Thus, it is wider than the confidence interval.

Communicating these uncertainties is quite crucial in decision-making because it provides a clearer understanding of the reliability and precision of predictions.

This transparency allows stakeholders to make more informed decisions by considering the range of possible outcomes and the associated risks.

Why is OLS Called an Unbiased Estimator?

The OLS estimator for linear regression (shown below) is known as an unbiased estimator.

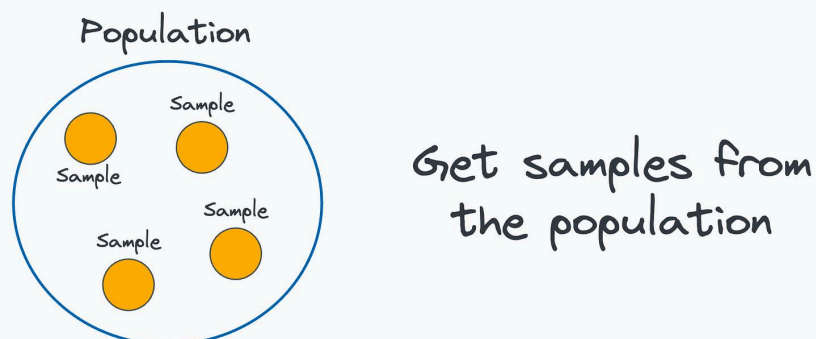
$$\theta = (X^T X)^{-1} X^T y$$

- What do we mean by that?
- Why is OLS called such?

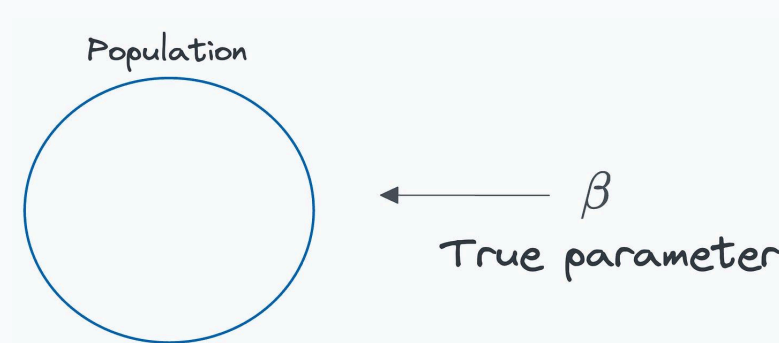
Background

The goal of statistical modeling is to make conclusions about the whole population.

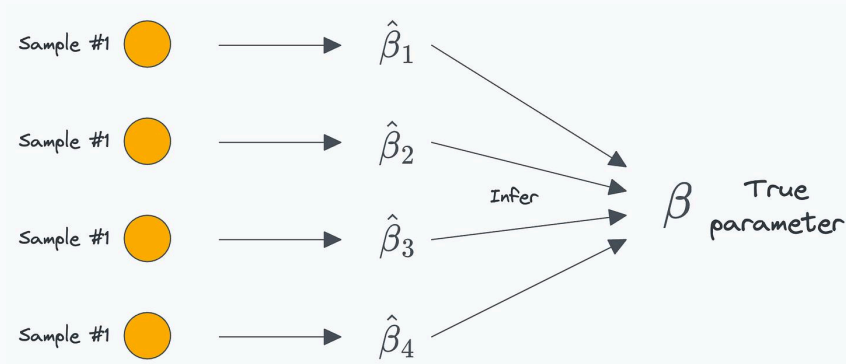
However, it is pretty obvious that observing the entire population is impractical.



In other words, given that we cannot observe (or collect data of) the entire population, we cannot obtain the true parameter (β) for the population:



Thus, we must obtain parameter estimates ($\hat{\beta}$) on samples and infer the true parameter (β) for the population from those estimates:



And, of course, we want these sample estimates ($\hat{\beta}$) to be reliable to determine the actual parameter (β).

The OLS estimator ensures that.

Let's understand how!

True population model

When using a linear regression model, we assume that the response variable (Y) and features (X) for the entire population are related as follows:

$$Y = \beta * X + \epsilon$$

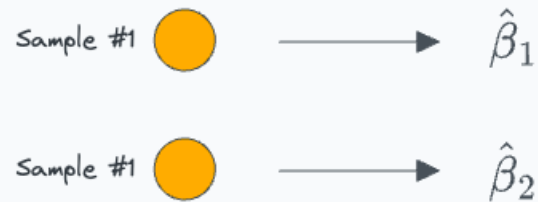
- β is the true parameter that we are not aware of.
- ϵ is the error term.

Expected Value of OLS Estimates

The closed-form solution of OLS is given by:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

What's more, as discussed above, using OLS on different samples will result in different parameter estimates:



Let's find the expected value of OLS estimates $E[\hat{\beta}]$.

Simply put, the expected value is the average value of the parameters if we run OLS on many samples.

This is given by:

$$E[\hat{\beta}] = E[(X^T X)^{-1} X^T Y]$$

Substitute $\hat{\beta}$ as the OLS solution

Here, substitute $Y = \beta X + \epsilon$:

$$\begin{aligned} E[\hat{\beta}] &= E[(X^T X)^{-1} X^T Y] \\ &= E[(X^T X)^{-1} X^T (\beta X + \epsilon)] \end{aligned}$$

If you are wondering how we can substitute $Y = \beta X + \epsilon$ when we don't know what β is, then here's the explanation:

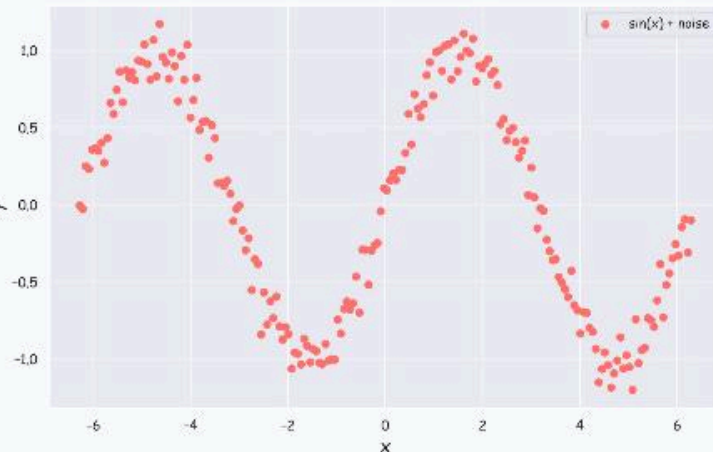
See, we can do that substitution because even if we don't know the parameter β for the whole population, we know that the sample was drawn from the population.

Thus, the equation in terms of the true parameters ($Y = \beta X + \epsilon$) still holds for the sample.

Let me give you an example.

Say the population data was defined by $y = \sin(x) + \epsilon$. Of course, we wouldn't know this, but just keep that aside for a second.

$$y = \sin(x) + \epsilon$$



Now, even if we were to draw samples from this population data, the true equation $y = \sin(x) + \epsilon$ would still be valid on the sampled data points, wouldn't it?

The same idea has been extended for expected value.

Coming back to the following:

$$\begin{aligned} E[\hat{\beta}] &= E[(X^T X)^{-1} X^T Y] \\ &= E[(X^T X)^{-1} X^T (\beta X + \epsilon)] \end{aligned}$$

Let's open the inner parenthesis:

$$E[\hat{\beta}] = E[\underbrace{(X^T X)^{-1} X^T X}_{\downarrow I} \beta] + E[\underbrace{(X^T X)^{-1} X^T}_{\text{Some constant}} \epsilon]$$

Simplifying, we get:

$$E[\hat{\beta}] = E[\beta] + (X^T X)^{-1} X^T \underbrace{E[\epsilon]}_{\text{zero}}$$

β is constant so its expected value is β

And finally, what do we get?

$$E[\hat{\beta}] = \beta$$

The expected value of parameter estimates on the samples equals the true parameter value β .

And this is precisely what the definition of an unbiased estimator is.

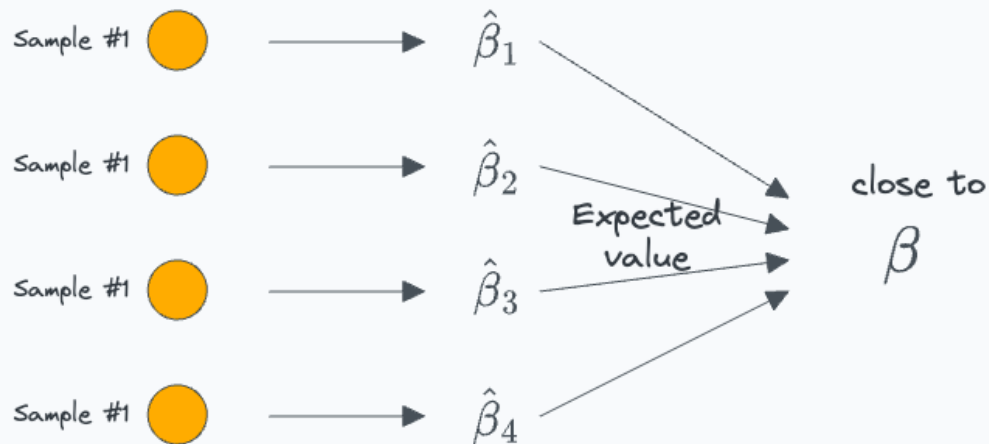
More formally, an estimator is called unbiased if the expected value of the parameters is equal to the actual parameter value.

And that is why we call OLS an unbiased estimator.

An important takeaway

Many people misinterpret unbiasedness with the idea that the parameter estimates from a single run of OLS on a sample are equal to the true parameter values.

Don't make that mistake.



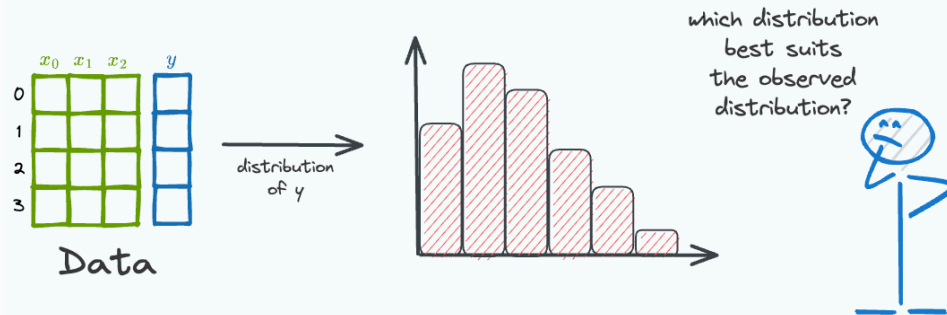
Instead, unbiasedness implies that if we were to generate OLS estimates on many different samples (drawn from the same population), then the expected value of obtained estimates will be equal to the true population parameter.

And, of course, all this is based on the assumption that we have good representative samples and that the assumptions of linear regression are not violated.

Bhattacharyya Distance

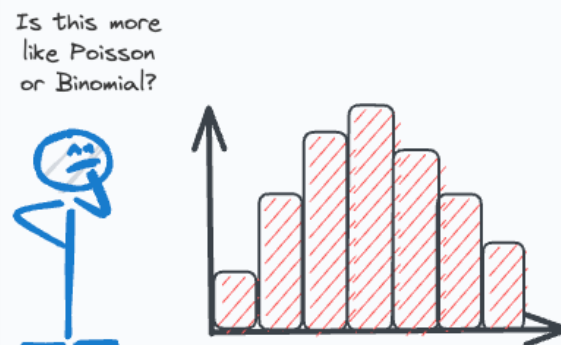
Assessing the similarity between two probability distributions is quite helpful at times. For instance, imagine we have a labeled dataset (X, y) .

By analyzing the label (y) distribution, we may hypothesize its distribution before building a statistical model.



We also looked at this in an earlier chapter on [generalized linear models \(GLMs\)](#).

While visual inspection is often helpful, this approach is quite subjective and may lead to misleading conclusions.



Thus, it is essential to be aware of quantitative measures as well. Bhattacharyya distance is one such reliable measure.

It quantifies the similarity between two probability distributions.

The core idea is to approximate the overlap between two distributions, which measures the “closeness” between the two distributions under consideration.

Bhattacharyya distance is measured as follows:

For two discrete probability distributions

$$D_B(P, Q) = -\ln \left(\sum_{x \in X} \sqrt{P(x) \cdot Q(x)} \right)$$

For two continuous probability distributions (replace summation with an integral):

$$D_B(P, Q) = -\ln \left(\int \sqrt{P(x) \cdot Q(x)} dx \right)$$

Its effectiveness is evident from the image below.

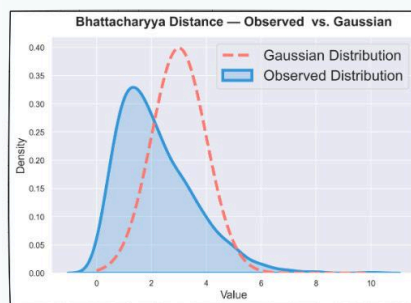
Here, we have an observed distribution (Blue). Next, we measure its distance from:

- Gaussian $\rightarrow 0.19$.
- Gamma $\rightarrow 0.03$.

A high Bhattacharyya distance indicates less overlap or more dissimilarity. This lets us conclude that the observed distribution resembles a Gamma distribution.

Bhattacharyya Distance: Explained Visually

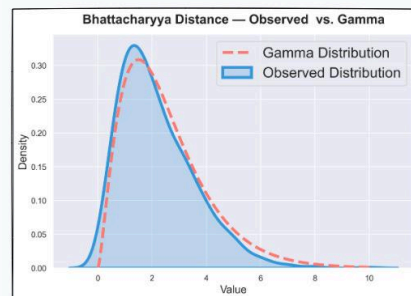
 blog.DailyDoseofDS.com



Bhattacharyya Distance **0.19**
(high)

Conclusion

✗ Distributions are less similar



Bhattacharyya Distance **0.03**
(low)

Conclusion

✓ Distributions are more similar

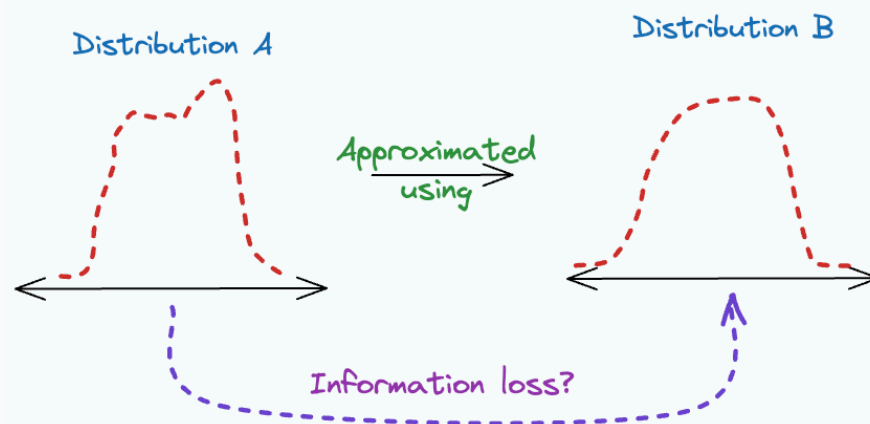
The results also resonate with visual inspection.

KL Divergence vs. Bhattacharyya distance

Now, many often get confused between KL Divergence and Bhattacharyya distance. Effectively, both are quantitative measures to determine the “similarity” between two distributions.

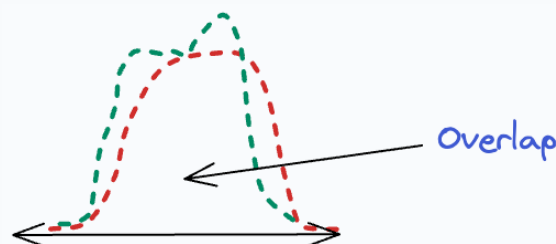
However, their notion of “similarity” is entirely different.

The core idea behind KL Divergence is to assess how much information is lost when one distribution is used to approximate another distribution.



The more information is lost, the more the KL Divergence and, consequently, the less the “similarity”. Also, approximating a distribution Q using P may not be the same as doing the reverse — P using Q. This makes KL Divergence asymmetric in nature.

Moving on, Bhattacharyya distance measures the overlap between two distributions.



This “overlap” is often interpreted as a measure of closeness (or distance) between the two distributions under consideration. Thus, Bhattacharyya distance primarily serves a distance metric, like Euclidean, for instance.

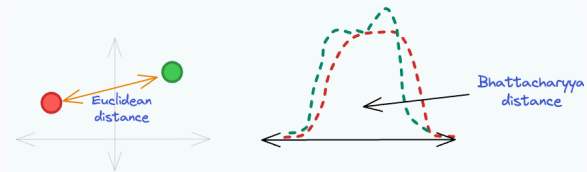
Being a distance metric, it is symmetric in nature. We can also verify this from the distance formula:

$$D_B(P, Q) = -\ln \left(\int \sqrt{P(x) \cdot Q(x)} dx \right)$$

$$\Downarrow$$

$$D_B(P, Q) = D_B(Q, P) \quad \text{Symmetric}$$

Just like we use Euclidean distance to find the distance between two points, we can use Bhattacharyya distance to find the distance between two distributions.



But if we intend to measure the amount of information lost when we approximate one distribution using another, KL divergence is more apt. In fact, KL divergence also serves as a loss function in machine learning algorithms at times (in t-SNE).

- Say we have an observed distribution (P) and want to approximate it with another simpler distribution Q.
- So, we can define a simpler parametric distribution Q.
- Next, we can measure the information lost by approximating P using Q with KL divergence.
- As we want to minimize the information lost, we can use KL divergence as our objective function.
- Finally, we can use gradient descent to determine the parameters of Q such that we minimize the KL divergence.

Bhattacharyya distance has many applications, not just in machine learning but in many other domains. For instance:

- Using this distance, we can simplify complex distributions to simple ones if the distance is low.
- In image processing, Bhattacharyya distance is often used for image matching. By comparing the color or texture distributions of images, it helps identify similar objects or scenes, etc.

The only small caveat is that Bhattacharyya distance does not satisfy the triangle inequality, so that's something to keep in mind.

Why Prefer Mahalanobis Distance Over Euclidean distance?

During distance calculation, Euclidean distance assumes independent axes.

All features (axes)
are independent

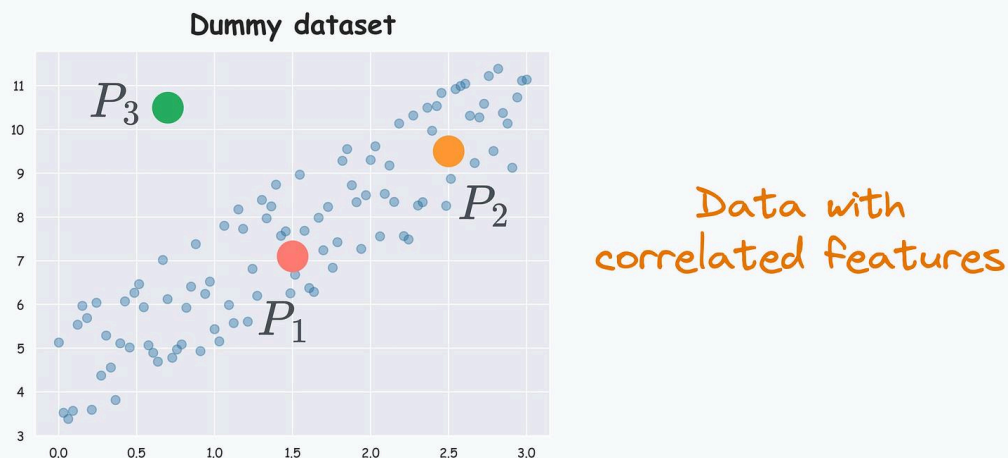
Euclidean
distance

$$\|x - y\|^2 = (x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_k - y_k)^2$$

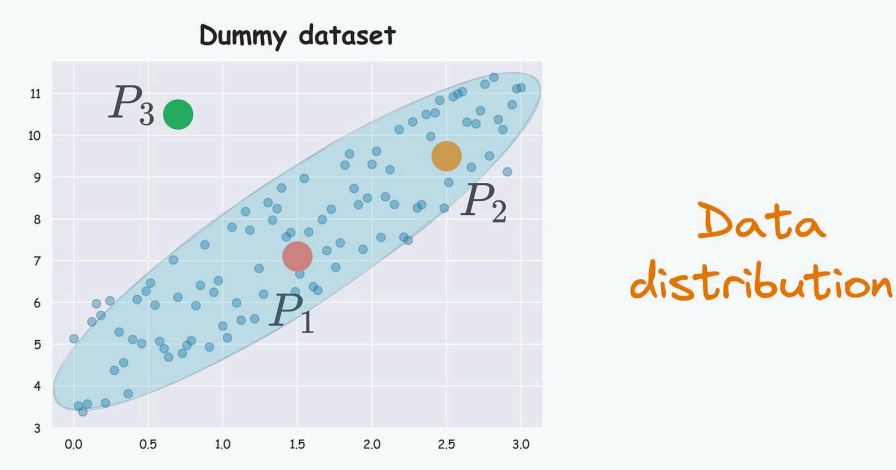
Thus, Euclidean distance will produce misleading results if your features are correlated. For instance, consider this dummy dataset below:



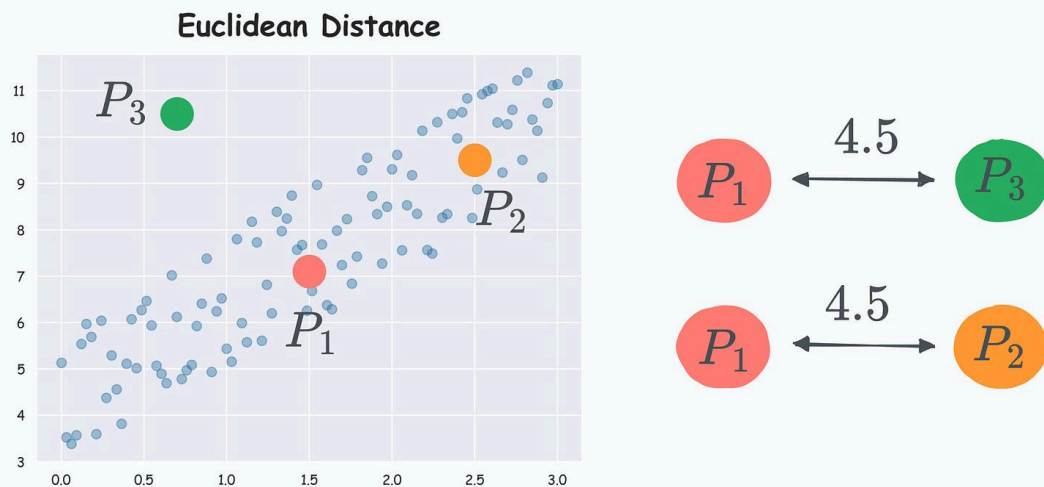
Clearly, the features are correlated. Here, consider three points marked P1, P2, and P3 in this dataset.



Considering the data distribution, something tells us that P_2 is closer to P_1 than P_3 . This is because P_2 lies more within the data distribution than P_3 .



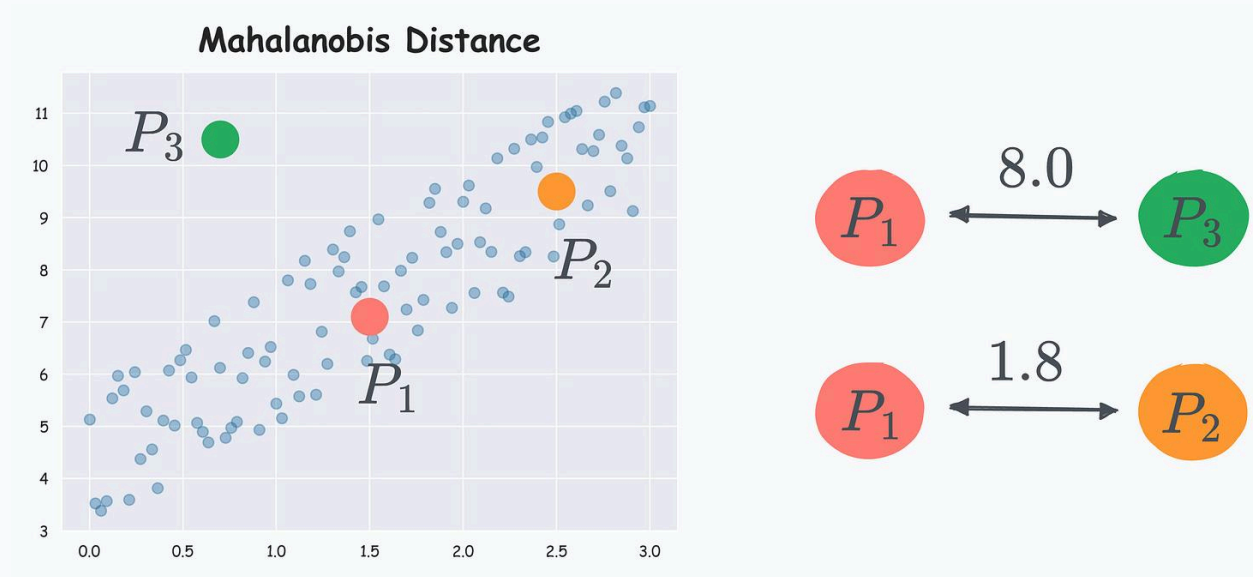
Yet, Euclidean distance ignores this, and P_2 and P_3 come out to be equidistant to P_1 , as depicted below:



Mahalanobis distance addresses this limitation. It is a distance metric that takes into account the data distribution.

As a result, it can measure how far away a data point is from the distribution, which Euclidean can not.

Referring to the earlier dataset again, with Mahalanobis distance, P_2 comes out to be closer to P_1 than P_3 .



How does it work?

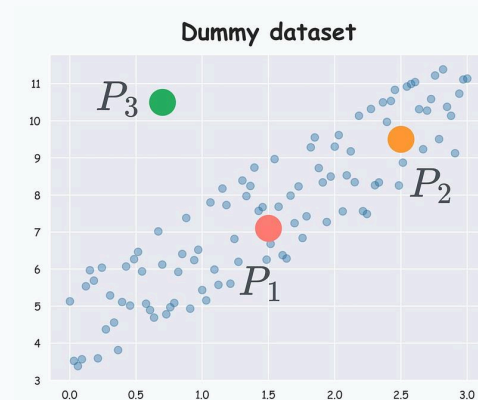
In a gist, the objective is to construct a new coordinate system with independent and orthogonal axes. The steps are:

- Step 1: Transform the columns into uncorrelated variables.
- Step 2: Scale the new variables to make their variance equal to 1.
- Step 3: Find the Euclidean distance in this new coordinate system.

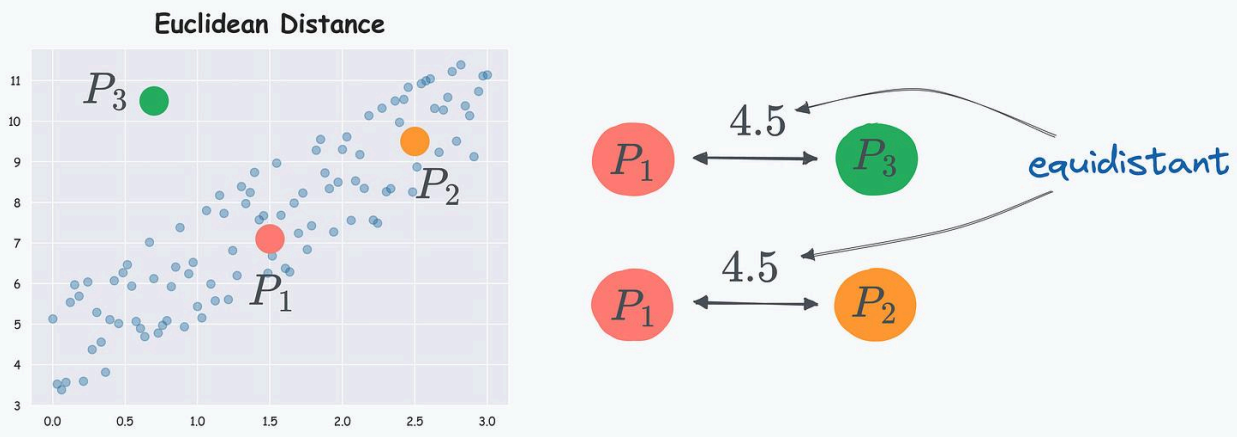
So, eventually, we do use Euclidean distance. However, we first transform the data to ensure that it obeys the assumptions of Euclidean distance.

Uses

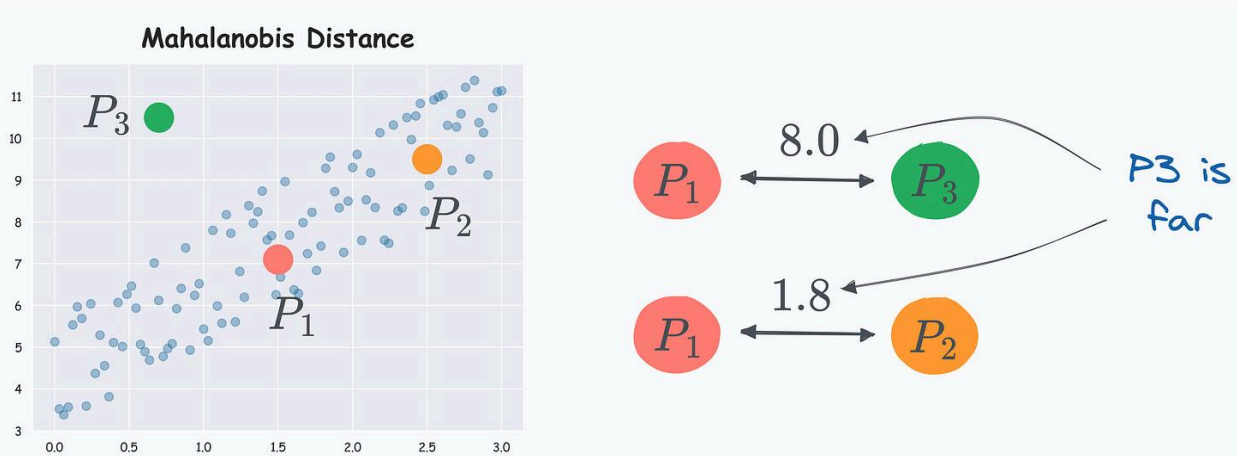
One of the most common use cases of Mahalanobis distance is outlier detection. Reconsidering the dataset we discussed earlier where P_3 is clearly an outlier.



If we consider P_1 as the distribution's centroid and use Euclidean distance, we will infer that P_3 is not an outlier as both P_2 and P_3 are equidistant to P_1 .



Using Mahalanobis distance, however, provides a clearer picture:



This becomes more useful in a high-dimensional setting where visualization is infeasible.

Another use case we typically do not hear of often, but that exists is a variant of kNN that is implemented with Mahalanobis distance instead.

Scipy implements the Mahalanobis distance, which you can check here:

<https://bit.ly/3LjAymm>.

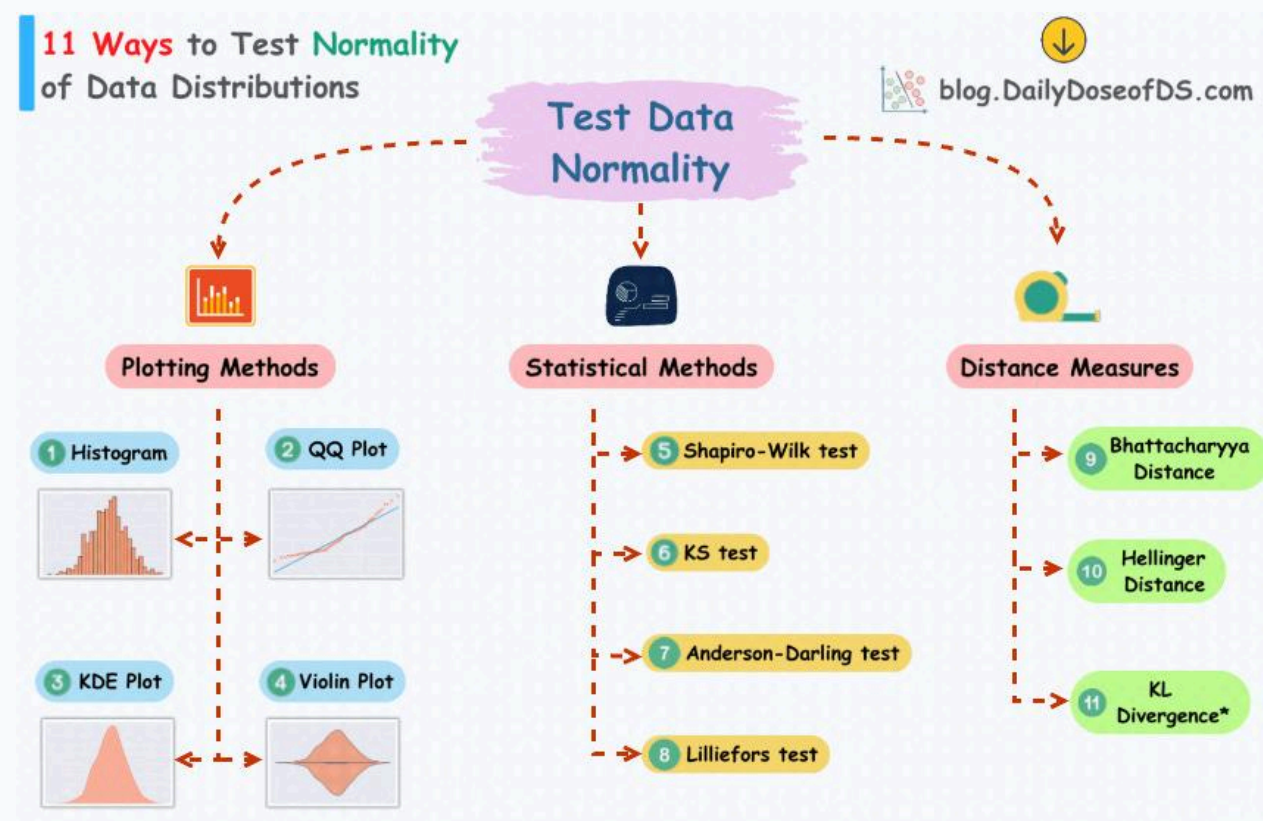
11 Ways to Determine Data Normality

Many ML models assume (or work better) under the presence of normal distribution.

For instance:

- Linear regression assumes residuals are normally distributed.
- At times, transforming the data to normal distribution can be beneficial.
- Linear discriminant analysis (LDA) is derived under the assumption of normal distribution, etc.

Thus, being aware of the ways to test normality is extremely crucial for data scientists. The visual below depicts the 11 essential ways to test normality.



Let's understand these in this chapter.

#1) Plotting Methods (self-explanatory)

- Histogram
- QQ Plot (We shall cover it in the plotting section of this book).
- KDE Plot
- Violin Plot

While plotting is often reliable, it is a subjective approach and prone to errors.

Thus, we must know reliable quantitative measures as well.

#2) Statistical Methods:

1) Shapiro-Wilk test:

- Finds a statistic using the correlation between the observed data and the expected values under a normal distribution.
- The p-value indicates the likelihood of observing such a correlation if the data were normally distributed.
- A high p-value indicates a normal distribution.

2) KS test:

- Measures the max difference between the cumulative distribution functions (CDF) of observed and normal distribution.
- The output statistic is based on the max difference between the two CDFs.
- A high p-value indicates a normal distribution.

3) Anderson-Darling test:

- Measures the differences between the observed data and the expected values under a normal distribution.
- Emphasizes the differences in the tail of the distribution.
- This makes it particularly effective at detecting deviations in the extreme values.

4) Lilliefors test:

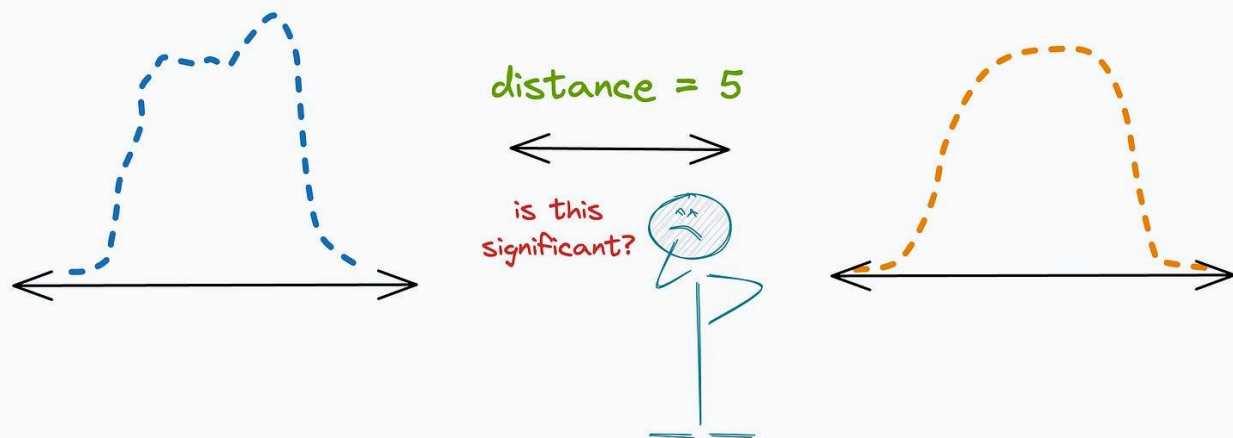
- It is a modification of the KS test.
- The KS test is appropriate in situations where the parameters of the reference distribution are known.
- If the parameters are unknown, Lilliefors is recommended.
- Get started: [Statsmodel Docs](#).

#3) Distance Measures

Distance measures are another reliable and more intuitive way to test normality.

But they can be a bit tricky to use.

See, the problem is that a single distance value needs more context for interpretability.



For instance, if the distance between two distributions is 5, is this large or small?

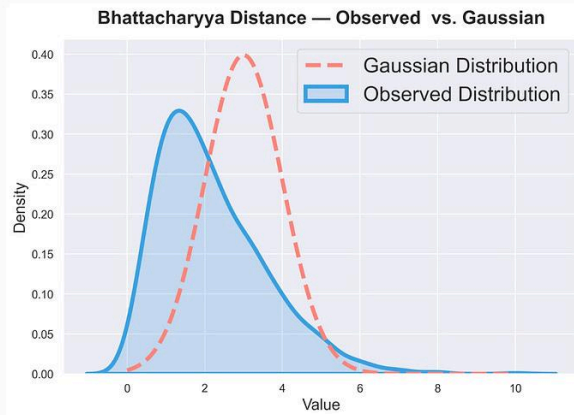
We need more context.

I prefer using these measures as follows:

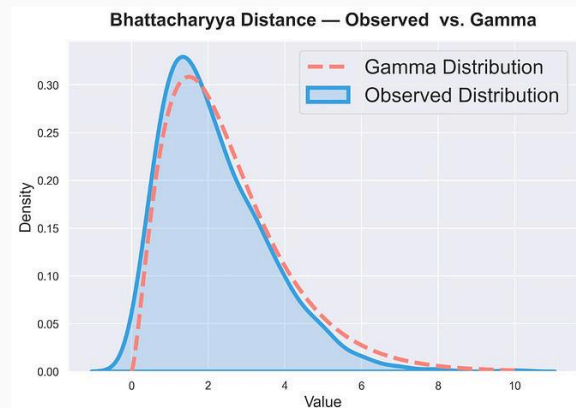
- Find the distance between the observed distribution and multiple reference distributions.
- Select the reference distribution with the minimum distance to the observed distribution.

Here are a few distance common and useful measures:

1) Bhattacharyya distance:



Bhattacharyya distance 0.19



Bhattacharyya distance 0.03

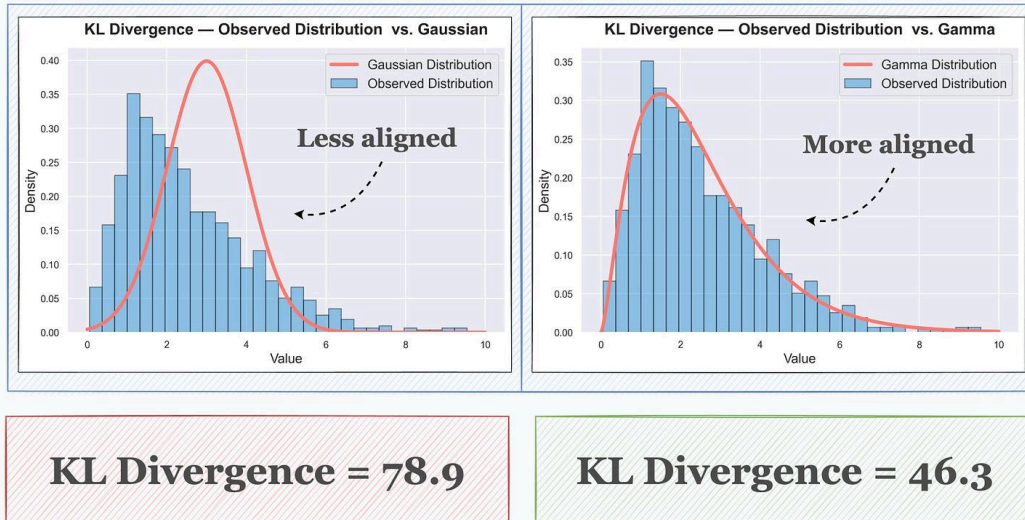
- Measure the overlap between two distributions.
- This “overlap” is often interpreted as closeness between two distributions.
- Choose the distribution that has the least Bhattacharyya distance to the observed distribution.

2) Hellinger distance:

- It is used quite similar to how we use the Bhattacharyya distance
- The difference is that Bhattacharyya distance does not satisfy triangular inequality.
- But Hellinger distance does.

3) KL Divergence:

KL Divergence



- It is not entirely a "distance metric" per se, but can be used in this case.
- Measure information lost when one distribution is approximated using another distribution.
- The more information is lost, the more the KL Divergence.
- Choose the distribution that has the least KL divergence from the observed distribution.

KL divergence is used as a loss function in the t-SNE algorithm.

Probability vs. Likelihood

In data science and statistics, many folks often use “probability” and “likelihood” interchangeably.

However, likelihood and probability DO NOT convey the same meaning.

And the misunderstanding is somewhat understandable, given that they carry similar meanings in our regular language.

While writing this chapter, I searched for their meaning in the Cambridge Dictionary. Here’s what it says:

- Probability: the level of possibility of something happening or being true.
- Likelihood: the chance that something will happen.

If you notice closely, “likelihood” is the only synonym of “probability”.

The screenshot shows the Cambridge Dictionary entry for the word "probability". At the top, the word "probability" is displayed in a large, bold font. Below it, the word is identified as a "noun" with the notation "[C or U]". Pronunciation guides for UK and US are provided. A yellow button labeled "Add to word list" is visible on the right. The definition is given as "the level of possibility of something happening or being true:". Below the definition, four example sentences are listed, each with a bullet point. At the bottom left, a "Synonym" box contains the word "likelihood", which is highlighted with a blue border and a blue arrow pointing to it from the right.

probability
 noun [C or U]
 UK /ˌprob.əˈbil.ə.ti/ US /ˌproʊ.bəˈbil.ə.ti/

C1
 the level of possibility of something happening or being true:

- What is the probability **of** winning?
- The probability **of** getting all the answers correct is about one in ten.
- **There's a high/strong probability (that)** (= it is very likely that) she'll be here.
- Until yesterday, the project was just a possibility, but now it has become a real probability (= it is likely to happen).

Synonym
 likelihood

Anyway.

In my opinion, it is crucial to understand that probability and likelihood convey very different meanings in data science and statistics.

Let’s understand!

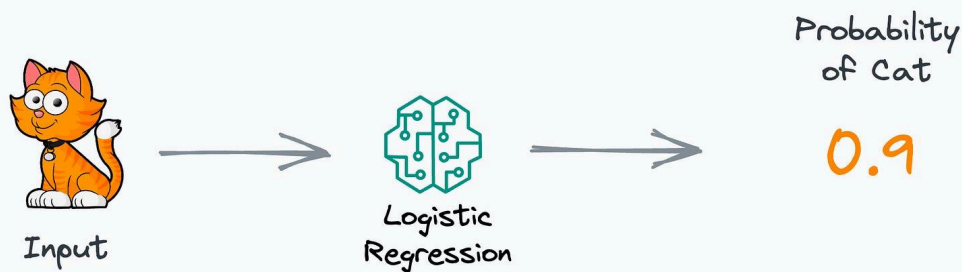
Probability is used in contexts where you wish to know the possibility/odds of an event.

For instance, what is the:

- Probability of obtaining an even number in a die roll?
- Probability of drawing an ace of diamonds from a deck?
- and so on...

When translated to ML, probability can be thought of as:

- What is the probability that a transaction is fraud?
- What is the probability that an image depicts a cat?
- and so on...



Essentially, many classification models, like logistic regression or a classification neural network, etc., assign the probability of a specific label to an input.

When calculating probability, the model's parameters are known. Also, we assume that they are trustworthy.

For instance, to determine the probability of a head in a coin toss, we mostly assume and trust that it is a fair coin.

Likelihood, on the other hand, is about explaining events that have already occurred.

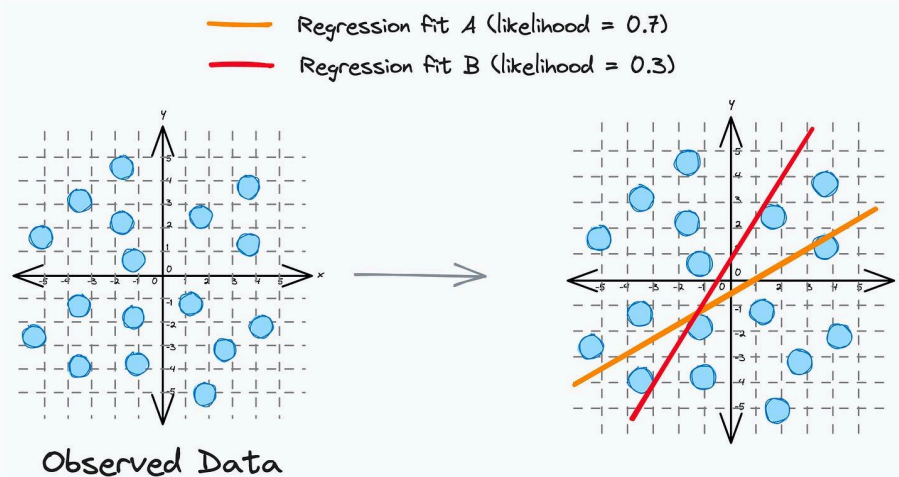
Unlike probability (where parameters are known and assumed to be trustworthy)...

...likelihood helps us determine if we can trust the parameters in a model based on the observed data.

Let me elaborate more on that.

Assume you have collected some 2D data and wish to fit a straight line with two parameters — slope (m) and intercept (c).

Here, likelihood is defined as the support provided by a data point for some particular parameter values in your model.

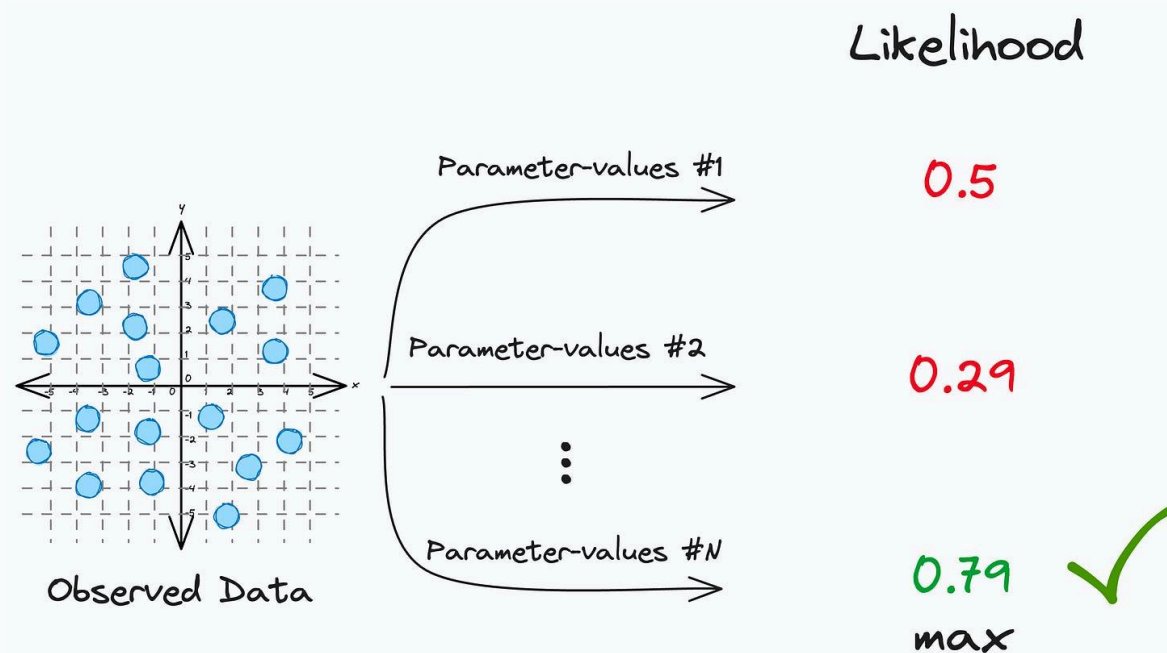


Here, you will ask questions like:

- If I model this data with the parameters:
 - $m=2$ and $c=1$, what is the likelihood of observing the data?
 - $m=3$ and $c=2$, what is the likelihood of observing the data?
 - and so on...

The above formulation popularly translates into the maximum likelihood estimation (MLE), which we discussed here: [\(MLE vs. EM — What's the Difference?\)](#)

In maximum likelihood estimation, you have some observed data and you are trying to determine the specific set of parameters (θ) that maximize the likelihood of observing the data.



Using the term “likelihood” is like:

- I have a possible explanation for my data. (In the above illustration, “explanation” can be thought of as the parameters you are trying to determine)
- How well does my explanation explain what I’ve already observed? This is precisely quantified with likelihood.

For instance:

- Observation: The outcomes of 10 coin tosses are “HHHHHHHHHTHH”.
- Explanation: I think it is a fair coin ($p=0.5$).
- What is the likelihood that my above explanation is true based on the observed data?

To summarize...

It is immensely important to understand that in data science and statistics, likelihood and probability DO NOT convey the same meaning.

As explained above, they are pretty different.

In probability:

- We determine the possibility of an event.
- We know the parameters associated with the event and assume them to be trustworthy.

In likelihood:

- We have some observations.
- We have an explanation (or parameters).
- Likelihood helps us quantify whether the explanation is trustworthy.

Hope that helped!

11 Key Probability Distributions in Data Science

Statistical models assume an underlying data generation process.

This is exactly what lets us formulate the generation process, using which we define the maximum likelihood estimation (MLE) step.



Thus, when dealing with statistical models, the model performance becomes entirely dependent on:

- Your understanding of the data generation process.
- The distribution you chose to model data with, which, in turn, depends on how well you understand various distributions.

We shall also look at this in the chapter on GLMs ([this chapter](#)).

Thus, it is crucial to be aware of some of the most important distributions and the type of data they can model.

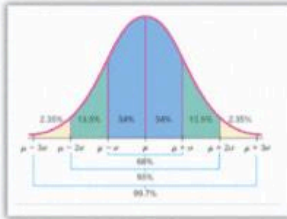
The visual below depicts the 11 most important distributions in data science:

11 Most Important Distributions in Data Science

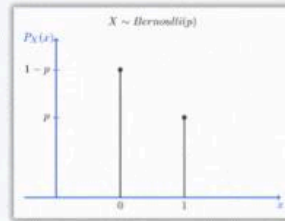


blog.DailyDoseofDS.com

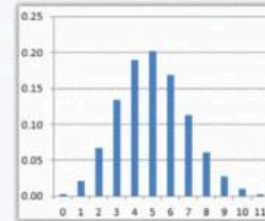
1 Normal Distribution



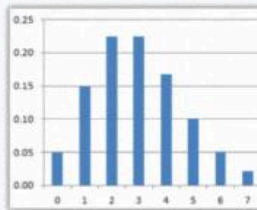
2 Bernoulli Distribution



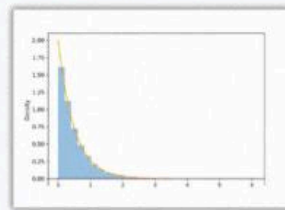
3 Binomial Distribution



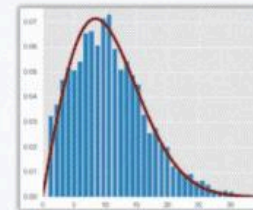
4 Poisson Distribution



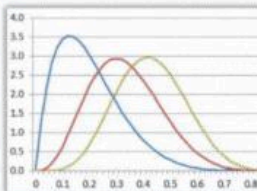
5 Exponential Distribution



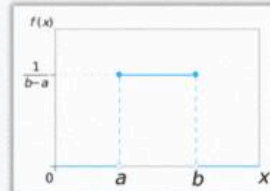
6 Gamma Distribution



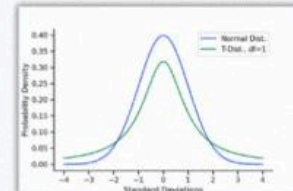
7 Beta Distribution



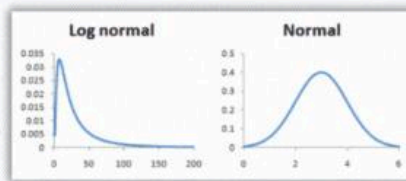
8 Uniform Distribution



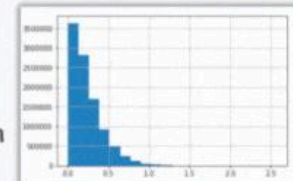
9 Student t-distribution



10 Log Normal Distribution



11 Weibull Distribution



Let's understand them briefly and how they are used.

Normal Distribution

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

- The most widely used distribution in data science.
- Characterized by a symmetric bell-shaped curve
- It is parameterized by two parameters—mean and standard deviation.
- Example: Height of individuals.

Bernoulli Distribution

$$P(X = x) = \begin{cases} p & \text{for } x = 1 \\ 1 - p & \text{for } x = 0 \end{cases}$$

- A discrete probability distribution that models the outcome of a binary event.
- It is parameterized by one parameter—the probability of success.
- Example: Modeling the outcome of a single coin flip.

Binomial Distribution

$$P(X = x) = \frac{n!}{x!(n-x)!} p^x (1 - p)^{(n-x)}$$

- It is Bernoulli distribution repeated multiple times.
- A discrete probability distribution that represents the number of successes in a fixed number of independent Bernoulli trials.
- It is parameterized by two parameters—the number of trials and the probability of success.